TdxQuant 简介

TdxQuant是由深圳市财富趋势科技股份有限公司研发的专业量化投研平台，专注于为国内量化投资者提供从策略研究到投资决策的全流程解决方案。平台以高效、简洁为核心设计理念，致力于降低量化交易门槛，提升策略开发与执行的效率。

依托通达信近三十余年在金融科技领域的深厚积累，TdxQuant集成了完备的实时和历史行情数据、金融数据库及稳定的交易系统基础设施，为策略的研发、回测、验证和执行提供了坚实可靠的技术支持。

平台采用分层化、模块化的服务体系，可灵活适配从高校学生、独立研究者、个人投资者到专业机构等不同用户的需求，实现从策略构思到交易落地的无缝衔接。

TdxQuant 服务介绍

TdxQuant 是一套基于通达信金融终端构建的 Python 量化策略运行框架。该框架通过 API 接口形式，为策略交易提供所需的行情数据获取与交易指令执行功能。

运行环境要求

TdxQuant 支持 64 位 Python 3.7、3.8、3.9、3.10、3.11、3.12、3.13、3.14等版本，系统会自动适配当前 Python 版本，建议使用3.13版本。
请注意：运行 TdxQuant 程序前，需预先启动支持TQ策略功能的 **通达信金融终端、通达信专业研究、量化模拟版或期货通等版本**。

核心运行逻辑

TdxQuant 以 **tqcenter** 行情模块为核心，专注于为量化交易者提供高效、直接的数据服务，主要包含以下内容：

- **行情数据**：实时与历史的快照、K 线、分笔（Tick）数据
- **基本面数据**：除权除息、基本财务、专业财务、股票交易数据、市场数据等
- **新股和合约等信息**：标的基础信息、可转债、新股申购等
- **分类数据**：市场类型、行业分类、自定义板块等

核心应用场景

TdxQuant提供覆盖量化投研全流程的核心功能模块，主要应用场景包括：

1. 策略研发与历史回测

平台提供“即用型”标准化数据。所有历史与实时数据均在服务端完成清洗、对齐，并预加载至客户端。支持用户快速获取指定时间维度的历史数据，并进行策略信号计算与回测分析。既提供复权因子，也提供各种类型的复权后的数据。

2. 实时监控与信号预警

支持实时行情数据订阅，用户可基于自定义的指标与因子模型进行在线计算。当预设条件触发时，系统通过信号接口实时推送预警信息至客户端，助力研究者及时捕捉市场动态与交易机会。

3. 交易模拟与实盘执行

平台构建了完整的策略交易闭环，提供模拟交易、券商实盘等两种执行环境：

- **模拟交易**：在仿真市场环境中，使用实时行情数据对策略进行持续跟踪与验证，评估其实际表现，全程无资金风险。
- **实盘交易**：通过稳定的交易总线，安全对接券商报盘系统，实现策略信号的自动化、高可靠性下单与交易管理。

量化交易的核心价值

1. 利用历史数据高效验证策略，提升研究效率数百倍

在验证交易策略时，历史回测是评估其有效性的关键环节，但传统人工方式难以处理海量数据与复杂计算。量化交易可在几分钟内完成一次全面回测，快速获得统计验证结果，极大提升了策略研发的迭代效率。

2. 实时捕捉基于概率的获胜机会

量化交易借助计算机强大的数据处理能力，能够从海量市场信息中发掘人工难以察觉的规律与机会。面对全市场数千只股票的实时波动，量化系统可同时监控多重条件，避免机会错失。它能够综合考量选股、择时、资产配置与风险管理，构建并执行具有较大概率的投资组合，追求收益最大化。

3. 实现科学、客观的投资决策

与传统主观投资不同，量化交易将投资理念、经验甚至市场直觉转化为严谨的数学模型。通过系统化的信号生成与执行机制，有效克服人性中的情绪偏差，使投资决策过程更具纪律性、可重复性与可优化性。

量化交易的工具挑战

工欲善其事，必先利其器。 对于个人投资者而言，独立搭建一套完整的量化交易体系，复杂繁琐，涉及数据、系统、策略等多层面的巨大投入。

一、需要准确、全面的金融数据基础

量化交易依赖于高质量的历史与实时数据，包括行情、财务、宏观及基本面数据等。构建和维护这样一个数据仓库，不仅需要持续的数据采购、清洗、更新与运维成本，还需在数据存储、访问速度

与系统稳定性方面进行深入的技术投入。

二、需要易用、可靠的量化交易系统

一个成熟的量化平台需要支持多样的策略开发语言、具备高速的回测与模拟引擎、提供科学的策略评估体系，并为实盘交易提供全方位的保障。过往，研究者往往需要兼具复杂的金融数据知识与工程构建能力。如今，TdxQuant让您只需专注于策略逻辑本身，其余复杂工作交给平台。

TdxQuant的核心优势

TdxQuant是一款集金融数据与策略投研工具于一体的量化平台，结构清晰，简洁易上手，数据获取快捷，算法资源丰富。我们的目标是为投资者提供"开箱即用"的完整解决方案。

1. 全方位保障策略安全与自主

- 支持策略在本地IDE环境中开发与运行，保障代码安全与私密性
- 分离式模块化架构，策略的编码和调试更加自由和灵活

2. 大幅降低量化交易门槛

- 提供高质量、高精度、快速接入的金融元数据
- 支持多种策略类型的便捷编写、回测、模拟与实盘

3. 助力构建专业量化成长路径

- 通过"投资学院"系统学习量化交易相关知识体系
- 通过"宽客社区"交流心得、解答疑惑
- 全程助力用户从入门到精通，成为专业的量化投资者

#版本更新说明

📋 更新日志

▼ 🗓️ 2026-06-19 更新说明(内测版)

- 1.支持直接调用计数
- 2.自动交易函数增加返回错误
- 3.download_file支持经营分析数据文件
- 4.部分函数的完善

▼ 🗓️ 2026-06-12 更新说明

- 1.修正撤单等细节,支持交易弹框中的市价显示
- 2.get_match_stkinfo支持全部品种
- 3.AI平台中,支持通达信TQ-Python和通达信TQ-Local两种Skill调用

▼ 🗓️ 2026-06-05 更新说明

- 1.支持批量内部运行，断开连接，删除策略，打开编辑、消息筛选
- 2.修复了内部启动处理带有空格的路径时会截断的问题，优化断开连接细节
- 3.底层逻辑优化，提升数据返回速度
- 4.新增接口：检索证券信息get_match_stkinfo
- 5.全面支持AI平台直接调用TQ数据(无需Python代码)

▼ 🗓️ 2026-05-22 更新说明

- 1.新增函数：get_pricevol 批量获取价格和成交量
- 2.新增函数：formula_get_all 获取对应种类的全部公式名称和代码
- 3.新增函数：formula_get_info 获取公式相关信息
- 4.新增函数：formula_process_mul_exp
- 5.新增逻辑：formula_process_mul系列count=-1时，可以根据start_time和end_time返回结果

▼ 🗓️ 2026-05-15 更新说明

- 1.策略管理器新增“新建文件”按钮，并优化相应体验
- 2.断开连接功能优化，可以主动退出策略
- 3.修复get_market_data获取数据时如果传入的代码包含尚未上市代码会报错的问题
- 4.新增get_gb_info_by_date，可以仅根据start和end获取指定时间段的股本

▼ 🗓️ 2026-05-09 更新说明

- 1.支持信用还券等下单类型
- 2.加入内部编辑器

▼ 🗓️ 2026-04-16 更新说明

- 1.下单函数支持期货期权等类型

- 2.支持多头持仓,期货动态权益等持仓资金栏目
- 3.增加.QHZ期货指数类型
- 4.支持880096,880097,880098等成份股

▼ 🗓️ **2026-03-27 更新说明**

- 新增函数：获取股票所属板块get_relation
- 新增函数：调用客户端功能接口exec_to_tdx
- 新增函数：撤单cancel_order_stock
- 新增函数：账户资产查询query_stock_asset
- 更新函数：交易类账户函数逻辑更新
- 更新函数：调用通达信公式返回值字段名由"Data"改为"Value"
- 更新函数：order_stock对于模拟账户自动下单
- 更新函数：order_stock新增信用交易：担保品买入、担保品卖出，融资买入，融券卖出
- 更新函数：get_stock_list_in_sector访问空的自定义板块会返回空集而不是报错
- 问题修复：修复了get_market_data、refresh_kline等函数无法处理期权的问题
- 其他更新：期货期权类型支持，新增相关宏定义（常量枚举）

▼ 🗓️ **2026-03-20 更新说明**

- 新增函数：获取资金账户句柄stock_account
- 新增函数：查询账户委托信息query_stock_orders
- 新增函数：查询账户持仓信息query_stock_positions
- 新增函数：交易执行函数order_stock
- 更新函数：get_stock_list_in_sector新增block_type=2，可取对应期货代码
- 更新函数：get_more_info新增字段QHMainYYMM
- 更新函数：get_stock_list新增参数92: 国内期货主力合约
- 更新函数：get_cb_info改名为get_kzz_info

▼ 🗓️ **2026-03-06 更新说明**

- 新增函数：获取跟踪指数的ETF信息get_trackzs_etf_info
- 更新函数：refresh_cache新增参数 'ZS' 表示沪深京指数
- 更新函数：get_stock_list新增参数91 跟踪指数的ETF信息
- 其他修正：未识别的市场后缀由默认的SZ改为OT
- 其他修正：修复get_market_data某些情况下会报NoneType的bug

▼ 🗓️ **2026-02-28 更新说明**

- 问题修复：修复了formula_process_mul_zb等入参retrun_count拼写错误问题
- 更新函数：get_more_info，get_cb_info，get_market_snapshot加上了字段筛选功能
- 更新函数：get_more_info等支持更多行情数据项，输出顺序进行归整
- 其他修正：tqcenter几处细节修改

▼ 🗓️ **2026-02-12 更新说明**

- 更新函数：send_user_block可以添加股票进自选股，自选股简称为ZXG
- 其他更新：批量调用公式内部优化提速
- 其他更新：新增港股指数（.HI）
- 其他更新：解决多个客户端同时运行时的TQ冲突的问题

▼ 🗓️ **2026-02-07 更新说明**

- 新增函数：批量调用选股公式formula_process_mul_xg
- 新增函数：批量调用指标公式formula_process_mul_zb
- 更新函数：get_stock_list、 get_sector_list、 get_stock_list_in_sector新增参数list_type，可以选择返回股票名称
- 更新函数：tdx_formula返回做出修改，条件选股和专家选股只返回'1'和'0'
- 更新函数：formula_zb新增参数xsflag，可以设置返回数据的小数位数
- 更新函数：download_file新增下载：最近舆情、综合信息文件
- 更新函数：get_stock_info新增部分数据字段输出

▼ 📅 **2026-01-31 更新说明**

- 新增功能：支持调用通达信公式进行计算
- 新增函数：格式化K线数据formula_format_data
- 新增函数：向通达信公式系统设置数据formula_set_data
- 新增函数：向通达信公式系统设置数据信息formula_set_data_info
- 新增函数：获取公式中的设置数据formula_get_data
- 新增函数：调用通达信技术指标公式formula_zb
- 新增函数：调用通达信条件选股公式formula_xg
- 新增函数：调用通达信专家系统公式formula_exp
- 新增函数：获取股票更多信息get_more_info
- 新增函数：获取每天的股本数据get_gb_info
- 更新函数：刷新行情缓存refresh_cache，新增参数force和market，可指定强制刷新或指定市场刷新
- 其他更新：新增中证指数（.CSI），中金所期货（.CFF），宏观数据（.HG）等市场后缀识别和数据获取
- 其他更新：获取非指定日期的股票交易数据，板块交易数据等数据时增加了对应日期返回。
- 问题修复：修复了部分市场数据返回时小数位数不对导致的精度问题。
- 问题修复：修复了获取Python3.9以及之前版本依赖库错误问题。

📅 **2026-01-17 正式发布**

安装Python及开发环境 →

安装Python及VSCode等开发环境

1.安装 Python 环境

安装Python：建议使用Python3.7及以上版本

1.1 下载地址：Python官网

特别提示：安装时候务必勾选Add Python to PATH（将Python添加到环境变量）

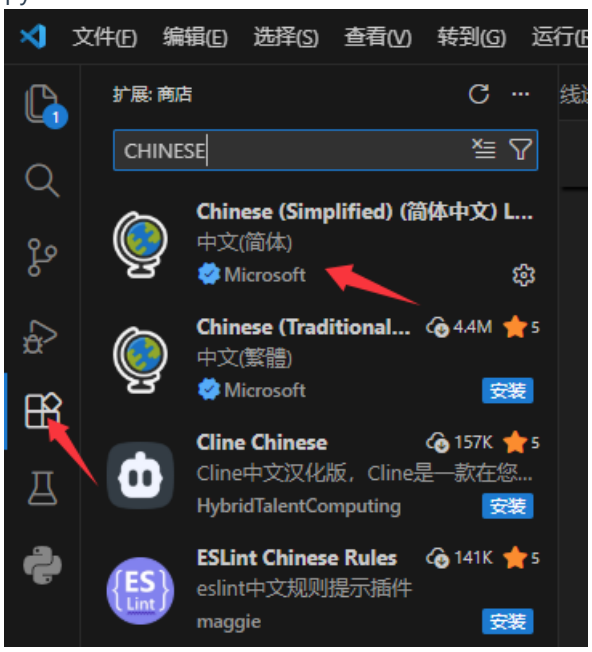
2.安装IDE 建议VSCode、PyCharm或Trae

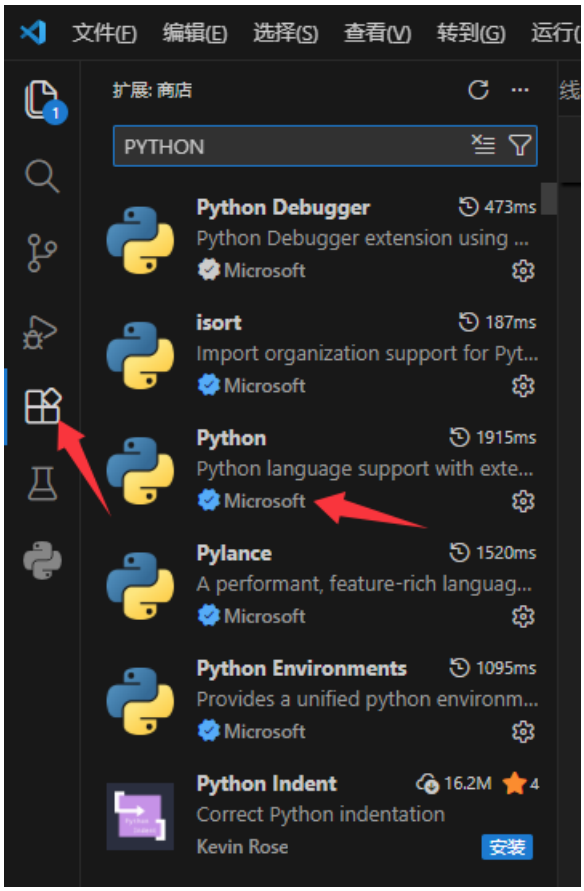
2.1 下载地址：Visual Studio Code官网

2.2 安装 Python 插件（Extensions）

VSCode安装好后，在VSCode终端-扩展-输入下文，分别添加相关扩展：

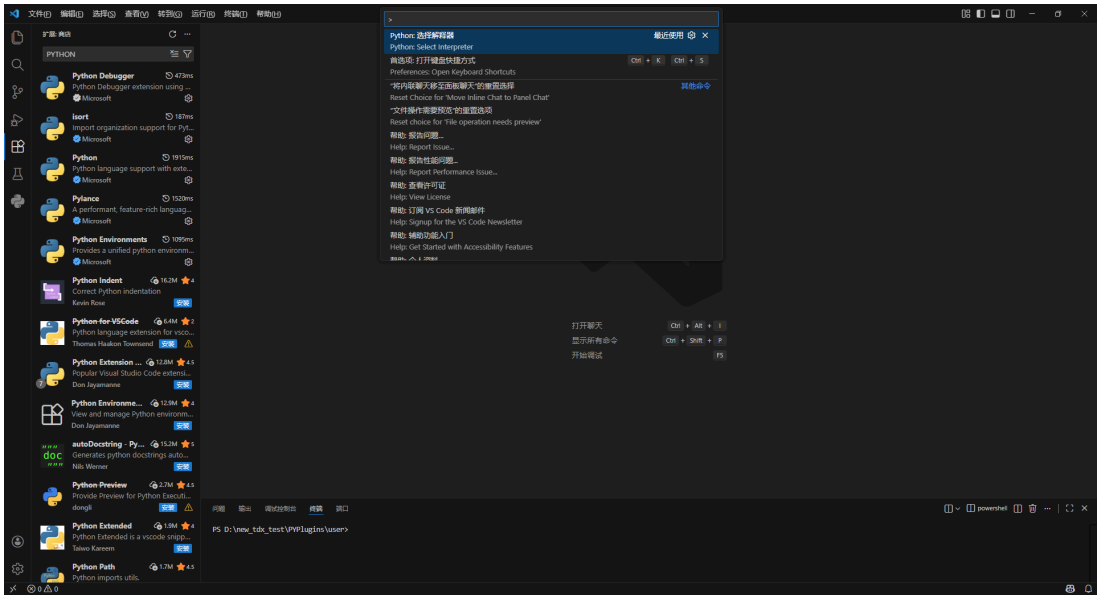
- 简体中文
- python





2.3 选择Python解释器：选择python3.13安装路径的exe

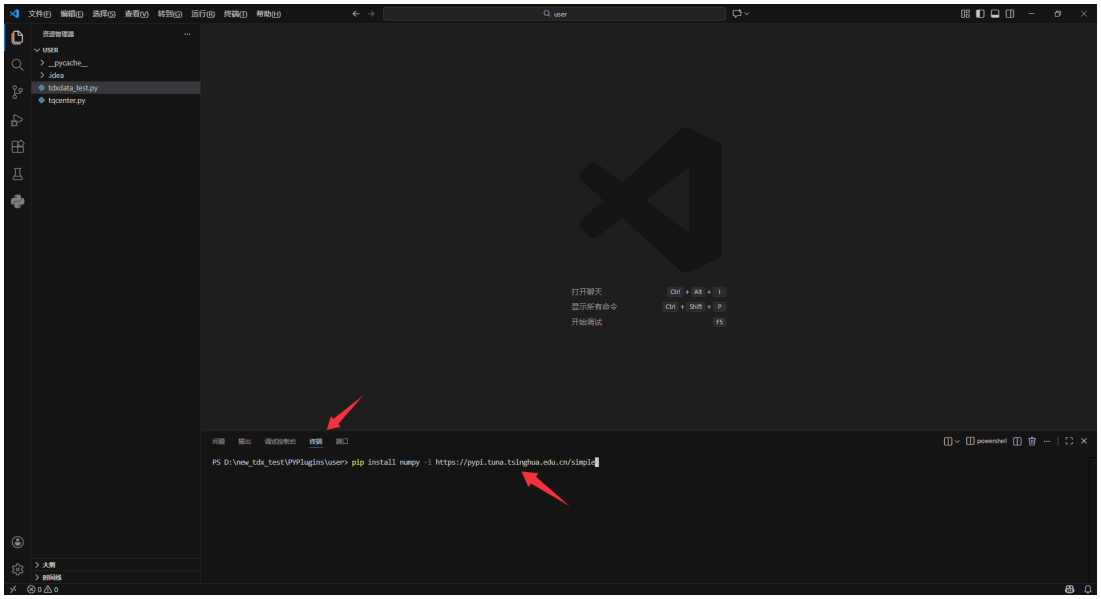
- 使用 Ctrl+Shift+P 快捷键打开 command palette 窗口
- 输入关键字 `python select` 并找到 `Python: Select Interpreter` 一项， 点击该项并在随后弹出的 Python 解释器列表中选择目标解释器：



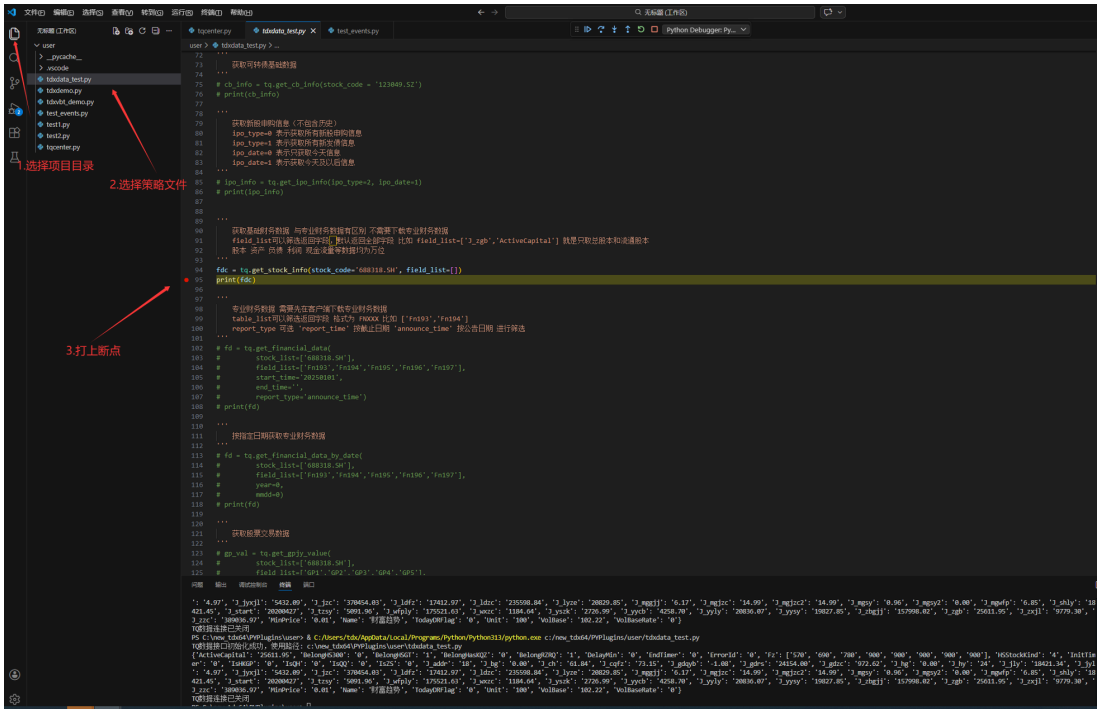
2.4 在VSCode终端-扩展-分别输入下文，常用库建议安装：

- pip install numpy -i https://pypi.tuna.tsinghua.edu.cn/simple
- pip install pandas -i https://pypi.tuna.tsinghua.edu.cn/simple

- pip install backtrader -i https://pypi.tuna.tsinghua.edu.cn/simple
- pip install vectorbt -i https://pypi.tuna.tsinghua.edu.cn/simple
- pip install ta-lib -i https://pypi.tuna.tsinghua.edu.cn/simple
- pip install matplotlib -i https://pypi.tuna.tsinghua.edu.cn/simple

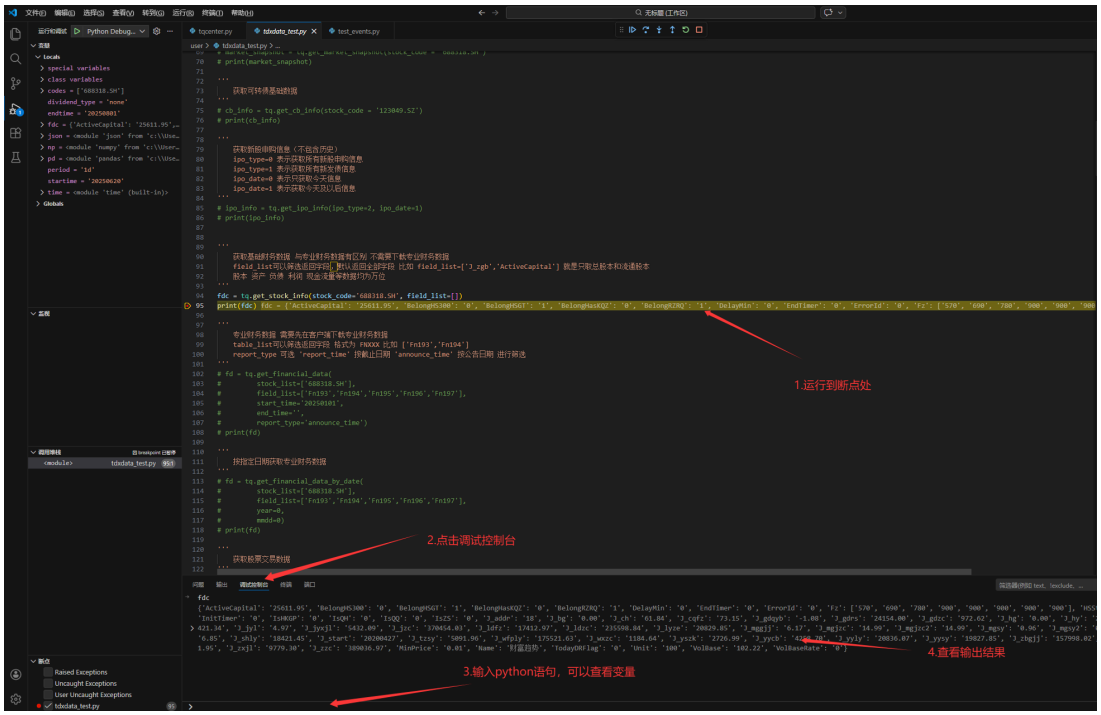


- 在 VSCode 中打开要调试的文件（如 tdxdemo.py）
- 在代码行号左侧单击，出现红点即表示断点已设置。
- 选择调试配置：点击左侧活动栏的"运行和调试"图标（或按 Ctrl+Shift+D),选择并启动调试配置（调试器类型选择 "Python Debugger" ）
- 自动生成配置：完成以上步骤后，VSCode会自动在项目根目录创建一个 .vscode 文件夹，并在里面生成 launch.json 文件.同时，调试下拉菜单就会出现，默认选中了"Python 文件"这个配置。
- 启动调试：按 F5 或点击绿色的"开始调试"按钮。



- 显示选择调试配置-" Python文件 "，调试打开的 Python 文件。

- 可以查看变量等等。



3.用户py的文件位置

在策略管理器界面，点击[文件位置]。
用户的py文件一般在客户端PYPlugins下面的user目录下面。py运行过程的生成的文件一般在PYPlugins下面的data和file目录下。

← 版本更新说明

安装通达信终端并获取数据 →

1. 安装通达信终端

1.1 下载地址

内测版下载入口： 通达信金融终端内测版
量化模拟下载入口： 金融终端(量化模拟)
正式版下载入口： 通达信金融终端、通达信专业研究版、期货通

1.2 登录通达信客户端

1.3 系统-盘后数据下载

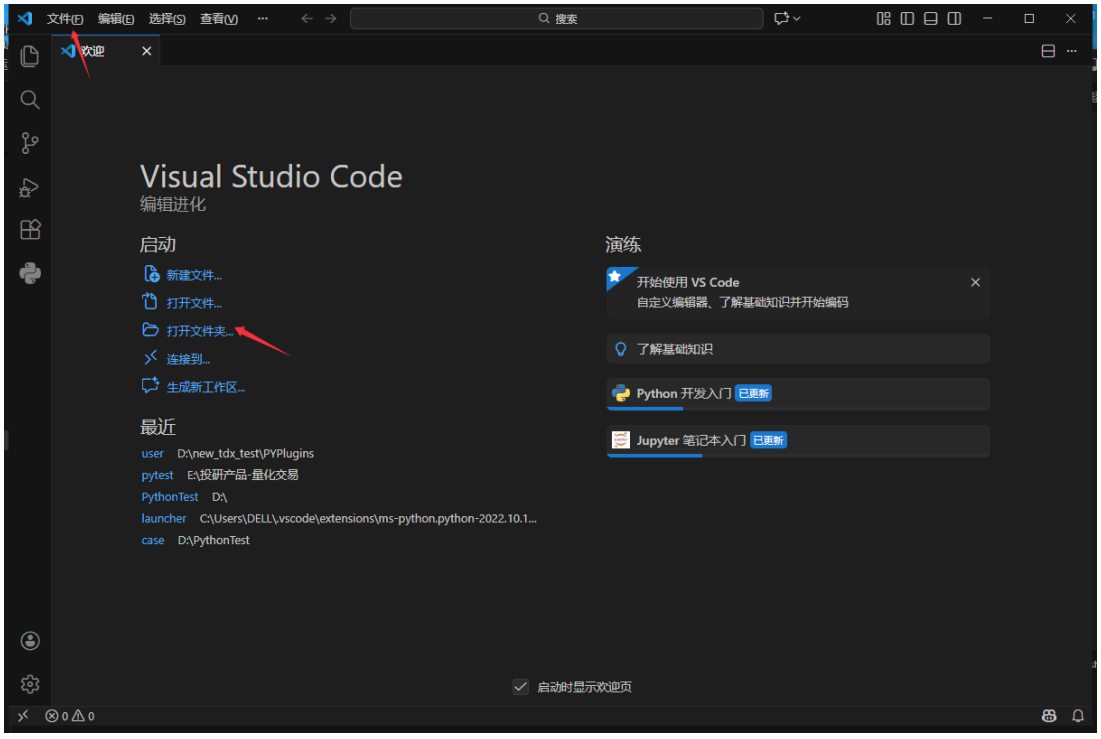
进行日线和分钟线等数据下载

2. 使用VSCode集成环境

2.1 使用VSCode运行py

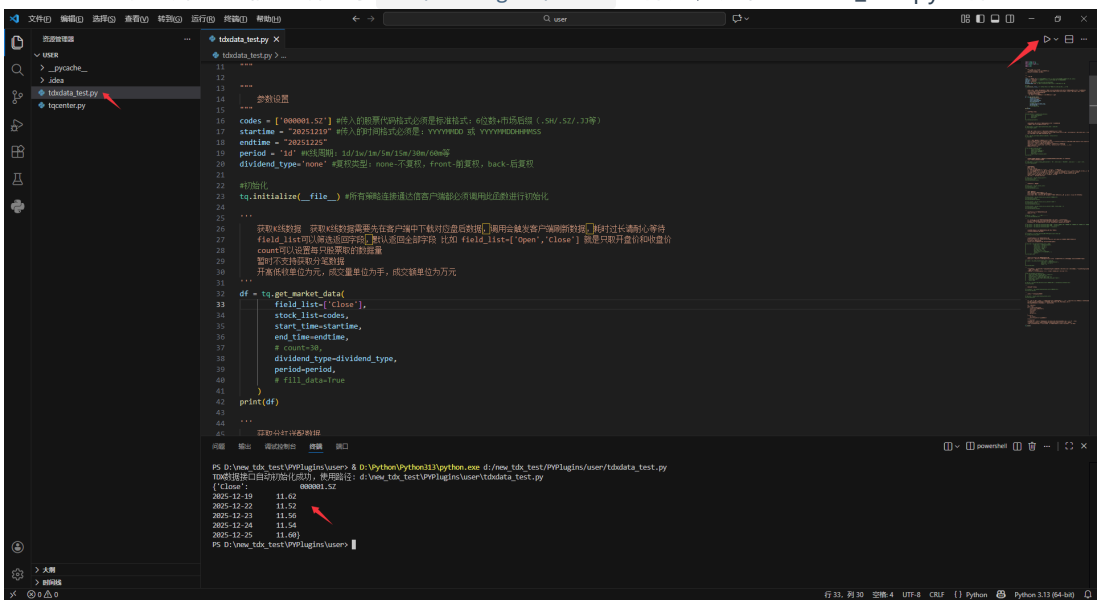
2.1.1 打开py文件

- 在 VS Code 中点击打开一个本地文件夹，“文件”->"打开文件夹"。



2.1.2 运行py文件

- 在VSCode中打开通达信终端目录 `.../PYPlugins/user` 文件夹，运行tdxdata_test.py文件。

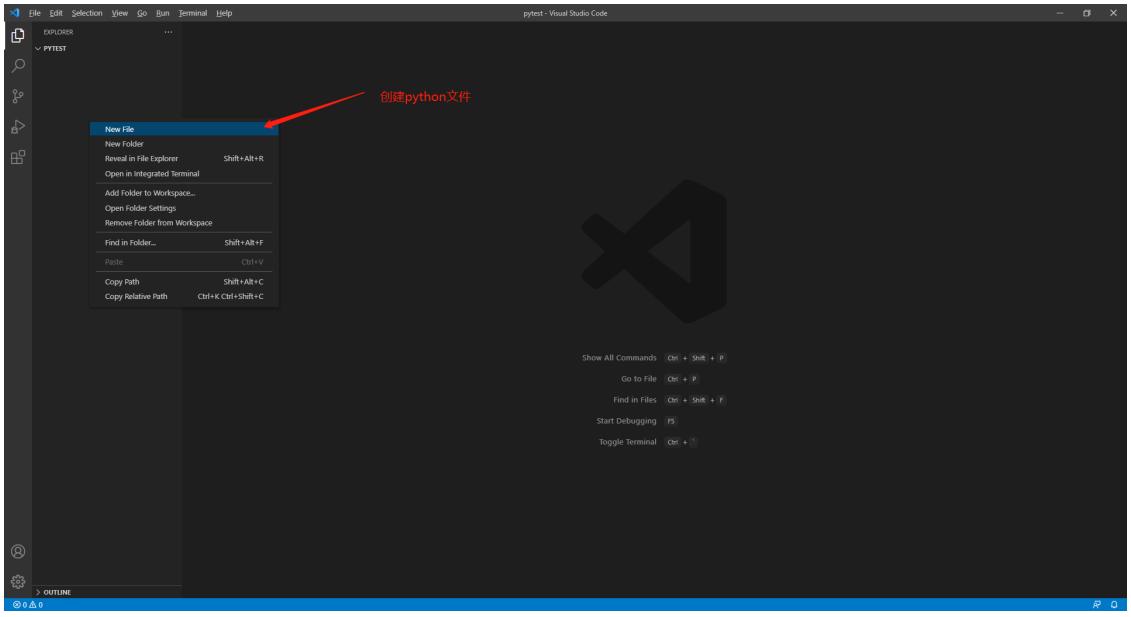


注意：客户端安装目录下面的 `.../PYPlugins/user` 文件夹中的 `tqcenter.py` 是最主要的TQData支撑文件，请勿修改或删除，否则需要重新下载。

2.2 使用VSCode编辑新文件

2.2.1 新建py文件

在打开的文件夹中鼠标右键创建新的".py" python 文件，文件名例如tdxdemo.py。

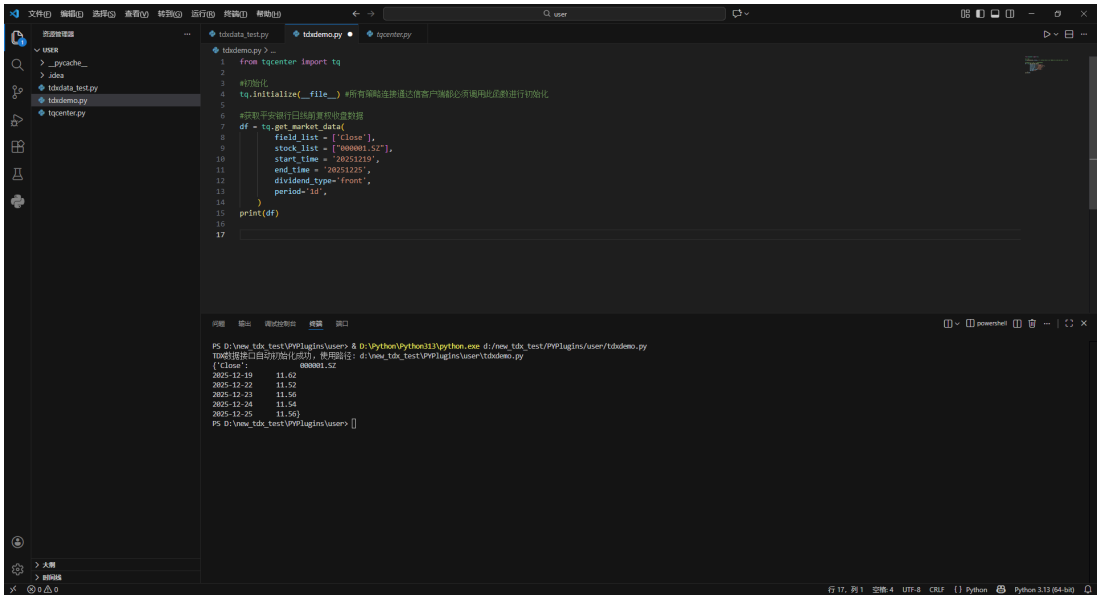


2.2.2 编辑py文件

```py

# 使用tqcenter的API函数查看平安银行日线数据示例
from tqcenter import tq

#初始化
tq.initialize(__file__) #所有策略连接通达信客户端都必须调用此函数进行初始化
```

```
7
8   #获取平安银行日线前复权收盘数据
9   df = tq.get_market_data(
10          field_list = ['Close'],
11          stock_list = ["000001.SZ"],
12          start_time = '20251219',
13          end_time = '20251225',
14          dividend_type='front',
15          period='1d',
16      )
17  print(df)
18
```

- 运行结果如图：



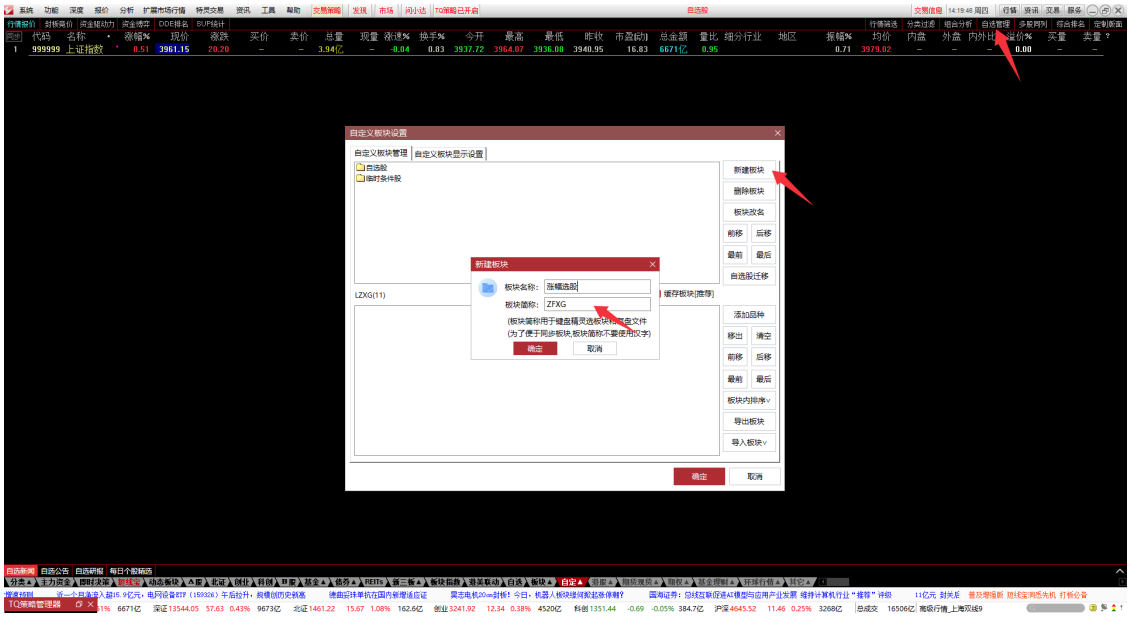
← 安装Python及开发环境

快速开始第一个策略 →

步骤分解

一个完整选股入自定义板块策略只需要两步:

1.客户端新增自定义板块



2.在VSCode里面运行以下python代码

实现运行函数：在这个策略里, 我们会根据运行结果做出相应操作:

```py
# 策略说明：如果运行时间点价格高出昨收5%，则进入涨幅选股板块，否则清空该板块
import pandas as pd
import numpy as np
from datetime import datetime
from tqcenter import tq

# 初始化tq
tq.initialize(__file__)

# 1. 基础配置
batch_codes = tq.get_stock_list_in_sector('通达信88')      # 目标板块
start_time = "20251025"                                    # 数据起始日期
target_end = datetime.now().strftime("%Y%m%d")             # 数据结束日期（当前日期）
target_gain = 5.0                                          # 目标涨幅（%），可修改
target_block_name = 'ZFXG'                                 # 目标自定义板块简称

# 2. 获取并整理收盘价数据
df_real = tq.get_market_data(
```

```
    field_list=['Close'],
    stock_list=batch_codes,
    start_time=start_time,
    end_time=target_end,
    dividend_type='front',  # 前复权
    period='1d',            # 日线
    fill_data=True          # 填充缺失数据
)
# 转换为「日期×股票代码」的收盘价宽表
close_df = tq.price_df(df_real, 'Close', column_names=batch_codes)

# 3. 核心：计算当日相较于昨日的涨幅（%）
# 昨日收盘价（向下平移1行）
prev_close = close_df.shift(1)
# 计算涨幅：(当日收盘价 - 昨日收盘价) / 昨日收盘价 × 100%
daily_gain = (close_df - prev_close) / prev_close * 100

# 4. 筛选符合条件的股票（最新交易日涨幅超target_gain%）
latest_date = daily_gain.index[-1]              # 最新交易日
latest_daily_gain = daily_gain.loc[latest_date] # 每只股票最新交易日的涨幅
# 筛选条件：涨幅 > target_gain%（排除NaN，避免数据异常）
target_stocks = latest_daily_gain[latest_daily_gain >
target_gain].sort_values(ascending=False)
target_stocks_list = target_stocks.index.tolist()  # 提取符合条件的股票代码列表

# 5. 结果输出与自定义板块操作（可按需注释）
print(f"\n=== 筛选结果（当日涨幅＞{target_gain}%）===")
if not target_stocks.empty:
    # ==================== 模块1：打印筛选结果 ====================
    print("【模块1：打印筛选结果】")
    print(f"符合条件的股票共 {len(target_stocks)} 只：")
    print(f"{'股票代码':<12} {'昨日收盘价':<12} {'当日收盘价':<12} {'当日涨幅':
<10}")
    print("-" * 50)
    for stock_code, gain in target_stocks.items():
        prev_price = prev_close.loc[latest_date, stock_code]
        curr_price = close_df.loc[latest_date, stock_code]
        print(f"{stock_code:<12} {prev_price:<12.2f} {curr_price:<12.2f} {gain:
<.2f}%")
    print("-" * 50)

    # ==================== 模块2：添加至自定义板块 ====================
    try:
        print("【模块2：自定义板块操作】")
        tq.send_user_block(block_code=target_block_name,
stocks=target_stocks_list, show=True)
        print(f"✅ 已成功将股票添加至自定义板块「{target_block_name}」")
    except Exception as e:
        print(f"❌ 添加自定义板块失败：{e}")
    print("-" * 50)



else:
    # ==================== 模块1：打印空结果 ====================
    print("【模块1：打印筛选结果】")
```

```
        print(f"暂无当日涨幅>{target_gain}%的股票")
        print("-" * 50)

        # ==================== 模块2：清空自定义选板块 ====================
        try:
            print("【模块2：自定义板块操作】")
            tq.send_user_block(block_code=target_block_name, stocks=[],show=True)
            print(f"✅ 已清空自定义板块「{target_block_name}」")
        except Exception as e:
            print(f"❌ 清空自定义板块失败：{e}")
        print("-" * 50)
```

结果示例

VSCode端

通达信终端

← 安装通达信终端并获取数据

初始化initialize →

通用函数

数据订阅函数：包括对行情数据进行订阅/取消订阅、刷新和缓存等函数。
与客户端交互函数：包括发生消息到TQ策略管理器界面、发生信号到客户端个股界面、发送预警到客户端TQ策略信号等。
数据信息文件包：我们提供各类特定的数据信息文件包。具体见download_file。

初始化initialize

```py
initialize(__file__) #所有策略连接通达信客户端都必须调用此函数进行初始化
```

调用方法:

```py
from tqcenter import tq

tq.initialize(__file__)
```

注意事项:

1."initialize"不可修改。
2.该函数用于初始化，任何一个策略都必须有该函数。

← 快速开始第一个策略

订阅行情subscribe_hq →

订阅行情subscribe_hq

订阅股票实时更新

```py
subscribe_hq(stock_list: List[str] = [],callback = None):
```

输入参数

参数	是否必选	参数类型	参数说明
stock_list	Y	List[str]	订阅的证券代码
callback	Y	str	回调函数

- 订阅股票更新 传入回调函数，订阅的股票有更新时，系统会调用回调函数，最多订阅100条
- 回调函数格式定义为on_data(datas) datas格式为 {"Code":"XXXXXX.XX","ErrorId":"0"}

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)

# 回调函数 功能为收到更新后请求最新的report数据
def my_callback_func(data_str):
    print("Callback received data:", data_str)
    code_json = json.loads(data_str)
    print(f"codes = {code_json.get('Code')}")
    report_ptr = tq.get_report_data(code_json.get('Code'))
    print(report_ptr)
    return None

sub_hq = tq.subscribe_hq(stock_list=['688318.SH'], callback=my_callback_func)
print(sub_hq)

# 收到更新时策略需要正在运行
#while True:
#    time.sleep(1)

```

数据样本

```
{
   "Error" : "订阅688318.SH更新成功.",
   "ErrorId" : "0",
   "run_id" : "1"
}
```


← 初始化initialize

取消订阅更新unsubscribe_hq →

取消订阅更新unsubscribe_hq

取消订阅股票实时更新

```py
unsubscribe_hq(stock_list: List[str] = []):
```

输入参数

参数	是否必选	参数类型	参数说明
stock_list	Y	List[str]	证券代码

- 订阅股票更新 传入回调函数，订阅的股票有更新时，系统会调用回调函数，最多订阅100条
- 回调函数格式定义为on_data(datas) datas格式为 {"Code":"XXXXXX.XX","ErrorId":"0"}

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
un_sub_ptr = tq.unsubscribe_hq(stock_list=['688318.SH'])
print(un_sub_ptr)
```

数据样本

```
{
   "Error" : "取消全部订阅更新失败.",
   "ErrorId" : "0",
   "run_id" : "1"
}
```

← 订阅行情subscribe_hq

获得订阅列表get_subscribe_hq_stock_list →

获得订阅列表get_subscribe_hq_stock_list

获得当前策略订阅的股票列表

```py
get_subscribe_hq_stock_list():
```

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)

sub_list = tq.get_subscribe_hq_stock_list()
print(sub_list)
```

数据样本

```
['600519.SH']
```

← 取消订阅更新unsubscribe_hq

刷新行情缓存refresh_cache →

刷新行情缓存(最新snapshot和K线数据)refresh_cache

刷新行情缓存(最新snapshot和K线数据)。如果不调用，首次取snapshot和K线时系统会自动刷新一次行情

```py
def refresh_cache(market: str = 'AG',
                  force: bool = False):
```

输入参数

参数	是否必选	参数类型	参数说明
force	Y	bool	是否强制刷新
market	Y	str	指定刷新的市场

- force为false时距离上次刷新不足10分钟则不会刷新，为true时强制刷新。
- market赋值： 'AG'表示A股，'HK'表示港股，'US'表示美股，'QH'表示国内期货，'QQ'表示股票期权，'NQ'表示新三板，'ZZ'表示中证和国证等指数，'OF'表示基金净值，'ZS' 表示沪深京指数，'OJ' 表示期货期权。

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)
refresh_cache = tq.refresh_cache()
print(refresh_cache)
```

数据样本

使用后会在客户端弹出刷新数据的加载界面，加载完成后才会有返回

```
{
   "Error" : "Refresh Cache Success.",
   "ErrorId" : "0",
   "run_id" : "1"
}
```

← 获得订阅列表get_subscribe_hq_stock_list

缓存历史K线refresh_kline →

刷新历史K线缓存refresh_kline

根据股票和周期刷新历史K线缓存，如果本地没有下载完整的日线等数据，则可以调用这个函数定向下载某些品种某些周期的历史K线数据

```py
refresh_kline(stock_list: List[str] = [], period: str = '')
```

输入参数

参数	是否必选	参数类型	参数说明
stock_list	Y	List[str]	证券代码列表，证券代码格式为6位数+市场后缀（.SH/.SZ/.BJ等）
period	Y	str	周期 1d为日线、1m为一分钟线、5m为五分钟线，只支持这三种，其它周期的数据均由这三种数据生成

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
refresh_kline = tq.refresh_kline(stock_list=['688318.SH'],period='1d')
print(refresh_kline)
```

数据样本

注：如果在盘中交易时间段下载1m和5m分钟线，只能下载到截止上个交易日的数据
使用后会在客户端弹出刷新数据的加载界面，加载完成后才会有返回

```
{
   "Error" : "refresh kline cache success.",
   "ErrorId" : "0",
   "run_id" : "1"
}
```

← 刷新行情缓存refresh_cache　　下载特定数据文件download_file →

发送消息到通达信客户端send_message

发送消息给通达信客户端的TQ策略界面

```py
send_message(msg_str: str) -> Dict:
```

输入参数

参数	是否必选	参数类型	参数说明
msg_str	Y	str	消息字符串

- 传入的字符串使用 | 可以让客户端将其分为两条（插入 \n 也可以分行显示）

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
msg_str = "这是第一行. | 这是第二行. "
tq.send_message(msg_str)
```

← 获取交易日列表get_trading_dates

发送预警信号到客户端send_warn →

发送预警信号send_warn

往客户端发送指定股票的预警信号

```py
send_warn(stock_list:        List[str] = [],
              time_list:         List[str] = [],
              price_list:        List[str] = [],
              close_list:        List[str] = [],
              volum_list:        List[str] = [],
              bs_flag_list:      List[str] = [],
              warn_type_list:    List[str] = [],
              reason_list:       List[str] = [],
              count:         int  = 1) -> Dict:
```

输入参数

参数	是否必选	参数类型	参数说明
stock_list	Y	List[str]	证券代码列表
time_list	Y	List[str]	时间列表
price_list	N	List[str]	现价列表
close_list	N	List[str]	收盘价列表
volum_list	N	List[str]	成交额列表
bs_flag_list	N	List[str]	买卖标志：0买1卖2未知
warn_type_list	N	List[str]	预警类型：0常规预警（目前仅支持）
reason_list	N	List[str]	预警原因
count	N	int	有效数据个数

- price_list、close_list、volum_list、bs_flag_list、warn_type_list 均要求为纯数字字符串List
- bs_flag_list 0买1卖2未知，长度小于count的会自动补为2。
- reason_list每个元素有效长度为25个汉字（50个英文）|
- count限定入参中每个list中的有效数据个数，即每个list前count个数据会传给客户端
- stock_list与其他list的元素数据是一一对应的，即stock_list的第一个元素对应的预警信息是其他list的第一个元素，同一只股票的多个预警信息，则在stock_list中加入多次该股票

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
warn_res = tq.send_warn(stock_list = ['688318.SH','688318.SH','600519.SH'],
             time_list = ['20251215141115','20251215142100','20251215143101'],
             price_list= ['123.45','133.45','1823.45'],
             close_list= ['122.50','132.50','1822.50'],
             volum_list= ['1000','2000','15000'],
             bs_flag_list= ['0'],
             warn_type_list= ['0'],
             reason_list= ['价格突破预警线','收盘价突破预警线','成交量突破预警线'],
             count=3)
print(warn_res)
```

数据样本

```
{'Error': '发送预警信号成功.', 'ErrorId': '0', 'run_id': '1'}
```

← 发送消息到TQ策略界面send_message

发送文件到客户端send_file →

发送文件到客户端send_file

往通达信客户端发送文件名，可由TQ策略数据浏览中打开

```py
send_file(file: str) -> Dict:
```

输入参数

参数	是否必选	参数类型	参数说明
file	Y	str	文件路径

- 文件放于 .\PYPlugins\file\ 文件夹中时，file可仅传入文件名
- 文件放于其他位置时，file需要传入绝对路径
- 目前支持的文件类型：txt，pdf，html

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
file = "test.txt"
tq.send_file(file)
```

← 发送预警信号到客户端send_warn

发送回测数据send_bt_data →

发送回测数据send_bt_data

往客户端发送指定股票的回测数据

```py
send_bt_data(stock_code:          str  = '',
             time_list:         List[str] = [],
             data_list:         List[List[str]] = [],
             count:        int  = 1) -> Dict:
```

输入参数

参数	是否必选	参数类型	参数说明
stock_code	Y	List[str]	证券代码
time_list	Y	List[str]	时间列表
data_list	N	List[List[str]]	回测数据列表
count	N	int	有效数据个数

- data_list为二维List，每个子元素对应time_list的一个元素时间点，且每个子元素最多有16个有效纯数字字符串，即data_list每个子List的前16个数据为一个时间点的有效数据
- count限定入参中每个list中的有效数据个数，即每个list前count个数据会传给客户端

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)

bt_data = tq.send_bt_data(stock_code = '688318.SH',
                          time_list = ['20251215141115'],
                          data_list = [['11']],
                          count = 1)
print(bt_data)
```

数据样本

```
{'Error': '发送回测结果成功.', 'ErrorId': '0', 'run_id': '1'}
```

← 发送文件到客户端send_file

打印数据到客户端print_to_tdx →

下载特定数据文件download_file

10大股东数据文件、ETF申赎清单文件、最近舆情信息文件、股票综合信息文件、经营分析数据文件

```py
download_file(stock_code: str = '',
                    down_time:str = '',
                    down_type:int = 1):
```

输入参数

参数	是否必选	参数类型	参数说明
stock_code	Y	List[str]	证券代码
down_time	Y	List[str]	指定日期
down_type	Y	List[str]	指定下载类型

- down_type=1时，下载10大股东数据文件，down_time为指定日期
- down_type=2时，下载ETF申赎清单文件，down_time为指定日期
- down_type=3时，下载最近舆情信息文件，其余两项无效
- down_type=4时，下载股票综合信息文件，其余两项无效
- down_type=5时，下载经营分析数据文件，down_time为指定日期
- 下载的文件保存在 .\PYPlugins\data 文件夹
- down_type=1时，下载的文件中含指定日期所在年度的所有10大股东数据和流通股东数据
- down_type=5时，下载的文件中含指定日期所在年度的所有经营分析数据

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
# 下载10大股东数据
down_ptr_10 = tq.download_file(stock_code='688318.SH',
down_time='20241231',down_type=1)
print(down_ptr_10)
# 下载ETF申赎数据
dowm_ptr_etf = tq.download_file(stock_code='159109.SH',
down_time='20260227',down_type=2)
print(dowm_ptr_etf)
```

数据样本

```
{
   "ErrorId" : "0",
   "Msg" : "下载十大股东数据[2025]成功。",
   "run_id" : "1"
}

{
   "ErrorId" : "0",
   "Msg" : "下载ETF申述清单[20250101]成功。",
   "run_id" : "1"
}
```

← 缓存历史K线refresh_kline

获取交易日列表get_trading_dates →

获取交易日列表get_trading_dates

根据指定时间段获取交易日列表

```py
get_trading_dates(market: str,
                  start_time: str,
                  end_time: str,
                  count:int = -1) -> List:
```

输入参数

参数	是否必选	参数类型	参数说明
market	Y	str	市场代码（暂固定为SH）
start_time	N	str	起始日期
end_time	N	str	结束日期
count	N	int	返回最近的count个交易日

- 需要先在客户端下载上证指数（999999）的盘后数据 目前仅支持A股
- count > 0时，限制返回从结束日期往前最近的count个在限定时间段中的交易日

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)

trade_dates = tq.get_trading_dates(market = 'SH', start_time = '20220101',
end_time = '', count = 10);
print(trade_dates)
```

数据样本

```
['20251211', '20251212', '20251215', '20251216', '20251217', '20251218',
'20251219', '20251222', '20251223', '20251224']
```

← 下载特定数据文件download_file

发送消息到TQ策略界面send_message →

导出多组数据到通达信客户端 print_to_tdx

将计算数据导出到通达信客户端展示

```py
print_to_tdx(df_list:          list[pd.DataFrame] = [],
             sp_name:          str  = "",
             xml_filename:     str  = "",
             jsn_filenames:    list[str] = None,
             vertical:         int  = None,
             horizontal:       int  = None,
             height:           list[str | float] = None,
             table_names:      list[str] = None) -> None:

```

输入参数

参数	是否必选	参数类型	参数说明
df_list	Y	list[pd.DataFrame]	多组数据的DataF 表，每组table对 DataFrame；每个 DataFrame非空且 日期（datetime64 符串类型），后续 因子名称；列表长 组数
sp_name	N	str	生成.sp文件的名称 空时默认生成pytl
xml_filename	N	str	生成的xml文件名 含.xml后缀），为空 达信面板配置关联 填
jsn_filenames	Y	list[str]	每组数据对应的.js 表，列表非空且长 组数（与df_list一 名建议包含.jsn后
vertical	N	int	纵向排列的table组 （≥1），与horizo 一，horizontal优

参数	是否必选	参数类型	参数说明
horizontal	N	int	横向排列的table组（≥1），优先级高vertical，未指定时vertical或1组
height	N	list[str | float]	自定义每组gridct表，长度需等于组为数值/字符串（比），未指定时自组数，最后一组高
table_names	N	list[str]	每组展示面板的标长度需等于组数；时自动使用对应jsn_filenames的前题

- df_list、jsn_filenames长度必须与vertical/horizontal指定的组数完全一致，否则会抛出ValueError异常
- height参数值为高度占比（如0.3/"0.3"），表示该面板占整体展示区域的比例，仅支持0-1之间的数值
- 未指定vertical/horizontal时，默认按1组纵向排列展示，自动计算面板高度

← 发送回测数据send_bt_data

调用客户端功能exec_to_tdx →

调用客户端功能

客户端根据入参执行指定功能

```py
def exec_to_tdx(url:str = ''):
```

输入参数

参数	是否必选	参数类型	参数说明
url	Y	str	功能调用串或网址

若是功能串，请以 http://www.treeid 开头

主要功能串

功能串	说明和示例
inhttp	内部打开 比如：http://www.treeid/inhttp://.....
dlghttp	内部对话框打开 比如：http://www.treeid/dlghttp://.....&tdxmyietitle=标题&tdxmyiewidth=500&tdxmyieheight=300&noborder=0
localurl	内部打开(非对话框) 比如：http://www.treeid/localurlc:\pa\tips.html.....
dlglocalurl	内部打开(对话框) 比如：http://www.treeid/dlglocalurlc:\pa\tips.mht.....
code_	进入某只股票(只传入代码)
breed_	到某个品种(可以传入市场和代码,如果不清楚市场,在代码前加-即可进行模糊处理), 比如到财富趋势 http://www.treeid/breed_1#688318 市场：0#为深市 1#为沪市 2#为京市
zb_	指标公式 比如：http://www.treeid/zb_MACD
exp_	专家系统公式
padcode_	进入用户定制版面,后面是版面简称
ZXG	自选股列表

功能串	说明和示例
ETF	ETF基金
HK	显示港股
QH	显示期货
MAINQH	显示为主力期货合约
SORT67	排行(67)

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)

exec_res1 = tq.exec_to_tdx(url='http://www.treeid/MAINQH')

exec_res2 = tq.exec_to_tdx(url='http://www.treeid/dlghttp://www.tdx.com.cn')
print(exec_res2)
```

数据样本

```
{'ErrorId': '0', 'Msg': 'http://www.treeid/dlghttp://www.tdx.com.cn', 'run_id':
'1'}
```

← 打印数据到客户端print_to_tdx

检索证券信息get_match_stkinfo →

检索证券信息get_match_stkinfo

检索证券信息

```py
def get_match_stkinfo(key_word:str = ''):
```

输入参数

参数	是否必选	参数类型	参数说明
key_word	Y	str	关键词

输出数据

名称	类型	数值	说明
Code	Y	str	证券代码
Name	Y	str	证券名称

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
match_stkinfo = tq.get_match_stkinfo(key_word='通达信')
print(match_stkinfo)
```

数据样本

```
[{'Code': '880515.SH', 'Name': '通达信88'},
{'Code': '880818.SH', 'Name': '通达信热股'},
{'Code': '688318.SH', 'Name': '财富趋势'}]
```

← 调用客户端功能exec_to_tdx

获取K线数据get_market_data →

通达信TQ可以获取如下数据

股票数据：我们拥有所有A股、港股、美股、新三板等上市公司的股票行情数据、基本信息、财务数据、市场交易数据等。
基金数据：我们目前提供了多种在交易所上市的基金的行情、净值等数据，包含ETF、LOF、REITs等。
指数数据：我们支持沪深京指数、板块指数、港股指数、中证国证指数，包括指数的行情数据以及成分股数据。具体见get_index_stocks。
行业等板块：我们支持按通达信行业、概念、风格板块获取成分股，具体见获取A股板块代码列表get_sector_list及获取板块成份股get_stock_list_in_sector。
期货数据：我们支持中金所、郑商所、大商所、上期所、上期能源、广期所的行情数据，并包含历史产品的数据。
宏观数据：我们提供全方位的宏观数据（.HG），为投资者决策提供有力数据支持。
可转债和期权等数据：我们提供完整的沪深京可转债类数据、沪深个股期权等数据。
数据信息文件包：我们提供各类特定的数据信息文件包。具体见download_file。

获取K线行情get_market_data

根据股票，获取历史行情

```py
get_market_data(field_list: List[str] = [],
                    stock_list: List[str] = [],
                    period: str = '',
                    start_time: str = '',
                    end_time: str = '',
                    count: int = -1,
                    dividend_type: Optional[str] = None,
                    fill_data: bool = True) -> Dict:
```

输入参数

参数	是否必选	参数类型	参数说明
field_list	N	List[str]	字段筛选，传空则返回全部
stock_list	Y	List[str]	证券代码列表
period	Y	str	周期
start_time	N	str	起始时间
end_time	N	str	结束时间
count	N	int	返回数据个数（每只股票）
dividend_type	N	str	复权类型：none不复权、front前复权、back后复权
fill_data	N	bool	是否向后填充空缺数据

count小于等于0或者count为空：
1、开始日期结束日期数据数据；
2、开始日期无值， 从第一根k取到 结束日期
3、结束日期无值，从开始日期取到最后一根
4、都没值取全部本地数据
count大于0时：
1、结束日期往前n个数据
2、结束日期无时从最后一根k线往前取n。

返回数据

- 返回dict { field1 : value1, field2 : value2, ... }
- field1, field2, ... ： 数据字段
- value1, value2, ... ： pd.DataFrame 数据集，index为stock_list，columns为time_list
- 各字段对应的DataFrame维度相同、索引相同
- 只有dividend_type传入为none时，会返回有效的前复权因子ForwardFactor
- 后复权数据与取的数据个数有关，只在返回的数据中进行后复权
- 一次最多返回24000条数据，要获取完整分钟线需要多次分批获取
- 返回复权数据时，若该组数据时间内未发生权息变动，则复权价与未复权价相同，

数据	默认返回	数据类型	数据说明
Date	Y	str	日期
Time	Y	str	时间
Open	Y	str	开盘价
High	Y	str	最高价
Low	Y	str	最低价
Close	Y	str	收盘价
Volume	Y	str	成交量
Amount	Y	str	成交额
ForwardFactor	Y	str	前复权因子，当dividend_type=none时候返回有效值
VolInStock	N	str	持仓量

- 期货数据时Amount为0，非期货数据时VolInStock为0

接口使用

> 获取688318.SH从2025-12-20到今为止最新一条日K线的不复权数据

```py
from tqcenter import tq
tq.initialize(__file__)
df = tq.get_market_data(
        field_list=[],
        stock_list=['688318.SH'],
        start_time='20251220',
        end_time='',
        count=1,
        dividend_type='none',
        period='1d',
        fill_data=True
    )
```

```
print(df)

```

数据样本

```
{'Amount':             688318.SH
2025-12-24   29394.81,
'Low':             688318.SH
2025-12-24      128.0,
'Date':              688318.SH
2025-12-24  20251224.0,
'Volume':             688318.SH
2025-12-24  2257325.0,
'Close':             688318.SH
2025-12-24     131.58,
'Open':             688318.SH
2025-12-24     128.01,
'Time':             688318.SH
2025-12-24        0.0,
'High':             688318.SH
2025-12-24     131.87,
'ForwardFactor':             688318.SH
2025-12-24        1.0}
```

← 检索证券信息get_match_stkinfo

获取快照数据get_market_snapshot →

获取快照数据get_market_snapshot

根据股票，获取最新行情数据

```py
def get_market_snapshot(stock_code: str,
                        field_list: List = []) -> Dict:
```

输入参数

参数	是否必选	参数类型	参数说明
stock_code	Y	str	证券代码
field_list	N	List[str]	字段筛选，传空则返回全部

返回数据

数据	默认返回	数据类型	数据说明
ItemNum	Y	str	快照笔数
LastClose	Y	str	前收盘价
Open	Y	str	开盘价
Max	Y	str	最高价
Min	Y	str	最低价
Now	Y	str	现价
Volume	Y	str	总手
NowVol	Y	str	现手
Amount	Y	str	总成交金额
Inside	Y	str	内盘 板块指数时为跌停家数
Outside	Y	str	外盘 板块指数时为涨停家数
TickDiff	Y	str	笔涨跌
InOutFlag	Y	str	内外盘标志 0:Buy 1:Sell 2:Unknown

数据	默认返回	数据类型	数据说明
Jjjz	Y	str	基金净值
Buyp	Y	List[str]	五个买价
Buyv	Y	List[str]	对应的五个买盘量
Sellp	Y	List[str]	五个卖价
Sellv	Y	List[str]	对应的五个卖盘量
UpHome	Y	str	上涨家数 对于指数有效
DownHome	Y	str	下跌家数 对于指数有效
Before5MinNow	Y	str	5分钟前价格
Average	Y	str	均价
XsFlag	Y	str	小数位数
Zangsu	Y	str	涨速
ZAFPre3	Y	str	3日涨幅

接口使用

> 获取688318.SH从2025-12-20到今为止最新一条日K线的不复权数据

```py
from tqcenter import tq

tq.initialize(__file__)

market_snapshot = tq.get_market_snapshot(stock_code = '688260.SH', field_list=
[])
print(market_snapshot)
```

数据样本

```
{'ItemNum': '3342',
'LastClose': '34.21',
'Open': '33.78',
'Max': '36.49',
'Min': '32.50',
'Now': '35.06',
'Volume': '122881',
'NowVol': '1449',
'Amount': '43068.48',
'Inside': '60373',
```

```
'Outside': '62509',
'TickDiff': '0.00',
'InOutFlag': '2',
'Jjjz': '0.00',
'Buyp': ['35.05', '35.04', '35.02', '35.01', '35.00'],
'Buyv': ['154', '9', '49', '136', '154'],
'Sellp': ['35.06', '35.07', '35.08', '35.09', '35.10'],
'Sellv': ['4', '31', '139', '4', '4'],
'UpHome': '0',
'DownHome': '0',
'Before5MinNow': '35.15',
'Average': '35.05',
'XsFlag': '2',
'Zangsu': '-0.25',
'ZAFPre3': '-1.83',
'ErrorId': '0'}
```


← 获取K线数据get_market_data

获取证券基本信息get_stock_info →

获取证券基本信息get_stock_info

根据股票，获取股票基础的财务数据

```py
get_stock_info(cls,
                    stock_code:str,
                    field_list: List = []) -> Dict:
```

输入参数

参数	是否必选	参数类型	参数说明
stock_code	Y	str	证券代码
field_list	Y	List[str]	字段筛选，不能为空

返回数据

数据	默认返回	数据类型	数据说明
Name	Y	str	证券名称
Unit	Y	str	交易单位
VolBase	Y	str	量比的基量
MinPrice	Y	str	最小价格变动
XsFlag	Y	str	价格小数位数
Fz[8]	Y	List[str]	开收市时间（4段）
DelayMin	Y	str	延时分钟数
QHVolBaseRate	Y	str	期货期权的每手乘数
HKVolBaseRate	Y	str	港股/日股/新加坡股 每手股数
BelongHS300	Y	str	是否属于沪深300
BelongHasKQZ	Y	str	是否含可转债
BelongRZRQ	Y	str	是否是融资融券标的
BelongHSGT	Y	str	是否属于沪深股通

数据	默认返回	数据类型	数据说明
IsHKGP	Y	str	是否是港股
IsQH	Y	str	是否是期货
IsQQ	Y	str	是否是期权
IsSTGP	Y	str	是否是ST股票
IsQuitGP	Y	str	是否是退市整理板股票
TodayDRFlag	Y	str	当天是否有除权除息(沪深京)
HSStockKind	Y	str	沪深京品种类型 0:指数,1:A股主板,2:北证A股,3:创业板,4:科创板,5:B股,6:债券,7:基金,8:权证,9:其它,10:非沪深京品种
ActiveCapital	Y	str	流通股本(万股)
J_zgb	Y	str	总股本(万股)
J_bg	Y	str	B股(万股)
J_hg	Y	str	H股(万股)
J_zzc	Y	str	总资产(万元)
J_ldzc	Y	str	流动资产(万元)
J_gdzc	Y	str	固定资产(万元)
J_wxzc	Y	str	无形资产(万元)
J_ldfz	Y	str	流动负债(万元)
J_cqfz	Y	str	少数股东权益(万元)
J_zbgjj	Y	str	资本公积金(万元)
J_jzc	Y	str	股东权益/净资产(万元)
J_yysy	Y	str	营业收入(万元)
J_yycb	Y	str	营业成本(万元)
J_yszk	Y	str	应收账款(万元)
J_yyly	Y	str	营业利润(万元)
J_tzsy	Y	str	投资收益(万元)
J_jyxjl	Y	str	经营现金净流量(万元)

数据	默认返回	数据类型	数据说明
J_zxjl	Y	str	总现金净流量(万元)
J_ch	Y	str	存货(万元)
J_lyze	Y	str	利润总额(万元)
J_shly	Y	str	税后利润(万元)
J_jly	Y	str	净利润(万元)
J_wfply	Y	str	未分配利益(万元)
J_jyl	Y	str	净资产收益率
J_mgwfp	Y	str	每股未分配
J_mgsy	Y	str	每股收益（折算为全年）
J_mgsy2	Y	str	季报每股收益 (财报中提供的每股收益)
J_mggjj	Y	str	每股公积金
J_mgjzc	Y	str	每股净资产
J_mgjzc2	Y	str	季报每股净资产 (财报中提供的每股收益)
J_gdqyb	Y	str	股东权益比
J_gdrs	Y	str	股东人数
J_HalfYearFlag	Y	str	报告期月份(3,6,9,12)
J_start	Y	str	上市日期
tdx_dycode	Y	str	通达信地域代码
tdx_dyname	Y	str	通达信地域
rs_hycode_sim	Y	str	通达信行业代码
rs_hyname	Y	str	通达信行业
blockzscode	Y	str	所属的行业板块指数代码
underly_setcode	Y	str	标的市场代码(比如：当前ETF跟踪的指数市场)
underly_code	Y	str	标的代码(比如：当前ETF跟踪的指数代码)

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
fdc = tq.get_stock_info(stock_code='688318.SH', field_list=[])
print(fdc)
```

数据样本


```
{'Name': '财富趋势',
'Unit': '100',
'VolBase': '102.22',
'MinPrice': '0.01',
'XsFlag': '2',
'Fz': ['570', '690', '780', '900', '900', '900', '900', '900'],
'DelayMin': '0',
'QHVolBaseRate': '0',
'HKVolBaseRate': '0',
'BelongHS300': '0',
'BelongHasKQZ': '0',
'BelongRZRQ': '1',
'BelongHSGT': '1',
'IsHKGP': '0',
'IsQH': '0',
'IsQQ': '0',
'IsSTGP': '0',
'IsQuitGP': '0',
'TodayDRFlag': '0',
'HSStockKind': '4',
'ActiveCapital': '25611.94',
'J_zgb': '25611.94',
'J_bg': '0.00',
'J_hg': '0.00',
'J_zzc': '389036.97',
'J_ldzc': '235598.84',
'J_gdzc': '972.62',
'J_wxzc': '1184.64',
'J_ldfz': '17412.97',
'J_cqfz': '73.15',
'J_zbgjj': '157998.02',
'J_jzc': '370454.03',
'J_yysy': '19827.85',
'J_yycb': '4258.70',
'J_yszk': '2726.99',
'J_yyly': '20836.07',
'J_tzsy': '5091.96',
'J_jyxjl': '5432.08',
'J_zxjl': '9779.30',
'J_ch': '61.84',
'J_lyze': '20829.85',
```

```
'J_shly': '18421.45',
'J_jly': '18421.34',
'J_wfply': '175521.63',
'J_jyl': '4.97',
'J_mgwfp': '6.85',
'J_mgsy': '0.96',
'J_mgsy2': '0.00',
'J_mggjj': '6.17',
'J_mgjzc': '14.46',
'J_mgjzc2': '14.46',
'J_gdqyb': '0.95',
'J_gdrs': '24154.00',
'J_HalfYearFlag': '9',
'J_start': '20200427',
'tdx_dycode': '18',
'tdx_dyname': '深圳板块',
'rs_hycode_sim': 'X4202',
'rs_hyname': '软件服务',
'blockzscode': '881355',
'underly_setcode': '0',
'underly_code': '',
'ErrorId': '0'}
```


← 获取快照数据get_market_snapshot

获取股票更多信息get_more_info →

获取股票更多信息get_more_info

获取指定股票更细节的信息

```py
def get_more_info(stock_code:str = '',
                  field_list: List = []):
```

输入参数

参数	是否必选	参数类型	参数说明
stock_code	Y	str	股票代码
field_list	N	List[str]	字段筛选，传空则返回全部

返回数据

数据	默认返回	数据类型	数据说明
MainBusiness	Y	str	主营构成
SafeValue	Y	str	安全分
ShineValue	Y	str	亮点数
ShapeValue	Y	str	短期形态+中期形态+长期形态 编号
TPFlag	Y	str	停牌标识
ZTPrice	Y	str	涨停价
DTPrice	Y	str	跌停价
HqDate	Y	str	行情日期
fHSL	Y	str	换手率
fLianB	Y	str	量比
Wtb	Y	str	委比
Zsz	Y	str	总市值(亿)

数据	默认返回	数据类型	数据说明
Ltsz	Y	str	流通市值(亿)
vzangsu	Y	str	量涨速
Fzhsl	Y	str	分钟换手率
FzAmo	Y	str	2分钟金额(万元)
VOpenZAF	Y	str	抢筹涨幅
ZAF	Y	str	涨幅
ZAFYesterday	Y	str	昨日涨幅
ZAFPre2D	Y	str	前天涨幅
ZAFPre5	Y	str	5日涨幅
ZAFPre10	Y	str	10日涨幅
ZAFPre20	Y	str	20日涨幅
ZAFPre30	Y	str	30日涨幅
ZAFPre60	Y	str	60日涨幅
ZAFYear	Y	str	年初至今涨幅
ZAFPreMyMonth	Y	str	涨幅(本月来)
ZAFPreOneYear	Y	str	涨幅(一年来)
Zjl	Y	str	主买净额(万元)
Zjl_HB	Y	str	主力净流入(万元)
TotalBVol	Y	str	总买量
TotalSVol	Y	str	总卖量
BCancel	Y	str	总撤买量
SCancel	Y	str	总撤卖量
L2TicNum	Y	str	L2逐笔成交数
L2OrderNum	Y	str	L2逐笔委托数
FCAmo	Y	str	封单额(万元)

数据	默认返回	数据类型	数据说明
FCb	Y	str	封成比
OpenAmo	Y	str	开盘金额(万元)(A股和板块指数有效)
OpenZTBuy	Y	str	竞价涨停买入金额(万元)
OpenAmoPre1	Y	str	昨开盘金额(万元)
OpenVolPre1	Y	str	昨开盘量
CJJEPre1	Y	str	昨成交额(万元)
CJJEPre3	Y	str	3日成交额(万元)
FDEPre1	Y	str	昨封单额(万元)
FDEPre2	Y	str	前封单额(万元)
ZTGPNum	Y	str	板块指数的涨停家数
LastStartZT	Y	str	几天
LastZTHzNum	Y	str	几板
EverZTCount	Y	str	连板天
ConZAFDateNum	Y	str	连涨天数
YearZTDay	Y	str	年涨停天数
MA5Value	Y	str	5日均价
HisHigh	Y	str	52周最高
HisLow	Y	str	52周最低
IPO_Price	Y	str	发行价
More_YJL	Y	str	ETF,LOF溢价率
BetaValue	Y	str	贝塔系数
DynaPE	Y	str	动态市盈率
MorePE	Y	str	市盈率(港股:动,其他扩展:静)
StaticPE_TTM	Y	str	市盈率(TTM)
DYRatio	Y	str	股息率

数据	默认返回	数据类型	数据说明
PB_MRQ	Y	str	市净率(MRQ)
IsT0Fund	Y	str	是否是T+0基金
IsZCZGP	Y	str	是否是注册制A股
IsKzz	Y	str	是否是可转债
Kzz_HSCode	Y	str	可转债对应的正股代码
QHMainYYMM	Y	str	主力合约关联的月份(期货),主力和次主力
FreeLtgb	Y	str	自由流通股本(万)
Yield	Y	str	应计利息(债券),占款天数(回购)
KfEarnMoney	Y	str	扣非净利润(万元)
RDInputFee	Y	str	研发费用(万元)
CashZJ	Y	str	货币资金(万元)
PreReceiveZJ	Y	str	合同负债(万元)
OtherQYJzc	Y	str	其它权益工具(万元)
StaffNum	Y	str	员工人数
RecentGGJYDate	Y	str	最近北上大额交易日
RecentHGDate	Y	str	最近回购预案日
RecentIncentDate	Y	str	最近股权激励预案日
NoticeDate_Recent	Y	str	最近业绩预告日
RecentReleaseDate	Y	str	最近解禁日
RecentDZDate	Y	str	最近定增日
ReportDate	Y	str	最近财报公告日期
ZTDate_Recent	Y	str	近2年最近涨停板日期
DTDate_Recent	Y	str	近2年最近跌停板日期
TopDate_Recent	Y	str	近2年最近龙虎榜日期
StopJYDate_Recent	Y	str	最近停牌日期

提示：

- 涨停跌停的判断：用get_more_info取FCAmo来判断涨停跌停，大于0是涨停，小于0是跌停。

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
more_info = tq.get_more_info(stock_code = '688318.SH', field_list=[])
print(more_info)
```

数据样本

```
{'MainBusiness': '软件服务收入',
'SafeValue': '98',
'ShineValue': '3',
'ShapeValue': '101308',
'TPFlag': '0',
'ZTPrice': '151.62',
'DTPrice': '101.08',
'HqDate': '20260227',
'fHSL': '0.86',
'fLianB': '0.89',
'Wtb': '-0.66',
'Zsz': '326.91',
'Ltsz': '326.91',
'vzangsu': '2.17',
'Fzhsl': '0.12',
'FzAmo': '514.92',
'VOpenZAF': '0.00',
'ZAF': '1.02',
'ZAFYesterday': '-1.21',
'ZAFPre2D': '1.99',
'ZAFPre5': '-1.56',
'ZAFPre10': '-3.44',
'ZAFPre20': '-10.76',
'ZAFPre30': '-10.13',
'ZAFPre60': '-1.59',
'ZAFYear': '-3.54',
'ZAFPreMyMonth': '-5.23',
'ZAFPreOneYear': '10.27',
'Zjl': '0.00',
'Zjl_HB': '0.00',
'TotalBVol': '1295.00',
'TotalSVol': '3555.00',
'BCancel': '42606.00',
'SCancel': '40266.00',
'L2TicNum': '6880',
'L2OrderNum': '29448',
'FCAmo': '0.00',
'FCb': '0.00',
'OpenAmo': '1069400.00',
'OpenFDE': '0.00',
```

```
'OpenAmoPre1': '77.93',
'OpenVolPre1': '61.00',
'CJJEPre1': '26056.68',
'CJJEPre3': '89751.03',
'FDEPre1': '0.00',
'FDEPre2': '0.00',
'ZTGPNum': '0',
'LastStartZT': '0',
'LastZTHzNum': '0',
'EverZTCount': '0',
'ConZAFDateNum': '1',
'YearZTDay': '0',
'MA5Value': '126.56',
'HisHigh': '180.86',
'HisLow': '83.41',
'IPO_Price': '107.41',
'More_YJL': '0.00',
'BetaValue': '2.31',
'DynaPE': '133.10',
'MorePE': '107.56',
'StaticPE_TTM': '94.99',
'DYRatio': '0.28',
'PB_MRQ': '8.82',
'IsT0Fund': '0',
'IsZCZGP': '1',
'IsKzz': '0',
'Kzz_HSCode': '0',
'FreeLtgb': '7935.14',
'Yield': '106.94',
'KfEarnMoney': '9778.22',
'RDInputFee': '5894.58',
'CashZJ': '60954.52',
'PreReceiveZJ': '11281.48',
'OtherQYJzc': '0.00',
'StaffNum': '446',
'RecentGGJYDate': '0',
'RecentHGDate': '0',
'RecentIncentDate': '0',
'NoticeDate_Recent': '0',
'RecentReleaseDate': '20230427',
'RecentDZDate': '0',
'ReportDate': '20251031',
'ZTDate_Recent': '20241008',
'DTDate_Recent': '0',
'TopDate_Recent': '20250625',
'StopJYDate_Recent': '0'}
```


← 获取证券基本信息get_stock_info

批量获取价格和成交量get_pricevol →

批量获取价量get_pricevol

批量获取指定股票集的前收盘价、现价和总成交量

```py
def get_pricevol(stock_list: List[str] = []):
```

输入参数

参数	是否必选	参数类型	参数说明
stock_list	Y	List[str]	证券代码列表

输出数据

名称	类型	数值	说明
LastClose	Y	str	前收盘价
Now	Y	str	现价
Volume	Y	str	成交量

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
all_stocks = tq.get_stock_list(market='23')
pv_info = tq.get_pricevol(stock_list=all_stocks)
print(pv_info)
```

数据样本

```
{'000001.SZ': {'LastClose': '10.70', 'Now': '10.71', 'Volume': '422239'},
'000002.SZ': {'LastClose': '3.47', 'Now': '3.46', 'Volume': '423677'},
'000063.SZ': {'LastClose': '34.83', 'Now': '34.99', 'Volume': '587361'},
'000100.SZ': {'LastClose': '4.37', 'Now': '4.17', 'Volume': '10003565'},
'000157.SZ': {'LastClose': '7.27', 'Now': '7.20', 'Volume': '227512'},
'000166.SZ': {'LastClose': '4.47', 'Now': '4.42', 'Volume': '445736'},
'000301.SZ': {'LastClose': '12.52', 'Now': '12.53', 'Volume': '100953'},
'000333.SZ': {'LastClose': '81.50', 'Now': '80.45', 'Volume': '99257'},
'000338.SZ': {'LastClose': '32.28', 'Now': '33.13', 'Volume': '422523'},
'000408.SZ': {'LastClose': '78.19', 'Now': '78.99', 'Volume': '36197'},
```

```
'000425.SZ': {'LastClose': '9.57', 'Now': '9.62', 'Volume': '418614'},
'000538.SZ': {'LastClose': '50.13', 'Now': '49.87', 'Volume': '33429'},...}
```

← 获取股票更多信息get_more_info

获取分红送配数据get_divid_factors →

获取分红送配数据get_divid_factors

根据股票，获取指定时间段内的分红送配等数据

```py
get_divid_factors(stock_code: str,
                          start_time: str,
                          end_time: str) -> pd.DataFrame:
```

输入参数

参数	是否必选	参数类型	参数说明
stock_code	Y	str	证券代码
start_time	N	str	起始时间
end_time	N	str	结束时间

返回数据

数据	默认返回	数据类型	数据说明
Type	Y	str	类型 1:除权除息 11:扩缩股 15:重新调整
Bonus	Y	str	红利
AlloPrice	Y	str	配股价
ShareBonus	Y	str	送股/扩缩股比例
Allotment	Y	str	配股

接口使用

注：此接口只支持沪深京的分红送配数据

> 例如：获取688318.SH全部分红配送数据

```py
from tqcenter import tq
tq.initialize(__file__)
divid_factors = tq.get_divid_factors(
        stock_code='688318.SH',
```

```
        start_time='',
        end_time='')
print(divid_factors)
```

数据样本

```
            Type  Bonus  AllotPrice  ShareBonus  Allotment
Date
2020-09-29     1    6.0         0.0         0.0        0.0
2021-05-27     1   10.0         0.0         0.0        0.0
2022-06-20     1   14.0         0.0         4.0        0.0
2023-06-13     1    5.0         0.0         4.0        0.0
2024-06-14     1    8.0         0.0         4.0        0.0
```

← 批量获取价格和成交量get_pricevol

获取股票所属板块get_relation →

获取股票所属板块

获取指定股票所属板块信息

```py
    def get_relation(stock_code:str = ''):
```

输入参数

参数	是否必选	参数类型	参数说明
stock_code	Y	str	股票代码

返回数据

数据	默认返回	数据类型	数据说明
BlockCode	Y	str	板块代码
BlockName	Y	str	板块名称
BlockType	Y	str	板块类型
GPNume	Y	str	成份股数量

- 没有板块代码的板块的BlockCode字段返回"0"

接口使用

```py
from tqcenter import tq
from tqcenter import tqconst
tq.initialize(__file__)

gp_block_res = tq.get_relation(stock_code='688318.SH')
print(gp_block_res)
```

数据样本

```
[{'BlockCode': '881355.SH', 'BlockName': '软件服务', 'BlockType': '行业',
'GPNume': '234'},
{'BlockCode': '880218.SH', 'BlockName': '深圳板块', 'BlockType': '地区', 'GPNume':
'427'},
{'BlockCode': '880592.SH', 'BlockName': '互联金融', 'BlockType': '概念', 'GPNume':
```

```
'211'},
{'BlockCode': '880722.SH', 'BlockName': '华为鸿蒙', 'BlockType': '概念', 'GPNume':
'262'},
{'BlockCode': '880916.SH', 'BlockName': '国产软件', 'BlockType': '概念', 'GPNume':
'266'},
{'BlockCode': '880948.SH', 'BlockName': '人工智能', 'BlockType': '概念', 'GPNume':
'1049'},
{'BlockCode': '880956.SH', 'BlockName': '腾讯概念', 'BlockType': '概念', 'GPNume':
'295'},
{'BlockName': '沪股通标的', 'BlockType': '风格', 'GPNume': '1763'},
{'BlockName': '融资融券', 'BlockType': '风格', 'GPNume': '4354'},
{'BlockCode': '880805.SH', 'BlockName': '保险重仓', 'BlockType': '风格', 'GPNume':
'200'},
{'BlockCode': '880878.SH', 'BlockName': '百元股', 'BlockType': '风格', 'GPNume':
'220'},
{'BlockName': '中证500', 'BlockType': '指数', 'GPNume': '500'},
{'BlockName': '中证800', 'BlockType': '指数', 'GPNume': '800'},
{'BlockName': '上证380', 'BlockType': '指数', 'GPNume': '380'},
{'BlockName': '金融科技', 'BlockType': '指数', 'GPNume': '59'},
{'BlockName': '科创100', 'BlockType': '指数', 'GPNume': '100'},
{'BlockName': '科创信息', 'BlockType': '指数', 'GPNume': '50'}]
```

← 获取分红送配数据get_divid_factors

获取新股申购信息get_ipo_info →

获取新股申购信息get_ipo_info

获取今天及未来的新股或新发债申购信息

```py
get_ipo_info(ipo_type:int = 0,
             ipo_date:int = 0):
```

输入参数

参数	是否必选	参数类型	参数说明
ipo_type	Y	str	自定义板块简称
ipo_date	Y	int	自定义板块名称

- ipo_type=0 表示获取新股申购信息
- ipo_type=1 表示获取新发债信息
- ipo_type=2 表示获取新股和新发债信息
- ipo_date=0 表示只获取今天信息
- ipo_date=1 表示获取今天及以后信息

返回数据

数据	默认返回	数据类型	数据说明
Code	Y	str	证券代码
Name	Y	str	证券名称
SGDate	Y	str	申购日期
SGPrice	Y	str	申购价格
SGCode	Y	str	申购代码
MaxSG	Y	str	申购上限
PE_Issue	Y	str	发行市盈率

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
```

```
ipo_info = tq.get_ipo_info(ipo_type=2, ipo_date=1)
print(ipo_info)
```

数据样本

```
[{'Code': '001248.SZ', 'Name': '华润新能源', 'SGDate': '20260622', 'SGPrice':
'0.00', 'SGCode': '1248', 'MaxSG': '0.00', 'PE_Issue': '0.00'},
{'Code': '920072.BJ', 'Name': '科莱瑞迪', 'SGDate': '20260615', 'SGPrice':
'0.00', 'SGCode': '920072', 'MaxSG': '0.00', 'PE_Issue': '0.00'},
{'Code': '001399.SZ', 'Name': '惠科股份', 'SGDate': '20260612', 'SGPrice':
'10.12', 'SGCode': '1399', 'MaxSG': '21.85', 'PE_Issue': '25.45'},
{'Code': '688797.SH', 'Name': '臻宝科技', 'SGDate': '20260612', 'SGPrice':
'44.56', 'SGCode': '787797', 'MaxSG': '0.90', 'PE_Issue': '31.31'},
{'Code': '603270.SH', 'Name': '金帝转债', 'SGDate': '20260615', 'SGPrice':
'100.00', 'SGCode': '754270', 'MaxSG': '0.00', 'PE_Issue': '0.00'},
{'Code': '603809.SH', 'Name': '豪26转债', 'SGDate': '20260615', 'SGPrice':
'100.00', 'SGCode': '754809', 'MaxSG': '0.00', 'PE_Issue': '0.00'}]
```

← 获取股票所属板块get_relation

获取每天的股本数据get_gb_info →

获取每天的股本数据get_gb_info

获取指定股票的股本数据

```py
def get_gb_info(stock_code:str = '',
                date_list: List[str] = [],
                count: int = 1):
```

输入参数

参数	是否必选	参数类型	参数说明
stock_code	Y	str	股票代码
date_list	Y	List[str]	日期数组
count	Y	int	日期有效个数

- date_list传入的日期须从小到大排序
- date_list有效数据个数须不小于count，且不能小于1

输出数据

名称	类型	数值	说明
Date	double		日期
Zgb	double		总股本
Ltgb	double		流通股本

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
gb_info = tq.get_gb_info(stock_code = '688318.SH', date_list=
['20250101','20250601'], count=2)
print(gb_info)
```

数据样本

```
[{'Date': 20250101, 'Zgb': 182942480.0, 'Ltgb': 182942480.0},
{'Date': 20250601,  'Zgb': 182942480.0, 'Ltgb': 182942480.0}]
```

← 获取新股申购信息get_ipo_info

根据时间段获取股本数据get_gb_info_by_date →

根据时间段获取股本数据get_gb_info_by_date

获取指定股票的股本数据

```py
    def get_gb_info_by_date( stock_code:str = '',
                    start_date: str = '',
                    end_date: str = ''):
```

输入参数

参数	是否必选	参数类型	参数说明
stock_code	Y	str	股票代码
start_date	Y	str	开始日期
end_date	Y	str	截止日期

- 须通过客户端或refresh_kline下载对应股票的日K线数据

输出数据

名称	类型	数值	说明
Date	double		日期
Zgb	double		总股本
Ltgb	double		流通股本

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
gb_info_date = tq.get_gb_info_by_date(stock_code='688318.SH',
start_date='20260101', end_date='')
print(gb_info_date)
```

数据样本

```
[{'Date': 20260105, 'Ltgb': 256119392.0, 'Zgb': 256119392.0},
{'Date': 20260106, 'Ltgb': 256119392.0, 'Zgb': 256119392.0},
```

```
...,{'Date': 20260513, 'Ltgb': 256119392.0, 'Zgb': 256119392.0},
{'Date': 20260514, 'Ltgb': 256119392.0, 'Zgb': 256119392.0},
{'Date': 20260518, 'Ltgb': 256119392.0, 'Zgb': 256119392.0}]
```

← 获取每天的股本数据get_gb_info

获取专业财务数据get_financial_data →

财务类数据

获取专业财务数据get_financial_data

根据股票，获取指定时间段内的专业财务数据，与基础财务数据不同，需要先在客户端中下载专业财务数据

```py
get_financial_data(stock_list: List[str] = [],
                   field_list: List[str] = [],
                   start_time: str = '',
                   end_time: str = '',
                   report_type: str = 'report_time') -> Dict:
```

输入参数

参数	是否必选	参数类型	参数说明
stock_list	Y	List[str]	证券代码列表例如 `["600519.SH"]`
field_list	Y	List[str]	字段筛选，不能为空，字段名须与系统定义一致（如 `FN193` ）
start_time	Y	str	起始时间，格式 `YYYYMMDD` ，如 `'20250101'`
end_time	N	str	结束时间，格式 `YYYYMMDD` ，为空表示无结束限制
report_type	N	bool	按截止日期还是公告日期筛选，可选值： `'announce_time'` （按公告日期筛选）或 `'tag_time'` （按报告期筛选）

输出数据

名称	类型	数值	说明
announce_time	int		公告日期
tag_time	int		报告期
FN1	double		基本每股收益
FN2	double		扣除非经常性损益每股收益
FN3	double		每股未分配利润

名称	类型	数值	说明
FN4	double		每股净资产
FN5	double		每股资本公积金
FN6	double		净资产收益率
FN7	double		每股经营现金流量
FN8	double		货币资金
FN9	double		交易性金融资产
FN10	double		应收票据
FN11	double		应收账款
FN12	double		预付款项
FN13	double		其他应收款
FN14	double		应收关联公司款
FN15	double		应收利息
FN16	double		应收股利
FN17	double		存货
FN18	double		其中：消耗性生物资产
FN19	double		一年内到期的非流动资产
FN20	double		其他流动资产
FN21	double		流动资产合计
FN22	double		可供出售金融资产
FN23	double		持有至到期投资
FN24	double		长期应收款
FN25	double		长期股权投资
FN26	double		投资性房地产
FN27	double		固定资产
FN28	double		在建工程
FN29	double		工程物资
FN30	double		固定资产清理

名称	类型	数值	说明
FN31	double		生产性生物资产
FN32	double		油气资产
FN33	double		无形资产
FN34	double		开发支出
FN35	double		商誉
FN36	double		长期待摊费用
FN37	double		递延所得税资产
FN38	double		其他非流动资产
FN39	double		非流动资产合计
FN40	double		资产总计
FN41	double		短期借款
FN42	double		交易性金融负债
FN43	double		应付票据
FN44	double		应付账款
FN45	double		预收款项
FN46	double		应付职工薪酬
FN47	double		应交税费
FN48	double		应付利息
FN49	double		应付股利
FN50	double		其他应付款
FN51	double		应付关联公司款
FN52	double		一年内到期的非流动负债
FN53	double		其他流动负债
FN54	double		流动负债合计
FN55	double		长期借款
FN56	double		应付债券
FN57	double		长期应付款

名称	类型	数值	说明
FN58	double		专项应付款
FN59	double		预计负债 （非流动负债）
FN60	double		递延所得税负债
FN61	double		其他非流动负债
FN62	double		非流动负债合计
FN63	double		负债合计
FN64	double		实收资本（或股本）
FN65	double		资本公积
FN66	double		盈余公积
FN67	double		减：库存股
FN68	double		未分配利润
FN69	double		少数股东权益
FN70	double		外币报表折算价差
FN71	double		非正常经营项目收益调整
FN72	double		所有者权益（或股东权益）合计
FN73	double		负债和所有者（或股东权益）合计
FN98	double		销售商品、提供劳务收到的现金
FN99	double		收到的税费返还
FN100	double		收到其他与经营活动有关的现金
FN101	double		经营活动现金流入小计
FN102	double		购买商品、接受劳务支付的现金
FN103	double		支付给职工以及为职工支付的现金
FN104	double		支付的各项税费
FN105	double		支付其他与经营活动有关的现金
FN106	double		经营活动现金流出小计
FN107	double		经营活动产生的现金流量净额
FN108	double		收回投资收到的现金

名称	类型	数值	说明
FN109	double		取得投资收益收到的现金
FN110	double		处置固定资产、无形资产和其他长期资产收回的现金净额
FN111	double		处置子公司及其他营业单位收到的现金净额
FN112	double		收到其他与投资活动有关的现金
FN113	double		投资活动现金流入小计
FN114	double		购建固定资产、无形资产和其他长期资产支付的现金
FN115	double		投资支付的现金
FN116	double		取得子公司及其他营业单位支付的现金净额
FN117	double		支付其他与投资活动有关的现金
FN118	double		投资活动现金流出小计
FN119	double		投资活动产生的现金流量净额
FN120	double		吸收投资收到的现金
FN121	double		取得借款收到的现金
FN122	double		收到其他与筹资活动有关的现金
FN123	double		筹资活动现金流入小计
FN124	double		偿还债务支付的现金
FN125	double		分配股利、利润或偿付利息支付的现金
FN126	double		支付其他与筹资活动有关的现金
FN127	double		筹资活动现金流出小计
FN128	double		筹资活动产生的现金流量净额
FN129	double		四、汇率变动对现金的影响
FN130	double		四(2)、其他原因对现金的影响
FN131	double		五、现金及现金等价物净增加额
FN132	double		期初现金及现金等价物余额
FN133	double		期末现金及现金等价物余额

名称	类型	数值	说明
FN134	double		净利润
FN135	double		加：资产减值准备
FN136	double		固定资产折旧、油气资产折耗、生产性生物资产折旧
FN137	double		无形资产摊销
FN138	double		长期待摊费用摊销
FN139	double		处置固定资产、无形资产和其他长期资产的损失
FN140	double		固定资产报废损失
FN141	double		公允价值变动损失
FN142	double		财务费用
FN143	double		投资损失
FN144	double		递延所得税资产减少
FN145	double		递延所得税负债增加
FN146	double		存货的减少
FN147	double		经营性应收项目的减少
FN148	double		经营性应付项目的增加
FN149	double		其他
FN150	double		经营活动产生的现金流量净额2
FN151	double		债务转为资本
FN152	double		一年内到期的可转换公司债券
FN153	double		融资租入固定资产
FN154	double		现金的期末余额
FN155	double		减：现金的期初余额
FN156	double		加：现金等价物的期末余额
FN157	double		减：现金等价物的期初余额
FN158	double		现金及现金等价物净增加额
FN159	double		流动比率(非金融类指标)

名称	类型	数值	说明
FN160	double		速动比率(非金融类指标)
FN161	double		现金比率(%)(非金融类指标)
FN162	double		利息保障倍数(非金融类指标)
FN163	double		非流动负债比率(%)(非金融类指标)
FN164	double		流动负债比率(%)(非金融类指标)
FN166	double		有形资产净值债务率(%)
FN167	double		权益乘数(%)
FN168	double		股东的权益/负债合计(%)
FN169	double		有形资产/负债合计(%)
FN170	double		经营活动产生的现金流量净额/负债合计(%)(非金融类指标)
FN171	double		EBITDA/负债合计(%)(非金融类指标)
FN172	double		应收帐款周转率(非金融类指标)
FN173	double		存货周转率(非金融类指标)
FN174	double		运营资金周转率(非金融类指标)
FN175	double		总资产周转率(非金融类指标)
FN176	double		固定资产周转率(非金融类指标)
FN177	double		应收帐款周转天数(非金融类指标)
FN178	double		存货周转天数(非金融类指标)
FN179	double		流动资产周转率(非金融类指标)
FN180	double		流动资产周转天数(非金融类指标)
FN181	double		总资产周转天数(非金融类指标)
FN182	double		股东权益周转率(非金融类指标)
FN183	double		营业收入增长率(%)
FN184	double		净利润增长率(%)
FN185	double		净资产增长率(%)
FN186	double		固定资产增长率(%)
FN187	double		总资产增长率(%)

名称	类型	数值	说明
FN188	double		投资收益增长率(%)
FN189	double		营业利润增长率(%)
FN190	double		扣非每股收益同比(%)
FN191	double		扣非净利润同比(%)
FN192	double		暂无
FN193	double		成本费用利润率(%)
FN194	double		营业利润率(非金融类指标)
FN195	double		营业税金率(非金融类指标)
FN196	double		营业成本率(非金融类指标)
FN197	double		净资产收益率
FN198	double		投资收益率
FN199	double		销售净利率(%)
FN200	double		总资产净利率
FN201	double		净利润率(非金融类指标)
FN202	double		销售毛利率(%)(非金融类指标)
FN203	double		三费比重(非金融类指标)
FN204	double		管理费用率(非金融类指标)
FN205	double		财务费用率(非金融类指标)
FN206	double		扣除非经常性损益后的净利润
FN207	double		息税前利润(EBIT)
FN208	double		息税折旧摊销前利润(EBITDA)
FN209	double		EBITDA/营业总收入(%)(非金融类指标)
FN210	double		资产负债率(%)
FN211	double		流动资产比率(非金融类指标)
FN212	double		货币资金比率(非金融类指标)
FN213	double		存货比率(非金融类指标)
FN214	double		固定资产比率

名称	类型	数值	说明
FN215	double		负债结构比(非金融类指标)
FN216	double		归属于母公司股东权益/全部投入资本(%)
FN217	double		股东的权益/带息债务(%)
FN218	double		有形资产/净债务(%)
FN219	double		每股经营性现金流(元)
FN220	double		营业收入现金含量(%)(非金融类指标)
FN221	double		经营活动产生的现金流量净额/经营活动净收益(%)
FN222	double		销售商品提供劳务收到的现金/营业收入(%)
FN223	double		经营活动产生的现金流量净额/营业收入
FN224	double		资本支出/折旧和摊销
FN225	double		每股现金流量净额(元)
FN226	double		经营净现金比率（短期债务）(非金融类指标)
FN227	double		经营净现金比率（全部债务）
FN228	double		经营活动现金净流量与净利润比率
FN229	double		全部资产现金回收率
FN230	double		营业收入
FN231	double		营业利润
FN232	double		归属于母公司所有者的净利润
FN233	double		扣除非经常性损益后的净利润
FN234	double		经营活动产生的现金流量净额
FN235	double		投资活动产生的现金流量净额
FN236	double		筹资活动产生的现金流量净额
FN237	double		现金及现金等价物净增加额
FN238	double		总股本
FN239	double		已上市流通A股
FN240	double		已上市流通B股

名称	类型	数值	说明
FN241	double		已上市流通H股
FN242	double		股东人数(户)
FN243	double		第一大股东的持股数量
FN244	double		十大流通股东持股数量合计(股)
FN245	double		十大股东持股数量合计(股)
FN246	double		机构总量（家）
FN247	double		机构持股总量(股)
FN248	double		QFII机构数
FN249	double		QFII持股量
FN250	double		券商机构数
FN251	double		券商持股量
FN252	double		保险机构数
FN253	double		保险持股量
FN254	double		基金机构数
FN255	double		基金持股量
FN256	double		社保机构数
FN257	double		社保持股量
FN258	double		私募机构数
FN259	double		私募持股量
FN260	double		财务公司机构数
FN261	double		财务公司持股量
FN262	double		年金机构数
FN263	double		年金持股量
FN264	double		十大流通股东持有的流通A股合计(股)[注：2019半年报之前，季度报告中，若股东持股除了流通A股、还有流通B股或流通H股，指标264取的是包含流通B股或流通H股的流通股数]
FN265	double		第一大流通股东持股量(股)

名称	类型	数值	说明
FN266	double		自由流通股(股)[注：1.自由流通股=已流通A股-持股5%以上股东的流通A股（一致行动人算一起）；2.指标按报告期展示，新股在上市日的下个报告期才有数据]
FN267	double		受限流通A股(股)
FN268	double		一般风险准备(金融类)
FN269	double		其他综合收益(利润表)
FN270	double		综合收益总额(利润表)
FN271	double		归属于母公司股东权益(资产负债表)
FN272	double		银行机构数(家)(机构持股)
FN273	double		银行持股量(股)(机构持股)
FN274	double		一般法人机构数(家)(机构持股)
FN275	double		一般法人持股量(股)(机构持股)
FN276	double		近一年净利润(元)
FN277	double		信托机构数(家)(机构持股)
FN278	double		信托持股量(股)(机构持股)
FN279	double		特殊法人机构数(家)(机构持股)
FN280	double		特殊法人持股量(股)(机构持股)
FN281	double		加权净资产收益率(每股指标)
FN282	double		扣非每股收益(单季度财务指标)
FN283	double		最近一年营业收入(万元)
FN284	double		国家队持股数量（万股)[注：本指标统计包含汇金公司、证金公司、外汇管理局旗下投资平台、国家队基金、国开、养老金以及中科汇通等国家队机构持股数量]
FN285	double		业绩预告-本期归母净利润同比增幅下限%[注：指标285至294展示未来一个报告期的数据。例，3月31日至6月29日这段时间内展示的是中报的数据；如果最新的财务报告后面有多个报告期的业绩预告/快报，只能展示最新的财务报告后面的一个报告期的业绩预告/快报]

名称	类型	数值	说明
FN286	double		业绩预告-本期归母净利润同比增幅上限%
FN287	double		业绩快报-归母净利润
FN288	double		业绩快报-扣非净利润
FN289	double		业绩快报-总资产
FN290	double		业绩快报-净资产
FN291	double		业绩快报-每股收益
FN292	double		业绩快报-摊薄净资产收益率
FN293	double		业绩快报-加权净资产收益率
FN294	double		业绩快报-每股净资产
FN295	double		应付票据及应付账款(资产负债表)
FN296	double		应收票据及应收账款(资产负债表)
FN297	double		递延收益(资产负债表-非流动负债)
FN298	double		其他综合收益(资产负债表)
FN299	double		其他权益工具(资产负债表)
FN300	double		其他收益(利润表)
FN301	double		资产处置收益(利润表)
FN302	double		持续经营净利润(利润表)
FN303	double		终止经营净利润(利润表)
FN304	double		研发费用(利润表)
FN305	double		其中:利息费用(利润表-财务费用)
FN306	double		其中:利息收入(利润表-财务费用)
FN307	double		近一年经营活动现金流净额
FN308	double		近一年归母净利润(万元)
FN309	double		近一年扣非净利润(万元)
FN310	double		近一年现金净流量(万元)
FN311	double		基本每股收益（单季度）
FN312	double		营业总收入(单季度)(万元)

名称	类型	数值	说明
FN313	double		业绩预告公告日期 [注：本指标展示未来一个报告期的数据。例,3月31日至6月29日这段时间内展示的是中报的数据；如果最新的财务报告后面有多个报告期的业绩预告/快报，只能展示最新的财务报告后面的一个报告期的业绩预告/快报的数据；公告日期格式为YYMMDD，例：190101代表2019年1月1日]
FN314	double		财报公告日期 [注：日期格式为YYMMDD,例：190101代表2019年1月1日]
FN315	double		业绩快报公告日期 [注：本指标展示未来一个报告期的数据。例,3月31日至6月29日这段时间内展示的是中报的数据；如果最新的财务报告后面有多个报告期的业绩预告/快报，只能展示最新的财务报告后面的一个报告期的业绩预告/快报的数据；公告日期格式为YYMMDD，例：190101代表2019年1月1日]
FN316	double		近一年投资活动现金流净额(万元)
FN317	double		业绩预告-本期归母净利润下限(万元) [注：指标317至318展示未来一个报告期的数据。例，3月31日至6月29日这段时间内展示的是中报的数据；如果最新的财务报告后面有多个报告期的业绩预告/快报，只能展示最新的财务报告后面的一个报告期的业绩预告/快报]
FN318	double		业绩预告-本期归母净利润上限(万元)
FN319	double		营业总收入TTM(万元)
FN320	double		员工总数(人)
FN321	double		每股企业自由现金流
FN322	double		每股股东自由现金流
FN323	double		近一年营业利润(万元)
FN324	double		净利润（单季度）(万元)
FN325	double		北上资金数（家）(机构持股)
FN326	double		北上资金持股量（股）(机构持股)
FN327	double		有息负债率

名称	类型	数值	说明
FN328	double		营业成本（单季度）(万元)
FN329	double		投入资本回报率（ROIC）(获利能力分析)
FN330	double		业绩快报-营业收入（本期）
FN331	double		业绩快报-营业收入（上期）
FN332	double		业绩快报-营业利润（本期）
FN333	double		业绩快报-营业利润（上期）
FN334	double		业绩快报-利润总额（本期）
FN335	double		业绩快报-利润总额（上期）
FN336	double		审计意见 [注：0-未审计,1-无保留意见,2-带强调事项段的无保留意见,3-保留意见,4-无法表示意见,5-否定意见及其他]
FN337	double		股利支付率（%）
FN338	double		近一年营业成本-非金融类(万元)
FN339	double		近一年营业成本-金融类(万元)
FN340	double		业绩预告-本期扣非后净利润下限(万元)
FN341	double		业绩预告-本期扣非后净利润上限(万元)
FN342	double		业绩预告-本期扣非后净利润同比增长下限（%）
FN343	double		业绩预告-本期扣非后净利润同比增长上限（%）
FN344	double		业绩预告-预告基本每股收益下限(元)
FN345	double		业绩预告-预告基本每股收益上限(元)
FN346	double		业绩预告-预告基本每股收益同比增长下限（%）
FN347	double		业绩预告-预告基本每股收益同比增长上限（%）
FN348	double		业绩预告-预告扣非后基本每股收益下限(元)
FN349	double		业绩预告-预告扣非后基本每股收益上限(元)

名称	类型	数值	说明
FN350	double		业绩预告-预告扣非后基本每股收益同比增长下限（%）
FN351	double		业绩预告-预告扣非后基本每股收益同比增长上限（%）
FN352	double		业绩预告-预告营业收入下限(万元)
FN353	double		业绩预告-预告营业收入上限(万元)
FN354	double		业绩预告-预告营业收入同比增长下限（%）
FN355	double		业绩预告-预告营业收入同比增长上限（%）
FN356	double		业绩预告-预告扣除后营业收入下限(万元)
FN357	double		业绩预告-预告扣除后营业收入上限(万元)
FN358	double		主营业务收入(内销)(万元)
FN359	double		主营业务收入(外销)(万元)
FN360	double		资管计划机构数(家)
FN361	double		资管计划持股量(股)
FN362	double		财务总评分
FN401	double		专项储备(万元)
FN402	double		结算备付金(万元)
FN403	double		拆出资金(万元)
FN404	double		发放贷款及垫款(万元)(流动资产科目)
FN405	double		衍生金融资产(万元)
FN406	double		应收保费(万元)
FN407	double		应收分保账款(万元)
FN408	double		应收分保合同准备金(万元)
FN409	double		买入返售金融资产(万元)
FN410	double		划分为持有待售的资产(万元)
FN411	double		发放贷款及垫款(万元)(非流动资产科目)

名称	类型	数值	说明
FN412	double		向中央银行借款(万元)
FN413	double		吸收存款及同业存放(万元)
FN414	double		拆入资金(万元)
FN415	double		衍生金融负债(万元)
FN416	double		卖出回购金融资产款(万元)
FN417	double		应付手续费及佣金(万元)
FN418	double		应付分保账款(万元)
FN419	double		保险合同准备金(万元)
FN420	double		代理买卖证券款(万元)
FN421	double		代理承销证券款(万元)
FN422	double		划分为持有待售的负债(万元)
FN423	double		预计负债(万元) （流动负债）
FN424	double		递延收益(万元)（流动负债科目，公告此科目的股票较少，大部分公司没有此数据）
FN425	double		其中:优先股(万元)(非流动负债科目)
FN426	double		永续债(万元)(非流动负债科目)
FN427	double		长期应付职工薪酬(万元)
FN428	double		其中:优先股(万元)(所有者权益科目)
FN429	double		永续债(万元)(所有者权益科目)
FN430	double		债权投资(万元)
FN431	double		其他债权投资(万元)
FN432	double		其他权益工具投资(万元)
FN433	double		其他非流动金融资产(万元)
FN434	double		合同负债(万元)
FN435	double		合同资产(万元)
FN436	double		其他资产(万元)
FN437	double		应收款项融资(万元)

名称	类型	数值	说明
FN438	double		使用权资产(万元)
FN439	double		租赁负债(万元)
FN440	double		发放贷款及垫款(万元) [注：金融类科目]
FN441	double		应收款项(万元) [注：证券类指标]
FN442	double		存出保证金(万元) [注：证券类指标]
FN443	double		现金及存放中央银行款项(万元) [注：金融类科目]
FN444	double		贵金属(万元) [注：金融类科目]
FN445	double		以公允价值计量且其变动计入当期损益的金融资产(万元) [注：金融类科目]
FN446	double		代理业务资产(万元) [注：金融类科目]
FN447	double		应收款项类投资(万元) [注：金融类科目]
FN448	double		同业及其它金融机构存放款项(万元) [注：金融类科目]
FN449	double		以公允价值计量且其变动计入当期损益的金融负债(万元) [注：金融类科目]
FN450	double		吸收存款(万元) [注：金融类科目]
FN451	double		代理业务负债(万元) [注：金融类科目]
FN452	double		其他负债(万元) [注：金融类科目]
FN453	double		发放贷款及垫款(万元) [注：金融类科目]
FN501	double		稀释每股收益(元)
FN502	double		营业总收入(万元)
FN503	double		汇兑收益(万元)
FN504	double		其中:归属于母公司综合收益(万元)
FN505	double		其中:归属于少数股东综合收益(万元)
FN506	double		利息收入(万元)
FN507	double		已赚保费(万元)
FN508	double		手续费及佣金收入(万元)
FN509	double		利息支出(万元)

名称	类型	数值	说明
FN510	double		手续费及佣金支出(万元)
FN511	double		退保金(万元)
FN512	double		赔付支出净额(万元)
FN513	double		提取保险合同准备金净额(万元)
FN514	double		保单红利支出(万元)
FN515	double		分保费用(万元)
FN516	double		其中:非流动资产处置利得(万元)
FN517	double		信用减值损失(万元)
FN518	double		净敞口套期收益(万元)
FN519	double		营业总成本(万元)
FN520	double		信用减值损失(万元、2019格式)
FN521	double		资产减值损失(万元、2019格式)
FN522	double		其他业务收入(万元) [注：金融类科目]
FN523	double		业务及管理费(万元) [注：金融类科目]
FN524	double		其他业务成本(万元) [注：金融类科目]
FN561	double		加:其他原因对现金的影响2(万元)(现金的期末余额科目)
FN562	double		客户存款和同业存放款项净增加额(万元)
FN563	double		向中央银行借款净增加额(万元)
FN564	double		向其他金融机构拆入资金净增加额(万元)
FN565	double		收到原保险合同保费取得的现金(万元)
FN566	double		收到再保险业务现金净额(万元)
FN567	double		保户储金及投资款净增加额(万元)
FN568	double		处置以公允价值计量且其变动计入当期损益的金融资产净增加额(万元)
FN569	double		收取利息、手续费及佣金的现金(万元)
FN570	double		拆入资金净增加额(万元)
FN571	double		回购业务资金净增加额(万元)

名称	类型	数值	说明
FN572	double		客户贷款及垫款净增加额(万元)
FN573	double		存放中央银行和同业款项净增加额(万元)
FN574	double		支付原保险合同赔付款项的现金(万元)
FN575	double		支付利息、手续费及佣金的现金(万元)
FN576	double		支付保单红利的现金(万元)
FN577	double		其中:子公司吸收少数股东投资收到的现金(万元)
FN578	double		其中:子公司支付给少数股东的股利、利润(万元)
FN579	double		投资性房地产的折旧及摊销(万元)
FN580	double		信用减值损失(万元)
FN581	double		使用权资产折旧（万元）
FN582	double		收取利息和手续费净增加额(万元) [注：金融类科目]
FN583	double		支付手续费的现金(万元) [注：金融类科目]
FN584	double		发行债券支付的现金(万元) [注：金融类科目]

返回值说明

- **返回类型**：`dict`，键为股票代码（如 `'600519.SH'`），值为 **pandas.DataFrame**。
- **DataFrame 列**：
  - 用户请求的财务字段（如 `FN193`, `FN194` ... 大写）。
  - `announce_time`：公告日期，格式 `YYYYMMDD`。
  - `tag_time`：报告期截止日期，格式 `YYYYMMDD`。
- **行**：按时间顺序排列的财务数据记录。

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)

fd = tq.get_financial_data(
        stock_list=['688318.SH'],
        field_list=['Fn193','Fn194','Fn195','Fn196','Fn197'],
        start_time='20250101',
```

```
        end_time='',
        report_type='announce_time')
print(fd)
```

数据样本

```
{'600519.SH':     FN193  FN194  FN195 FN196  FN197 announce_time  tag_time
0  164.82  70.03  15.76  8.07  36.99       20250403  20241231
1  193.43  73.19  14.16  8.03  10.39       20250430  20250331
2  166.69  70.22  15.60  8.70  19.02       20250813  20250630
3  162.47  69.67  16.07  8.71  25.14       20251030  20250930}
```

← 根据时间段获取股本数据get_gb_info_by_date 获取指定日期财务数据get_financial_data_by_date →

获取指定日期专业财务数据get_financial_data_by_date

根据股票，获取指定日期的专业财务数据，与基础财务数据不同，需要先在客户端中下载专业财务数据

```py
get_financial_data_by_date(stock_list: List[str] = [],
                           field_list: List[str] = [],
                           year: int = 0,
                           mmdd: int = 0) -> Dict:
```

输入参数

参数	是否必选	参数类型	参数说明
stock_list	Y	List[str]	证券代码列表
field_list	Y	List[str]	字段筛选，不能为空（如 `FN193` ）
year	Y	int	指定年份
mmdd	Y	int	指定月日

- 如果year和mmdd都为0,表示最新的财报;
- 如果year为0,mmdd为小于300的数字,表示最近一期向前推mmdd期的数据,如果是331,630,930,1231这些,表示最近一期的对应季报的数据;
- 如果mmdd为0,year为一数字,表示最近一期向前推year年的同期数据;
- 季报分界点为:0331,0630,0930,1231
- 需要先在客户端中下载财务数据包

输出数据

同get_financial_data一样。

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)

fd = tq.get_financial_data_by_date(
        stock_list=['688318.SH'],
        field_list=['Fn193','Fn194','Fn195','Fn196','Fn197'],
        year=0,
```

```
        mmdd=0)
print(fd)
```

数据样本

```
{'600519.SH':
{'FN193': '162.47',
'FN194': '69.67',
'FN195': '16.07',
'FN196': '8.71',
'FN197': '25.14'}}
```

← 获取专业财务数据get_financial_data

获取股票交易数据get_gpjy_value →

获取股票交易数据get_gpjy_value

根据股票，获取指定时间段内的股票交易数据，需要先在客户端中下载股票数据包

```py
get_gpjy_value(stock_list: List[str] = [],
                    field_list: List[str] = [],
                    start_time: str = '',
                    end_time: str = '') -> Dict:
```

输入参数

参数	是否必选	参数类型	参数说明
stock_list	Y	List[str]	证券代码列表
field_list	Y	List[str]	字段筛选，不能为空
start_time	N	str	起始时间
end_time	N	str	结束时间

输出数据

名称	类型	数值	说明
GP01	double		股东人数 股东户数(户)
GP02	double		龙虎榜 买入总计(万元) 卖出总计(万元)[注：该指标展示20230717日之后的数据]
GP03	double		融资融券1 融资余额(万元) 融券余量(股)
GP04	double		大宗交易 成交均价(元) 成交额(万元)
GP05	double		增减持1 成交均价(元) 变动股数(股)
GP06	double		陆股通持股量 持股数量(股)[注：该指标展示20170317日之后的数据]
GP07	double		陆股通市场成交净额 陆股通市场净买入(万元)[注：官方只公布了每日的前十名数据]
GP08	double		龙虎榜机构(卖方)数据 卖方机构个数 机构卖出金额(万元)
GP09	double		龙虎榜机构(买方)数据 买方机构个数 机构买入金额(万元)

名称	类型	数值	说明
GP10	double		近3月机构调研情况 近3月机构调研次数 近3月调研机构数量
GP11	double		融资融券2 融资买入额(万元) 融资偿还额(万元)
GP12	double		融资融券3 融券卖出量(股) 融券偿还量(股)
GP13	double		融资融券4 融资净买入(万元) 融券净卖出(股)
GP14	double		涨停数据 涨停金额(即板上成交,万元) 开板次数[注：该指标展示20180319日之后的数据]
GP15	double		涨跌停 涨跌停状态 封单金额(万元)[注：涨停取2,曾涨停取1,跌停取-2,曾跌停取-1;跌停和曾跌停时,封单金额取负值 该指标展示20160926日之后的数据]
GP16	double		总市值 总市值(万元)
GP17	double		龙虎榜营业部数据 买入金额(万元) 卖出金额(万元)
GP18	double		龙虎榜沪深股通数据 买入金额(万元) 卖出金额(万元)
GP19	double		每周股票质押数量 无限售股份质押数(万) 有限售股份质押数(万)[注：该指标展示20180316日之后的数据]
GP20	double		每周股票质押比例 质押比例(%)[注：该指标展示20180316日之后的数据]
GP21	double		股息率 股息率(%)
GP22	double		涨跌停 封成比 封流比[注：该指标展示20180319日之后的数据]
GP23	double		拟增减持 拟增持数量(万股) 拟减持数量(万股)
GP24	double		涨停 首次涨停时间 涨停最大封单额(万) [注：首次涨停时间展示20160301之后的数据，涨停最大封单额展示20200730之后的数据]
GP25	double		盘前盘后成交量 开盘成交量(手) 盘后固定成交量(手) [注：盘后固定成交量只包含科创板和创业板]
GP26	double		拟增减持金额 拟增持金额(万元) 拟减持金额(万元)
GP27	double		人气排名 市场人气排名 行业人气排名 [注：行业排名为通达信二级研究行业排名]
GP28	double		股票回购 回购均价(元) 回购数量(万股)
GP29	double		证券信息 是否复牌日 是否更名日 [注：是否复牌日说明：0-不是复牌日，n(n>0)-停牌n个交易日之后的复牌日；是否更名日说明：0-未更名，1-常规更名，2-加ST，3-加*ST，4-摘帽，5-其他]

名称	类型	数值	说明
GP30	double		分红送转 派息金额(万元) 送转数量(股) [注：对应展示日期为除权除息日]
GP31	double		转融券 期初余量(股) 期末余量(股)
GP32	double		转融券 融出数量(股) 融出市值(元)
GP33	double		跌停数据 跌停金额(万元) 开板次数 [注：该指标展示20180319日之后的数据,暂无跌停金额数据]
GP34	double		跌停 首次跌停时间 跌停最大封单额(万) [注：首次跌停时间展示20160301之后的数据，跌停最大封单额展示20200730之后的数据]
GP35	double		增减持2 增持数量(股) 减持数量(股)
GP36	double		竞价涨停买 买入金额(万元) [注：该指标展示20241101日之后的数据]
GP37	double		龙虎榜2 上榜类型连续交易日(天) [注：该指标展示上榜类型中指代的连续交易日类型]
GP38	double		涨停相关1 近1年涨停次数 近1年溢价5%次数
GP39	double		涨停相关2 近1年首板封板率(%) 近1年次日红盘率(%)
GP40	double		涨停相关3 近1年连板率(%) 最后涨停时间
GP41	double		股权登记日 配股股权登记日
GP42	double		龙虎榜专业机构买卖净额 买方成交净额(万元) 卖方成交净额(万元)
GP43	double		配股实施 配股价格(元) 配股数量(万股)
GP44	double		股票评分 综合评分
GP45	double		评级系数 评级系数
GP46	double		拟询价转让 拟转让股数(万股) 拟转让占总股本(%)

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)

gp_val = tq.get_gpjy_value(
        stock_list=['688318.SH'],
        field_list=['GP1','GP2','GP3','GP4','GP5'],
        start_time='20250101',
        end_time='20250102')
```

```
print(gp_val)

```

数据样本

```
{'688318.SH': {'GP3': [{'Date': '20250102', 'Value': ['141405.89',
'11113.00']}]}}
```

← 获取指定日期财务数据get_financial_data_by_date

获取指定日期股票交易数据get_gpjy_value_by_date →

获取指定日期股票交易数据get_gpjy_value_by_date

根据股票，获取指定时间段内的股票交易数据，需要先在客户端中下载股票数据包

```py
def get_gpjy_value_by_date(stock_list: List[str] = [],
                           field_list: List[str] = [],
                           year: int = 0,
                           mmdd: int = 0) -> Dict:
```

输入参数

参数	是否必选	参数类型	参数说明
stock_list	Y	List[str]	证券代码列表
field_list	Y	List[str]	字段筛选，不能为空
year	Y	int	指定年份
mmdd	Y	int	指定月日

- 如果year为0,mmdd为0,表示最新数据,mmdd为1,2,3...,表示倒数第2,3,4...个数据。
- 需要先在客户端中下载股票数据包

输出数据

同get_gpjy_value一样。

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)

gp_one = tq.get_gpjy_value_by_date(
        stock_list=['688318.SH'],
        field_list=['GP1','GP2','GP3','GP4','GP5'],
        year=0,mmdd=0)
print(gp_one)
```

数据样本

```
{'688318.SH': {'GP1': ['24154.00', '0.00'], 'GP2': ['20574.12', '18728.85'],
'GP3': ['140464.83', '55043.00'], 'GP4': ['169.80', '5943.00'], 'GP5':
['103.00', '-7000.00']}}
```

← 获取股票交易数据get_gpjy_value

获取板块交易数据get_bkjy_value →

获取板块交易数据get_bkjy_value

根据板块代码，获取指定时间段内的板块交易数据，需要先在客户端中下载股票数据包

```py
get_bkjy_value(stock_list: List[str] = [],
                    field_list: List[str] = [],
                    start_time: str = '',
                    end_time: str = '') -> Dict:
```

输入参数

参数	是否必选	参数类型	参数说明
stock_list	Y	List[str]	证券代码列表
field_list	Y	List[str]	字段筛选，不能为空
start_time	N	str	起始时间
end_time	N	str	结束时间

输出数据

名称	类型	数值	说明
BK5	double		市盈率TTM 整体法 算术平均
BK6	double		市净率MRQ 整体法 算术平均
BK7	double		市销率TTM 整体法 算术平均
BK8	double		市现率TTM 整体法 算术平均
BK9	double		涨跌数 上涨家数 下跌家数
BK10	double		板块总市值(亿元) 整体法 算术平均
BK11	double		板块流通市值(亿元) 整体法 算术平均
BK12	double		涨停数 涨停家数 曾涨停家数[注：该指标展示20160926日之后的数据]
BK13	double		跌停数 跌停家数 曾跌停家数[注：该指标展示20160926日之后的数据]

名称	类型	数值	说明
BK14	double		涨停数据 市场高度(不含ST股和未开板新股) 2板及以上涨停个数(不含ST股和未开板新股)[注：该指标展示20180319日之后的数据]
BK15	double		融资融券 沪深京融资余额(万元) 沪深京融券余额(万元)
BK16	double		陆股通资金流入 沪股通流入金额(亿元) 深股通流入金额(亿元)[注：该指标展示20170320日之后的数据]
BK17	double		开盘成交数 开盘成交额(万元) 开盘成交量(万股)
BK18	double		板块股息率(%) 算数平均 整体法
BK19	double		板块自由流通市值(亿元) 整体法 算术平均

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)

bk_data = tq.get_bkjy_value(stock_list=['880660.SH'],
        field_list=['BK5','BK6','BK7','BK8','BK9'],
        start_time='20250101',
        end_time='20250102')
print(bk_data)
```

数据样本

```
{'880660.SH': {'BK5': [{'Date': '20250102', 'Value': ['55.28', '55.50']}],
'BK6': [{'Date': '20250102', 'Value': ['4.62', '3.79']}],
'BK7': [{'Date': '20250102', 'Value': ['5.25', '8.22']}],
'BK8': [{'Date': '20250102', 'Value': ['46.52', '312.41']}],
'BK9': [{'Date': '20250102', 'Value': ['0.00', '35.00']}, {'Date': '20260130',
'Value': ['10.00', '25.00']}]}}
```

← 获取指定日期股票交易数据get_gpjy_value_by_date

获取指定日期板块交易数据get_bkjy_value_by_date →

获取指定日期板块交易数据get_bkjy_value_by_date

根据板块代码，获取指定日期的板块交易数据，需要先在客户端中下载股票数据包

```py
get_bkjy_value_by_date(stock_list: List[str] = [],
                                    field_list: List[str] = [],
                                    year: int = 0,
                                    mmdd: int = 0) -> Dict:
```

输入参数

参数	是否必选	参数类型	参数说明
stock_list	Y	List[str]	证券代码列表
field_list	Y	List[str]	字段筛选，不能为空
year	Y	int	指定年份
mmdd	Y	int	指定月日

- 如果year为0,mmdd为0,表示最新数据,mmdd为1,2,3...,表示倒数第2,3,4...个数据。
- 需要先在客户端中下载股票数据包

输出数据

同get_bkjy_value一样。

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)

bk_one = tq.get_bkjy_value_by_date(stock_list=['880660.SH'],
                                    field_list=
['BK9','BK10','BK11','BK12','BK13'],
                                    year=0,mmdd=0)
print(bk_one)
```

数据样本

```
{'880660.SH': {'BK10': ['6705.83', '191.60'], 'BK11': ['6183.65', '176.68'],
'BK12': ['0.00', '0.00'], 'BK13': ['0.00', '0.00'], 'BK9': ['3.00', '31.00']}}
```

← 获取板块交易数据get_bkjy_value

获取市场交易数据get_scjy_value →

获取市场交易数据

获取指定时间段内的市场交易数据，需要先在客户端中下载股票数据包

```py
get_scjy_value(field_list: List[str] = [],
                    start_time: str = '',
                    end_time: str = '') -> Dict:
```

输入参数

参数	是否必选	参数类型	参数说明
field_list	Y	List[str]	字段筛选，不能为空
start_time	N	str	起始时间
end_time	N	str	结束时间

输出数据

名称	类型	数值	说明
SC01	double		融资融券 沪深京融资余额(万元) 沪深京融券余额(万元)
SC02	double		陆股通资金流入 沪股通流入金额(亿元) 深股通流入金额(亿元) [注：沪股通限制展示2000条数据，深股通展示自20161205以后的数据]
SC03	double		沪深京涨停股个数 涨停股个数 曾涨停股个数 [注：该指标展示20160926日之后的数据]
SC04	double		沪深京跌停股个数 跌停股个数 曾跌停股个数 [注：该指标展示20160926日之后的数据]
SC05	double		上证50股指期货 净持仓(手)[注：该指标展示20171009日之后的数据]
SC06	double		沪深300股指期货 净持仓(手) [注：该指标展示20171009日之后的数据]
SC07	double		中证500股指期货 净持仓(手) [注：该指标展示20171009日之后的数据]
SC08	double		ETF基金规模份额数据 ETF基金规模(亿份) ETF净申赎(亿份)

名称	类型	数值	说明
SC09	double		沪月新开A股账户 沪月新开A股账户(万户)
SC10	double		增减持统计 增持额(万元) 减持额(万元)[注：部分公司公告滞后,造成每天查看的数据可能会不一样]
SC11	double		大宗交易 溢价的大宗交易额(万元) 折价的大宗交易额(万元)
SC12	double		限售解禁 限售解禁计划额(亿元) 限售解禁股份实际上市金额(亿元)[注：该指标展示201802月之后的数据;部分股票的解禁日期延后，造成不同日期提取的某天的计划额可能不同]
SC13	double		分红 市场总分红额(亿元)[注：除权派息日的A股市场总分红额]
SC14	double		募资 市场总募资额(亿元)[注：发行日期/除权日期的首发、配股和增发的总募资额]
SC15	double		打板资金 封板成功资金(亿元) 封板失败资金(亿元) [注：该指标展示20160926日之后的数据]
SC16	double		龙虎榜 买入总金额(亿元) 卖出总金额(亿元)
SC17	double		龙虎榜机构数据 买入金额(亿元) 卖出金额(亿元)
SC18	double		龙虎榜营业部数据 买入金额(亿元) 卖出金额(亿元)
SC19	double		龙虎榜沪深股通数据 买入金额(亿元) 卖出金额(亿元)
SC20	double		陆股通净买入 沪股通净买入额(亿元) 深股通净买入额(亿元)
SC21	double		每周无限售质押率 深市质押率(%) 沪市质押率(%)[注：该指标展示20180128日之后的数据]
SC22	double		每周有限售质押率 深市质押率(%) 沪市质押率(%)[注：该指标展示20180128日之后的数据]
SC23	double		连板家数 连板股个数(包含ST和未开板新股) 连板股个数(不含ST股和未开板新股）[注：该指标展示20180319日之后的数据]
SC24	double		沪深京涨跌停股个数 涨停股个数(不含ST股和未开板新股) 跌停股个数（不含ST股）[注：该指标展示20160926日之后的数据]
SC25	double		融资融券 沪深京融资买入额（万元）沪深京融券卖出量（万股）
SC26	double		每周市场质押比 每周市场质押比例（%）[注：该指标展示20180316日之后的数据]
SC27	double		央行公开市场净投放 央行公开市场净投放 (亿元)
SC28	double		历史A股新高新低数 历史新高A股股票个数 历史新低A股股票个数(上市满一年的股票)
SC29	double		120天A股新高新低数 120天新高A股股票个数 120天新低A股股票个数(上市满一年的股票)

名称	类型	数值	说明
SC30	double		涨停数据 市场高度(不含ST股和未开板新股) 2板以上涨停个数(不含ST股和未开板新股)[注：该指标展示20180319日之后的数据]
SC31	double		涨跌家数 涨家数（剔除停牌） 跌家数（剔除停牌）
SC32	double		20天A股新高新低数 20天新高A股股票个数 20天新低A股股票个数(上市满一年的股票)
SC33	double		市场总封单金额 涨停封单金额（亿元）跌停封单金额（亿元）[注：该指标展示20160926日之后的数据]
SC34	double		涨跌股成交量 上涨股成交量(万手) 下跌股成交量(万手)
SC35	double		涨停数据 换手板家数 回封率(%) [注：两个指标都剔除了未开板新股，换手板家数展示20190605日之后的数据，回封率展示20180927日之后的数据]
SC36	double		曾涨跌停股个数 曾涨停股个数(剔除ST股和未开板新股) 曾跌停股个数(剔除ST股) [注：该指标展示20160926日之后的数据]
SC37	double		转融券 融出市值(亿元) 期末余额(亿元)
SC38	double		ETF基金规模金额数据 ETF基金规模(亿元) ETF净申赎(亿元)
SC39	double		涨跌5%家数 涨幅大于等于5%家数 跌幅大于等于5%家数
SC40	double		陆股通成交 陆股通成交总额(亿元) 陆股通成交总笔(万笔)
SC41	double		中证1000股指期货 净持仓(手) [注：该指标展示20220722日之后的数据]
SC42	double		沪深股通成交金额 沪股通成交总额(亿元) 深股通成交总额(亿元)

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)

sc_val = tq.get_scjy_value(field_list=['SC1','SC2','SC3','SC4','SC5'],
        start_time='20250101',end_time='20250102')
print(sc_val)
```

数据样本

```
{'SC1': [{'Date': '20250102', 'Value': ['184712288.00', '999820.06']}],
'SC2': [{'Date': '20250102', 'Value': ['0.00', '0.00']}],
'SC3': [{'Date': '20250102', 'Value': ['67.00', '49.00']}],
'SC4': [{'Date': '20250102', 'Value': ['32.00', '30.00']}],
```

```
'SC5': [{'Date': '20250102', 'Value': ['-21204.00', '0.00']}]}
```

← 获取指定日期板块交易数据get_bkjy_value_by_date

获取指定日期市场交易数据get_scjy_value_by_date →

获取指定日期市场交易数据get_scjy_value_by_date

获取指定时间的市场交易数据，需要先在客户端中下载股票数据包

```py
get_scjy_value_by_date(field_list: List[str] = [],
                               year: int = 0,
                               mmdd: int = 0) -> Dict:
```

输入参数

参数	是否必选	参数类型	参数说明
field_list	Y	List[str]	字段筛选，不能为空
year	Y	int	指定年份
mmdd	Y	int	指定月日

- 如果year为0,mmdd为0,表示最新数据,mmdd为1,2,3...,表示倒数第2,3,4...个数据。
- 需要先在客户端中下载股票数据包

输出数据

同get_scjy_value一样。

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)

sc_one = tq.get_scjy_value_by_date(field_list=
['SC6','SC7','SC8','SC9','SC10'],year=0,mmdd=0)
print(sc_one)
```

数据样本

```
{'SC10': ['0.00', '181415.13'], 'SC6': ['-30479.00', '0.00'], 'SC7':
['-26449.00', '0.00'], 'SC8': ['31752.86', '84.22'], 'SC9': ['993000.00',
'2900.00']}
```

← 获取市场交易数据get_scjy_value

获取股票的单个数据(非序列)get_gp_one_data →

获取股票的单个财务数据get_gp_one_data

根据证券代码，获取股票的单个数据

```py
get_gp_one_data(stock_list: List[str] = [],
                    field_list: List[str] = []) -> Dict:
```

输入参数

参数	是否必选	参数类型	参数说明
stock_list	Y	List[str]	证券代码列表
field_list	Y	List[str]	字段筛选，不能为空（如 `GO47` 表示是第47号个股数据最新业绩预告 本期扣非净利润预计同比增减幅上限%）这个值，GO为gp one的首字母大写

输出数据

名称	类型	数值	说明
GO1	double		发行价(元)
GO2	double		总发行数量(万股)
GO3	double		一致预期目标价(元)[注：一致预期值均为近半年内各家机构预测数值的平均值]
GO4	double		一致预期T年度
GO5	double		一致预期T年每股收益
GO6	double		一致预期T+1年每股收益
GO7	double		一致预期T+2年每股收益
GO8	double		一致预期T年净利润(万元)
GO9	double		一致预期T+1年净利润(万元)
GO10	double		一致预期T+2年净利润(万元)
GO11	double		一致预期T年营业收入(万元)

名称	类型	数值	说明
GO12	double		一致预期T+1年营业收入(万元)
GO13	double		一致预期T+2年营业收入(万元)
GO14	double		一致预期T年营业利润(万元)
GO15	double		一致预期T+1年营业利润(万元)
GO16	double		一致预期T+2年营业利润(万元)
GO17	double		一致预期T年每股净资产(元)
GO18	double		一致预期T+1年每股净资产(元)
GO19	double		一致预期T+2年每股净资产(元)
GO20	double		一致预期T年净资产收益率(%)
GO21	double		一致预期T+1年净资产收益率(%)
GO22	double		一致预期T+2年净资产收益率(%)
GO23	double		一致预期T年PE
GO24	double		一致预期T+1年PE
GO25	double		一致预期T+2年PE
GO26	double		最新解禁日(YYMMDD格式)
GO27	double		最新解禁数量（万股）
GO28	double		下一报告期的预约披露时间
GO29	double		最新持股机构家数
GO30	double		最新机构持股总量（万股）
GO31	double		最新持股基金家数
GO32	double		最新基金持股量（万股）
GO33	double		最新总股本（万股）
GO34	double		最新实际流通A股（万股）
GO35	double		最新业绩预告 报告期(YYMMDD格式)
GO36	double		最新业绩预告 本期归母净利润下限（万元）
GO37	double		最新业绩预告 本期归母净利润上限（万元）
GO38	double		最新业绩预告 本期归母净利润预计同比增减幅下限%

名称	类型	数值	说明
GO39	double		最新业绩预告 本期归母净利润预计同比增减幅上限%
GO40	double		最新业绩快报 报告期
GO41	double		最新业绩快报 归母净利润（万元）
GO42	double		分红募资 派现总额（万元）
GO43	double		分红募资 募资总额（万元）
GO44	double		最新业绩预告 本期扣非净利润下限(万元)
GO45	double		最新业绩预告 本期扣非净利润上限(万元)
GO46	double		最新业绩预告 本期扣非净利润预计同比增减幅下限%
GO47	double		最新业绩预告 本期扣非净利润预计同比增减幅上限%

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)

go = tq.get_gp_one_data(stock_list=['688318.SH'],field_list=
['GO1','GO2','GO3','GO4','GO5'])
print(go)
```

数据样本

```
{'688318.SH': {'GO1': '107.41', 'GO2': '1667.00', 'GO3': '0.00', 'GO4':
'2025.00', 'GO5': '1.74'}}
```

← 获取指定日期市场交易数据get_scjy_value_by_date　　获取系统分类成份股get_stock_list →

分类/板块成份股

获取系统分类成份股get_stock_list

根据入参返回指定证券代码列表

```py
    def get_stock_list(market = None,
                       list_type: int = 0) -> List:
```

输入参数

参数	是否必选	参数类型	参数说明
market	Y	str	指定代码
list_type	Y	int	返回数据类型

- list_type = 0 只返回代码，list_type = 1 返回代码和名称

```
默认为全部A股
    0:自选股 1:持仓股
    5:所有A股 6:上证指数成份股 7:上证主板 8:深证主板 9:重点指数
    10:所有板块指数 11:缺省行业板块 12:概念板块 13:风格板块 14:地区板块 15:缺省行业分类+概念板块 16:研究行业一级 17:研究行业二级 18:研究行业三级
    21:含H股 22:含可转债 23:沪深300 24:中证500 25:中证1000 26:国证2000 27:中证2000 28:中证A500
    30:REITs 31:ETF基金 32:可转债 33:LOF基金 34:所有可交易基金 35:所有沪深基金 36:T+0基金
    49:金融类企业 50:沪深A股 51:创业板 52:科创板 53:北交所
    101:国内期货 102:港股 103:美股
     91:ETF追踪的指数
     92:国内期货主力合约
```

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
stock_list = tq.get_stock_list('16')
print(stock_list)

stock_list2 = tq.get_stock_list('16',list_type=1)
print(stock_list2)
```

数据样本

```
['881001.SH', '881006.SH', '881015.SH', '881061.SH', '881070.SH', '881090.SH',
'881105.SH', '881129.SH', '881150.SH', '881166.SH', '881183.SH', '881199.SH',
'881211.SH', '881230.SH', '881260.SH', '881286.SH', '881292.SH', '881318.SH',
'881337.SH', '881351.SH', '881368.SH', '881385.SH', '881393.SH', '881405.SH',
'881417.SH', '881426.SH', '881441.SH', '881458.SH', '881469.SH', '881477.SH']

[{'Code': '881001.SH', 'Name': '煤炭'}, {'Code': '881006.SH', 'Name': '石油'},
{'Code': '881015.SH', 'Name': '化工'}, {'Code': '881061.SH', 'Name': '钢铁'},
{'Code': '881070.SH', 'Name': '有色'}, {'Code': '881090.SH', 'Name': '建材'},
{'Code': '881105.SH', 'Name': '农林牧渔'}, {'Code': '881129.SH', 'Name': '食品饮
料'}, {'Code': '881150.SH', 'Name': '纺织服饰'}, {'Code': '881166.SH', 'Name': '轻
工制造'}, {'Code': '881183.SH', 'Name': '家电'}, {'Code': '881199.SH', 'Name': '商
贸'}, {'Code': '881211.SH', 'Name': '汽车'}, {'Code': '881230.SH', 'Name': '医药医
疗'}, {'Code': '881260.SH', 'Name': '电力设备'}, {'Code': '881286.SH', 'Name': '国
防军工'}, {'Code': '881292.SH', 'Name': '机械设备'}, {'Code': '881318.SH', 'Name':
'电子'}, {'Code': '881337.SH', 'Name': '通信'}, {'Code': '881351.SH', 'Name': '计
算机'}, {'Code': '881368.SH', 'Name': '传媒'}, {'Code': '881385.SH', 'Name': '银
行'}, {'Code': '881393.SH', 'Name': '非银金融'}, {'Code': '881405.SH', 'Name': '建
筑'}, {'Code': '881417.SH', 'Name': '房地产'}, {'Code': '881426.SH', 'Name': '社会
服务'}, {'Code': '881441.SH', 'Name': '交通运输'}, {'Code': '881458.SH', 'Name':
'公用事业'}, {'Code': '881469.SH', 'Name': '环保'}, {'Code': '881477.SH', 'Name':
'综合'}]
```

← 获取股票的单个数据(非序列)get_gp_one_data

获取A股板块代码列表get_sector_list →

获取A股板块代码列表get_sector_list

获取A股全部板块代码列表

```py
def get_sector_list(list_type: int = 0) -> List:
```

输入参数

参数	是否必选	参数类型	参数说明
list_type	Y	int	返回数据类型

- list_type = 0 只返回代码，list_type = 1 返回代码和名称

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
block_list = tq.get_sector_list()
print(block_list)
block_list2 = tq.get_sector_list(list_type = 1)
print(block_list2)
```

注：此接口相当于 get_stock_list('10')

数据样本

```
['880081.SH', '880082.SH', '880201.SH', '880202.SH', '880203.SH', '880204.SH',
'880205.SH', '880206.SH', '880207.SH', '880208.SH', ...]

[{'Code': '880081.SH', 'Name': '轮动趋势'}, {'Code': '880082.SH', 'Name': '板块趋
势'}, {'Code': '880201.SH', 'Name': '黑龙江'}, {'Code': '880202.SH', 'Name': '新疆
板块'}, {'Code': '880203.SH', 'Name': '吉林板块'}, {'Code': '880204.SH', 'Name':
'甘肃板块'}, {'Code': '880205.SH', 'Name': '辽宁板块'}, {'Code': '880206.SH',
'Name': '青海板块'}, {'Code': '880207.SH', 'Name': '北京板块'},...]
```

← 获取系统分类成份股get_stock_list　　获取板块成份股get_stock_list_in_sector →

获取板块成份股get_stock_list_in_sector

根据板块代码获取其成份股列表

```py
def get_stock_list_in_sector(block_code: str,
                             block_type: int = 0,
                             list_type: int = 0) -> List:
```

输入参数

参数	是否必选	参数类型	参数说明
block_code	Y	str	板块代码
block_type	N	str	板块类型
list_type	Y	int	返回数据类型

- 获取A股成份股时支持板块名称或板块代码两种方式传入
- block_type=0 表示传入板块指数代码或板块指数名称（默认）
- block_type=1 表示传入自定义板块简称 需要是客户端中预先定义好自定义板块的简称 如果是ZXG表示是自选股；TJG表示是临时条件股
- list_type = 0 只返回代码，list_type = 1 返回代码和名称

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
#通过板块代码获取成份股
block_stocks = tq.get_stock_list_in_sector('880081.SH')
print(block_stocks)
print(len(block_stocks))

#通过板块名获取成份股
block_stocks = tq.get_stock_list_in_sector('钛金属')
print(block_stocks)
print(len(block_stocks))

block_stocks2 = tq.get_stock_list_in_sector('钛金属',list_type=1)
print(block_stocks2)

#获取自定义板块成份股
block_stocks = tq.get_stock_list_in_sector('CSBK', block_type = 1)
print(block_stocks)
```

```
print(len(block_stocks))

```

数据样本

```
['159922.SZ', '510500.SH', '512500.SH']
3
['000545.SZ', '000629.SZ', '000635.SZ', '000688.SZ', '000709.SZ', '000962.SZ',
'002136.SZ', '002140.SZ', '002145.SZ', '002149.SZ', '002167.SZ', '002386.SZ',
'002601.SZ', '002978.SZ', '300402.SZ', '300891.SZ', '600456.SH', '600727.SH',
'603067.SH', '603826.SH', '688122.SH', '688750.SH', '920068.BJ']
23
[{'Code': '000545.SZ', 'Name': '金浦钛业'}, {'Code': '000629.SZ', 'Name': '钒钛股
份'}, {'Code': '000635.SZ', 'Name': '英 力 特'}, {'Code': '000688.SZ', 'Name':
'国城矿业'}, {'Code': '000709.SZ', 'Name': '河钢股份'}, {'Code': '000962.SZ',
'Name': '东方钽业'}, {'Code': '002136.SZ', 'Name': '安 纳 达'}, {'Code':
'002140.SZ', 'Name': '东华科技'}, {'Code': '002145.SZ', 'Name': '钛能化学'},
{'Code': '002149.SZ', 'Name': '西部材料'}, {'Code': '002167.SZ', 'Name': '东方锆
业'}, {'Code': '002386.SZ', 'Name': '天原股份'}, {'Code': '002601.SZ', 'Name': '龙
佰集团'}, {'Code': '002978.SZ', 'Name': '安宁股份'}, {'Code': '300402.SZ', 'Name':
'宝色股份'}, {'Code': '300891.SZ', 'Name': '惠云钛业'}, {'Code': '600456.SH',
'Name': '宝钛股份'}, {'Code': '600727.SH', 'Name': '鲁北化工'}, {'Code':
'603067.SH', 'Name': '振华股份'}, {'Code': '603389.SH', 'Name': '*ST亚振'},
{'Code': '603826.SH', 'Name': '坤彩科技'}, {'Code': '688122.SH', 'Name': '西部超
导'}, {'Code': '688750.SH', 'Name': '金天钛业'}, {'Code': '920068.BJ', 'Name': '天
工股份'}]
['600000.SH', '600004.SH', '600006.SH', '600007.SH', '600008.SH', '600009.SH',
'600010.SH']
7
```

注意

get_stock_list_in_sector 入参的板块只能是自定义板块或者15板块指数
不支持系统 全部A股 沪深A股等板块

← 获取A股板块代码列表get_sector_list

获取自定义板块列表get_user_sector →

自选股/自定义板块

获取自定义板块列表get_user_sector

获取自定义板块代码列表

```py
get_user_sector(cls) -> List:
```

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
user_list = tq.get_user_sector()
print(user_list)
print(len(user_list))
```

数据样本

```
[{'Code': 'CSBK', 'Name': '测试板块'}, {'Code': 'CSBK2', 'Name': '测试板块2'}]
```

← 获取板块成份股get_stock_list_in_sector

添加自定义板块成份股send_user_block →

添加自定义板块成份股

往指定自定义板块中添加成份股

```py
send_user_block(block_code: str = '',
                stock_list: List[str] = [],
                show: bool = False) -> Dict:
```

输入参数

参数	是否必选	参数类型	参数说明
block_code	Y	str	自定义板块简称
stock_list	Y	List[str]	添加的自选股
show	N	bool	客户端是否切换至对应板块界面

- block_code 为客户端已有的自定义板块简称，如果不存在则无效果，空则为添加到临时条件股
- block_code存在，传入空列表则表示清空该板块所有股票，否则为添加新股票
- 自选股的block_code为ZXG

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
zxg_result = tq.send_user_block(block_code='CSBK', stock_list=
["600000.SH","600004.SH","000001.SZ","000002.SZ"])
```

数据样本

```
{'Error': 'Add User Block Completed', 'ErrorId': '0', 'run_id': '1'}
```

← 获取自定义板块列表get_user_sector

清空自定义板块成份股clear_sector →

清空自定义板块成份股

清空指定通达信客户端自定义板块的成份股

```py
clear_sector(block_code:str = ''):
```

输入参数

参数	是否必选	参数类型	参数说明
block_code	Y	str	自定义板块简称

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
clear_ptr = tq.clear_sector(block_code='CSBK')
print(clear_ptr)
```

数据样本

```
{
   "Error" : "清空CSBK板块成功",
   "ErrorId" : "0",
   "run_id" : "1"
}
```

← 添加自定义板块成份股send_user_block

创建自定义板块create_sector →

创建自定义板块

在通达信客户端中创建自定义板块

```py
create_sector(block_code:str = '',
                    block_name:str = ''):
```

输入参数

参数	是否必选	参数类型	参数说明
block_code	Y	str	自定义板块简称
block_name	Y	str	自定义板块名称

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
create_ptr = tq.create_sector(block_code='CSBK2', block_name='测试板块2')
print(create_ptr)
```

数据样本

```
{
   "Error" : "创建CSBK2板块成功",
   "ErrorId" : "0",
   "run_id" : "1"
}
```

← 清空自定义板块成份股clear_sector

删除自定义板块delete_sector →

删除自定义板块

删除通达信客户端中的自定义板块

```py
delete_sector(block_code:str = ''):
```

输入参数

参数	是否必选	参数类型	参数说明
block_code	Y	str	自定义板块简称

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
delete_ptr = tq.delete_sector(block_code='CSBK')
print(delete_ptr)
```

数据样本

```
{
   "Error" : "删除CSBK板块成功",
   "ErrorId" : "0",
   "run_id" : "1"
}
```

← 创建自定义板块create_sector

重命名自定义板块rename_sector →

重命名自定义板块

重命名通达信客户端中的自定义板块

```py
rename_sector(block_code:str = '',
                    block_name:str = ''):
```

输入参数

参数	是否必选	参数类型	参数说明
block_code	Y	str	自定义板块简称
block_name	Y	str	重命名后的自定义板块名称

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
rename_ptr = tq.rename_sector(block_code='CSBK', block_name='测试板块重命名')
print(rename_ptr)
```

数据样本

```
{
   "Error" : "重命名CSBK板块成功",
   "ErrorId" : "0",
   "run_id" : "1"
}
```

← 删除自定义板块delete_sector

跟踪指数的ETF信息get_trackzs_etf_info →

ETF/可转债/期货数据

获取跟踪指数的ETF信息get_trackzs_etf_info

根据指数代码获取跟踪它的ETF的信息

```py
def get_trackzs_etf_info(zs_code: str = ''):
```

输入参数

参数	是否必选	参数类型	参数说明
zs_code	Y	str	指数代码

输出数据

名称	类型	说明
Code	str	证券代码
Name	str	证券名称
NowPrice	str	现价
PreClose	str	昨收
IOPV	str	净值
Zgb	str	净额（万份）
Sz	str	规模（亿元）

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)

trackzs_etf_info = tq.get_trackzs_etf_info(zs_code='950162.CSI')
print(trackzs_etf_info)
```

数据样本

```
[{'Code': '589210.SH', 'Name': '科创芯片设计ETF', 'NowPrice': '1.208', 'PreClose':
'1.192', 'IOPV': '1.2071', 'Zgb': '7646.90', 'Sz': '0.92'},
```

```
{'Code': '589070.SH', 'Name': '科创芯片设计ETF', 'NowPrice': '0.954', 'PreClose':
'0.942', 'IOPV': '0.9547', 'Zgb': '65129.30', 'Sz': '6.21'},
{'Code': '588780.SH', 'Name': ' 科创芯片设计ETF', 'NowPrice': '0.875', 'PreClose':
'0.866', 'IOPV': '0.8756', 'Zgb': '106790.20', 'Sz': '9.34'},
{'Code': '589170.SH', 'Name': '科创芯片设计ETF', 'NowPrice': '0.969', 'PreClose':
'0.956', 'IOPV': '0.9685', 'Zgb': '37890.90', 'Sz': '3.67'},
{'Code': '589250.SH', 'Name': '芯设计PY', 'NowPrice': '0.000', 'PreClose':
'0.000', 'IOPV': '0.0000', 'Zgb': '0.00', 'Sz': '0.00'},
{'Code': '589030.SH', 'Name': '科创芯片设计ETF', 'NowPrice': '1.013', 'PreClose':
'1.000', 'IOPV': '1.0130', 'Zgb': '48407.70', 'Sz': '4.90'}]
```


← 重命名自定义板块rename_sector

可转债信息get_kzz_info →

获取可转债信息get_kzz_info

根据可转债代码获取可转债信息

```py
def get_kzz_info(stock_code:str = '',
                 field_list: List[str] = []):
```

输入参数

参数	是否必选	参数类型	参数说明
stock_code	Y	str	可转债代码
field_list	N	List[str]	字段筛选，传空则返回全部

输出数据

名称	类型	说明
SetCode	str	证券市场
KZZCode	str	可转债代码
HSCode	str	正股代码
ZGPrice	str	转股价格
CurRate	str	当期利率
RestScope	str	剩余规模(万)
PutBack	str	回售触发价
ForceRedeem	str	强赎触发价
ZGDate	str	转股日
EndPrice	str	到期价
EndDate	str	到期日期
ZGRate	str	转股比率%
RealValue	str	纯债价值

名称	类型	说明
ExpireYield	str	到期收益率%
KZZScore	str	可转债评级
HSScore	str	主体评级
RedeemDate	str	赎回登记日期
RedeemPrice	str	赎回价格
PutDate	str	回售申报起始日期
PutPrice	str	回售价格
ZGCode	str	转股代码
AGPrice	str	正股当前价格
KZZPrice	str	可转债当前价格
KZZYj	str	溢价率
ZGValue	str	转股价值

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
kzz_info = tq.get_kzz_info(stock_code = '123039.SZ')
print(kzz_info)
```

数据样本

```
{'CurRate': '2.80',
'EndDate': '20251226',
'EndPrice': '115.00',
'ExpireYield': '0.00',
'ForceRedeem': '37.90',
'HSCode': '300577',
'HSScore': 'A+',
'KZZCode': '123039',
'KZZScore': 'A+',
'PutBack': '20.41',
'PutDate': '0',
'PutPrice': '0.00',
'RealValue': '0.00',
'RedeemDate': '0',
'RedeemPrice': '0.00',
'RestScope': '22044.02',
```

```
    'ZGCode': '123039',
    'ZGDate': '20200702',
    'ZGPrice': '29.15',
    'ZGRate': '1.15',
    'setcode': '0'}
```

← 跟踪指数的ETF信息get_trackzs_etf_info

格式化K线数据formula_format_data →

调用通达信公式

格式化K线数据formula_format_data

格式化get_market_data获取的K线数据

```py
    def formula_format_data(data_dict: Dict = {}):
```

输入参数

参数	是否必选	参数类型	参数说明
data_dict	Y	Dict	get_market_data获取格式的K线Dict

- get_market_data获取的K线数据不能直接用于设置公式参数，须先调用formula_format_data进行格式化
- formula_format_data返回值为List[Dict]，其中Dict的Key须有["Amount", "Volume", "Close", "Open", "High", "Low"]，用户可以直接提供符合条件的List提供给tdx_formula_set_data。

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)

test_md = tq.get_market_data(stock_list=['688318.SH'], count=5, period='1d')
format_md = tq.formula_format_data(test_md)
print(format_md)
```

数据样本

```
{'688318.SH': [
{'Date': '2026-01-20 00:00:00', 'Amount': 33930.29, 'Volume': 2345401.0,
'Close': 144.4, 'Open': 146.5, 'High': 146.98, 'Low': 142.65},
{'Date': '2026-01-21 00:00:00', 'Amount': 35841.09, 'Volume': 2472760.0,
'Close': 144.77, 'Open': 144.49, 'High': 146.5, 'Low': 143.1},
{'Date': '2026-01-22 00:00:00', 'Amount': 41598.79, 'Volume': 2878793.0,
'Close': 143.03, 'Open': 145.0, 'High': 147.0, 'Low': 142.5},
{'Date': '2026-01-23 00:00:00', 'Amount': 47131.04, 'Volume': 3256538.0,
'Close': 144.39, 'Open': 142.58, 'High': 146.88, 'Low': 142.58},
{'Date': '2026-01-26 00:00:00', 'Amount': 54141.73, 'Volume': 3761141.0,
'Close': 141.84, 'Open': 143.7, 'High': 146.77, 'Low': 141.8}]}
```

← 可转债信息get_kzz_info

向通达信公式设置数据formula_set_data →

向通达信公式设置数据formula_set_data

在调用公式前须先设置公式参数，此接口与formula_set_data_info作用一样，会互相覆盖

```py
    def formula_set_data(stock_code: str = '',
                         stock_period: str = '1d',
                         stock_data: List = [],
                         count: int = 1,
                         dividend_type: int = 0):
```

输入参数

参数	是否必选	参数类型	参数说明
stock_code	Y	str	股票代码
stock_period	Y	str	K线周期
stock_data	Y	List	指定格式的K线数据
count	Y	int	选取的K线数量
dividend_type	Y	int	复权类型

- 需要先在下载对应的盘后数据
- dividend_type的取值为：0不复权 1前复权 2后复权
- count为设定stock_data中生效的K线数据，即stock_data中有效数据不能小于count
- count须大于0，且最大不超过24000
- 设置的数据在断开连接前一直生效，后设置的数据会覆盖前面设置的数据

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)

test_md = tq.get_market_data(stock_list=['688318.SH'], count=5, period='1d')
format_md = tq.tdx_formula_format_data(test_md)
formula_set_k = tq.formula_set_data(stock_code='688318.SH', stock_period='1d',
stock_data=format_md['688318.SH'], count=len(format_md['688318.SH']))
print(formula_set_k)
```

数据样本

```
{'ErrorId': '0', 'Msg': '向通达信公式系统设置数据成功! ', 'run_id': '1'}
```

← 格式化K线数据formula_format_data

向通达信公式设置数据信息formula_set_data_info →

向通达信公式设置数据信息formula_set_data_info

在调用公式前须先设置公式参数，此接口与formula_set_data作用一样，会互相覆盖

```py
    def formula_set_data_info(stock_code: str = '',
                              stock_period: str = '1d',
                              start_time: str = '',
                              end_time: str = '',
                              count: int = -1,
                              dividend_type: int = 0):
```

输入参数

参数	是否必选	参数类型	参数说明
stock_code	Y	str	股票代码
stock_period	Y	str	K线周期
start_time	Y	str	起始时间
end_time	Y	str	结束时间
count	Y	int	截取K线数量
dividend_type	Y	int	复权类型

- 需要先在下载对应的盘后数据
- dividend_type的取值为：0不复权 1前复权 2后复权
- count为截取最新交易日开始往前的n条K线，当count参数不为0时，start_time和end_time失效
- count=-1时，获取所有数据，count=-2时，使用无序列数据
- 当count为0时，start_time和end_time生效，指定K线为对应时间段内
- count最大值为24000，count为-1时为获取对应股票全部K线
- 设置的数据在断开连接前一直生效，后设置的数据会覆盖前面设置的数据

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)

formula_set_res =
tq.formula_set_data_info(stock_code='688318.SH',stock_period='1d',
count=100,dividend_type=1)
```

```
print(formula_set_res)
```

数据样本

```
{'ErrorId': '0', 'Msg': '向通达信公式系统设置数据信息成功! ', 'run_id': '1'}
```

← 向通达信公式设置数据formula_set_data

获取公式中的设置数据formula_get_data →

获取公式中的设置数据formula_get_data

获取目前公式设置中的K线数据，使用前须先调用formula_set_data或formula_set_data_info设置公式数据

```py
    def formula_get_data(cls):
```

- 需要先在下载对应的盘后数据

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)

formula_set_res =
tq.formula_set_data_info(stock_code='688318.SH',stock_period='1d',
count=5,dividend_type=1)
formula_kline = tq.formula_get_data()
print(formula_kline)
```

数据样本

```
{'Code': '688318.SH', 'Data': [
{'Amount': 339302880.0, 'Close': 144.4, 'Date': '2026-01-20 00:00:00', 'High':
146.98, 'Low': 142.65, 'Open': 146.5, 'Volume': 2345401.0},
{'Amount': 358410880.0, 'Close': 144.77, 'Date': '2026-01-21 00:00:00', 'High':
146.5, 'Low': 143.1, 'Open': 144.49, 'Volume': 2472760.0},
{'Amount': 415987840.0, 'Close': 143.03, 'Date': '2026-01-22 00:00:00', 'High':
147.0, 'Low': 142.5, 'Open': 145.0, 'Volume': 2878793.0},
{'Amount': 471310432.0, 'Close': 144.39, 'Date': '2026-01-23 00:00:00', 'High':
146.88, 'Low': 142.58, 'Open': 142.58, 'Volume': 3256538.0},
{'Amount': 541417344.0, 'Close': 141.84, 'Date': '2026-01-26 00:00:00', 'High':
146.77, 'Low': 141.8, 'Open': 143.7, 'Volume': 3761141.0}], 'ErrorId': '0'}
```

← 向通达信公式设置数据信息formula_set_data_info　调用通达信公式进行计算formula_zb/xg/exp →

调用通达信公式进行计算formula_zb/xg/exp

调用通达信三种类型的公式

```py
    #调用技术指标公式
def formula_zb(formula_name: str = '',
               formula_arg: str = '',
               xsflag: int = -1):
    #调用条件选股公式
def formula_xg(formula_name: str = '',
               formula_arg: str = ''):
    #调用专家系统公式
def formula_exp(formula_name: str = '',
                formula_arg: str = ''):
```

输入参数

参数	是否必选	参数类型	参数说明
formula_name	Y	str	公式名称
formula_arg	Y	str	公式参数
xsflag	Y	int	数据精度

- 需要先在下载对应的盘后数据
- 目前支持调用技术指标公式、条件选股公式和专家系统公式，调用公式时请注意对应不同的调用接口和公式名
- formula_arg格式为"arg1,arg2,arg3,arg4,arg5"，arg须为纯数字字符串，最多支持16个。
- xsflag小于0时返回默认精度，最大可返回8位小数。
- 请注意一定要完整下载对应的盘后数据（或使用refresh_kline），得到结果与客户端不一致通常是由于设置的K线数量不足导致。

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)

formula_set_res =
tq.formula_set_data_info(stock_code='688318.SH',stock_period='1d',
count=20,dividend_type=1)
#技术指标公式MACD
formula_zb = tq.formula_zb(formula_name='MACD', formula_arg='12,26,9')
```

```
print(formula_zb)
#条件选股公式UPN
formula_xg = tq.formula_xg(formula_name='UPN', formula_arg='3')
print(formula_xg)
#专家系统公式CCI
formula_exp = tq.formula_zb(formula_name='CCI', formula_arg='12')
print(formula_exp)
```

数据样本

```
{'Value': {'DEA': [0.0, 0.01, -0.01, 0.03, 0.29, 0.63, 0.93, 1.25, 1.77, 2.27,
2.72, 3.08, 3.4, 3.57, 3.62, 3.58, 3.46, 3.3, 3.09, 2.83], 'DIF': [0.0, 0.05,
-0.07, 0.19, 1.33, 1.96, 2.16, 2.52, 3.84, 4.25, 4.55, 4.54, 4.64, 4.27, 3.81,
3.44, 2.97, 2.68, 2.21, 1.83], 'MACD': [0.0, 0.07, -0.13, 0.32, 2.07, 2.67,
2.46, 2.54, 4.13, 3.98, 3.65, 2.91, 2.49, 1.39, 0.38, -0.29, -0.98, -1.25,
-1.74, -2.02]}, 'ErrorId': '0'}
{'Value': {'UP3': [None, None, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]}, 'ErrorId': '0'}
{'Value': {'ENTERLONG': [None, None, None, None, None, None, None, None, None,
None, None, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0], 'EXITLONG': [None,
None, None, None, None, None, None, None, None, None, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0]}, 'ErrorId': '0'}
```

← 获取公式中的设置数据formula_get_data　　批量调用通达信公式formula_process_mul_xg/zb →

批量调用通达信公式formula_process_mul_xg/zb/exp

批量调用通达信公式无需使用formula_set_data和formula_set_data_info提前设置，formula_set_data和formula_set_data_info的设置也对批量调用不生效

```py
#批量调用选股公式
def formula_process_mul_xg(formula_name: str = '',
                           formula_arg: str = '',
                           return_count: int = 1,
                                return_date:bool = False,
                                stock_list: List[str] = [],
                                stock_period: str = '1d',
                                start_time: str = '',
                                end_time: str = '',
                                count: int = 0,
                                dividend_type: int = 0):
#批量调用指标公式
def formula_process_mul_zb(formula_name: str = '',
                             formula_arg: str = '',
                             xsflag: int = -1,
                             return_count: int = 1,
                             return_date:bool = False,
                             stock_list: List[str] = [],
                             stock_period: str = '1d',
                             start_time: str = '',
                             end_time: str = '',
                             count: int = 0,
                             dividend_type: int = 0):
```

输入参数

参数	是否必选	参数类型	参数说明
formula_name	Y	str	公式名称
formula_arg	Y	str	公式参数
xsflag	Y	int	数据精度
retrun_count	Y	int	设置每个返回值的返回数
formula_arg	Y	bool	设置是否返回日期
stock_list	Y	List[str]	股票代码列表
stock_period	Y	str	K线周期

参数	是否必选	参数类型	参数说明
start_time	Y	str	起始时间
end_time	Y	str	结束时间
count	Y	int	截取K线数量
dividend_type	Y	int	复权类型

- 需要先在下载对应的盘后数据
- dividend_type的取值为：0不复权 1前复权 2后复权
- count为截取最新交易日开始往前的n条K线，当count参数不为0时，start_time和end_time失效
- count=-1时，获取所有数据，count=-2时，使用无序列数据
- count=-1时如果 return_count=0，返回结果的日期区间受start_time和end_time限制
- 当count为0时，start_time和end_time生效，指定K线为对应时间段内
- count最大值为24000，count为-1时为获取对应股票全部K线
- 正常每个返回值的数据个数应该与count相同，但是return_count可以限制返回个数，去掉用不到的数据，以此提高能够返回的有效数据量；对于选股和多股指标排行场景，一般只需要返回最后一个数据进行判断股票是否选中或显示最后一个指标数据，return_count为1就可以。
- xsflag小于0时返回默认精度，最大可返回8位小数。
- 请注意一定要完整下载对应的盘后数据（或使用refresh_kline），以及retrun_count设置正确，保证结果都能返回。
- 得到结果与客户端不一致通常是由于设置的K线数量不足导致。

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)

#批量调用UPN 选股公式
mul_xg_res = tq.formula_process_mul_xg(
    formula_name='UPN',
    formula_arg='3',
    return_count=3,
    return_date=True,
    stock_list=['688318.SH','600519.SH','000001.SZ'],
    stock_period='1d',
    count=5,
    dividend_type=1)
print(mul_xg_res)

#批量调用CYX 指标公式
mul_zb_res = tq.formula_process_mul_zb(
    formula_name='CYX',
    formula_arg='12',
    return_count=3,
    return_date=True,
    stock_list=['688318.SH','600519.SH','000001.SZ'],
    stock_period='1d',
```

```
    count=5,
    dividend_type=1)
print(mul_zb_res)
```

数据样本

```
{'000001.SZ': {'UP3': [{'Date': '20260203', 'Value': '0'}, {'Date': '20260204',
'Value': '0'}, {'Date': '20260205', 'Value': '0'}]},
'600519.SH': {'UP3': [{'Date': '20260203', 'Value': '0'}, {'Date': '20260204',
'Value': '1'}, {'Date': '20260205', 'Value': '1'}]},
'688318.SH': {'UP3': [{'Date': '20260203', 'Value': '0'}, {'Date': '20260204',
'Value': '0'}, {'Date': '20260205', 'Value': '0'}]}, 'ErrorId': '0'}

{'000001.SZ': {'NOTEXT1': [{'Date': '20260203', 'Value': '11.06'}, {'Date':
'20260204', 'Value': '11.08'}, {'Date': '20260205', 'Value': '11.11'}],
'NOTEXT2': [{'Date': '20260203', 'Value': '10.85'}, {'Date': '20260204',
'Value': '10.91'}, {'Date': '20260205', 'Value': '10.96'}], 'OUTPUT1': ['全国性银
行 深圳板块 跨境支付CIPS ']},
'600519.SH': {'NOTEXT1': [{'Date': '20260203', 'Value': '1494.05'}, {'Date':
'20260204', 'Value': '1529.53'}, {'Date': '20260205', 'Value': '1565.00'}],
'NOTEXT2': [{'Date': '20260203', 'Value': '1446.08'}, {'Date': '20260204',
'Value': '1480.54'}, {'Date': '20260205', 'Value': '1515.00'}], 'OUTPUT1': ['酿酒
贵州板块 通达信88 白酒概念 ']},
'688318.SH': {'NOTEXT1': [{'Date': '20260203', 'Value': '136.60'}, {'Date':
'20260204', 'Value': '135.30'}, {'Date': '20260205', 'Value': '134.00'}],
'NOTEXT2': [{'Date': '20260203', 'Value': '131.74'}, {'Date': '20260204',
'Value': '131.48'}, {'Date': '20260205', 'Value': '131.22'}], 'OUTPUT1': ['软件服
务 深圳板块 腾讯概念 华为鸿蒙 国产软件 互联金融 人工智能 ']}, 'ErrorId': '0'}
```

← 调用通达信公式进行计算formula_zb/xg/exp　　获取指定种类的公式列表formula_get_all →

获取指定种类的公式列表formula_get_all

获取指定种类的公式列表

```py
def formula_get_all(formula_type: int = 0):
```

输入参数

参数	是否必选	参数类型	参数说明
formula_type	Y	int	公式种类标识

- 0 技术指标公式 1 条件选股公式 2 专家系统公式

输出数据

名称	类型	数值	说明
acCode	Y	str	公式代码
acName	Y	str	公式名称
isSys	Y	int	是否为系统公式

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)
formule_all = tq.formula_get_all(formula_type=0)
print(formule_all)
```

数据样本

```
[{'acCode': 'MA', 'acName': '均线', 'isSys': 1},
{'acCode': 'MA2', 'acName': '均线', 'isSys': 1},
{'acCode': 'ABI', 'acName': '绝对广量指标', 'isSys': 1},
{'acCode': 'ADL', 'acName': '腾落指标', 'isSys': 1},
{'acCode': 'ADR', 'acName': '涨跌比率', 'isSys': 1},...]
```

← 批量调用通达信公式formula_process_mul_xg/zb

获取指定公式信息formula_get_info →

获取指定公式信息formula_get_info

获取指定公式信息

```py
def formula_get_info(formula_type: int = 0, formula_code: str = ''):
```

输入参数

参数	是否必选	参数类型	参数说明
formula_type	Y	int	公式种类标识
formula_code	Y	int	公式代码

- 0 技术指标公式 1 条件选股公式 2 专家系统公式

输出数据

名称	类型	数值	说明
acCode	Y	str	公式代码
acName	Y	str	公式名称
isSys	Y	int	是否为系统公式
ParaNum	Y	int	入参数量
Para	Y	Set	公式入参参数
ParaName	Y	Set	公式入参名称
Min	Y	Set	公式入参最小值
Max	Y	Set	公式入参最大值
Default	Y	Set	公式入参默认值
LineNum	Y	int	出参数量
Line	Y	Set	公式出参参数
LineName	Y	Set	公式出参名称

接口使用

```py
from tqcenter import tq

tq.initialize(__file__)
formule_all = tq.formula_get_all(formula_type=0)
print(formule_all)
```

数据样本

```
{'acCode': 'MACD',
'acName': '平滑异同平均线',
'isSys': 1,
'ParaNum': 3,
'Para': [
{'ParaName': 'SHORT','Min': '2.0000','Max': '200.0000','Default': '12.0000'},
{'ParaName': 'LONG','Min': '2.0000', 'Max': '200.0000', 'Default': '26.0000'},
{'ParaName': 'MID', 'Min': '2.0000', 'Max': '200.0000', 'Default': '9.0000'}],
'LineNum': 3,
'Line': [{'LineName': 'DIF'}, {'LineName': 'DEA'}, {'LineName': 'MACD'}]}
```

← 获取指定种类的公式列表formula_get_all

获取资金账户句柄stock_account →

交易函数

获取资金账户句柄

获取指定资金账户的句柄

```py
    def stock_account(account:str = '',
                      account_type: str = 'stock') -> int:
```

输入参数

参数	是否必选	参数类型	参数说明
account	Y	str	资金账号
account_type	Y	str	账号类型

- 获取句柄须先在客户端登陆指定账号
- 返回值大于或等于0时为有效句柄，小于0时为无效句柄
- 涉及到交易函数的调用前，必须要先使用stock_account函数得到account句柄
- account为空时，默认为当前登陆账户
- account_type当前可选：'STOCK' 股票交易, 'CREDIT' 信用交易 'FUTURE' 期货交易 'OPTION' 期权交易。如果不设成参数，就是'STOCK'

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
myAccount = tq.stock_account(account="1190008847", account_type="STOCK")
print(myAccount)
```

数据样本

```
0
```

← 获取指定公式信息formula_get_info

查询账户资产query_stock_asset →

查询账户资产信息

查询指定账户的今日委托信息

```py
def query_stock_asset(account_id:int = -1):
```

输入参数

参数	是否必选	参数类型	参数说明
account_id	Y	int	资金账号句柄

返回数据

数据	默认返回	数据类型	数据说明
Currency	Y	str	币种
Balance	Y	str	余额
Cash	Y	str	可用余额
Asset	Y	str	资产
MarketValue	Y	str	总市值
TotalFreeze	Y	str	期货冻结资金
CloseProfit	Y	str	期货平仓盈亏
CurrentEquity	Y	str	期货动态权益
PreviousEquity	Y	str	期货静态权益
ProfitLoss	Y	str	期货持仓盈亏
TotalMargin	Y	str	期货持仓保证金

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
myAccount = tq.stock_account(account="1190008847", account_type="STOCK")
print(myAccount)
```

```
zc_res = tq.query_stock_asset(account_id=myAccount)
print(zc_res)
```

数据样本

```
{'Currency': '人民币', 'Balance': '30234.070', 'Cash': '30234.070', 'Asset':
'1233041.070', 'MarketValue': '1201690.000', 'ErrorId': '0'}
```

← 获取资金账户句柄stock_account

查询账户委托信息query_stock_orders →

查询账户委托信息

查询指定账户的今日委托信息

```py
def query_stock_orders(account_id:int = -1,
                       stock_code: str = '',
                       cancelable_only: bool = False):
```

输入参数

参数	是否必选	参数类型	参数说明
account_id	Y	int	资金账号句柄
stock_code	Y	str	证券代码
cancelable_only	Y	str	是否仅查询可撤委托（暂未生效）

- Status
  WTSTATUS_NULL 无效单(0)
  WTSTATUS_NOCJ 未成交(1)
  WTSTATUS_PARTCJ 部分成交(2)
  WTSTATUS_ALLCJ 全部成交(3)
  WTSTATUS_BCBC 部分成交部分撤单(4)
  WTSTATUS_ALLCD 全部撤单(5)
- 委托查询只能查询当日委托

返回数据

数据	默认返回	数据类型	数据说明
Wtbh	Y	str	委托编号
Code	Y	str	股票代码
Time	Y	str	时间，HHMMSS
BSFlag	Y	int	买卖标志,0买 1卖 -1撤单
KPFlag	Y	int	开平标志，0开仓1平仓2平今
WTFS	Y	str	市价方式，根据沪深市场不一样
Status	Y	int	委托状态

数据	默认返回	数据类型	数据说明
WtDate	Y	int	撤单标志，为1表示已撤,为2表示是夜盘单
CjPric	Y	str	成交价
CJVol	Y	str	成交数量 如果是撤,则为负值
WtPrice	Y	str	委托价
WtVol	Y	str	委托数量 如果是撤,则为负值

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
myAccount = tq.stock_account(account="1190008847", account_type="STOCK")
print(myAccount)
stock_orders = tq.query_stock_orders(account_id=myAccount, stock_code="")
print(stock_orders)
```

数据样本

```
[{'Wtbh': '48957', 'Code': '688318.SH', 'Time': '93605', 'BSFlag': -1, 'KPFlag':
0, 'WTFS': 0, 'Status': 0, 'WtPrice': '125.000', 'CjPrice': '0.000', 'CjVol':
'0', 'WtVol': '1000'},
{'Wtbh': '58545', 'Code': '688318.SH', 'Time': '93853', 'BSFlag': -1, 'KPFlag':
0, 'WTFS': 0, 'Status': 0, 'WtPrice': '125.000', 'CjPrice': '0.000', 'CjVol':
'0', 'WtVol': '1000'}]
```

← 查询账户资产query_stock_asset

查询账户持仓信息query_stock_positions →

查询账户持仓信息

查询指定账户的持仓信息

```py
def query_stock_positions(account_id:int = -1):
```

输入参数

参数	是否必选	参数类型	参数说明
account_id	Y	int	资金账号句柄

返回数据

数据	默认返回	数据类型	数据说明
Code	Y	str	证券代码
Cbj	Y	str	成本价
TotalVol	Y	str	总持仓
CanUseVol	Y	str	可用持仓
BuyPosition	Y	str	多头持仓（期货或期权）
BuyAvgPrice	Y	str	多头持仓均价（期货或期权）
BuyProfitLoss	Y	str	多头持仓盈亏（期货或期权）
SellPosition	Y	str	空头持仓（期货或期权）
SellAvgPrice	Y	str	空头持仓均价（期货或期权）
SellProfitLoss	Y	str	空头持仓盈亏（期货或期权）
TodayBuyPosition	Y	str	当日买入持仓（期货或期权）
TodaySellPosition	Y	str	当日卖出持仓（期货或期权）

接口使用

```py
from tqcenter import tq
tq.initialize(__file__)
```

```
myAccount = tq.stock_account(account="1190008847", account_type="STOCK")
print(myAccount)
stock_positions = tq.query_stock_positions(account_id=myAccount)
print(stock_positions)
```

数据样本

```
[{'Code': '000001.SZ', 'Cbj': '10.693', 'TotalVol': '100', 'CanUseVol': '100'},
{'Code': '000501.SZ', 'Cbj': '8.663', 'TotalVol': '1000', 'CanUseVol': '1000'},
{'Code': '000716.SZ', 'Cbj': '7.832', 'TotalVol': '100', 'CanUseVol': '100'},
{'Code': '000800.SZ', 'Cbj': '6.642', 'TotalVol': '1000', 'CanUseVol': '1000'},
{'Code': '000858.SZ', 'Cbj': '102.939', 'TotalVol': '500', 'CanUseVol': '500'},
{'Code': '002029.SZ', 'Cbj': '6.106', 'TotalVol': '100', 'CanUseVol': '100'},
{'Code': '002174.SZ', 'Cbj': '7.233', 'TotalVol': '10000', 'CanUseVol':
'10000'},
{'Code': '002251.SZ', 'Cbj': '5.262', 'TotalVol': '100', 'CanUseVol': '100'},
{'Code': '002555.SZ', 'Cbj': '24.648', 'TotalVol': '100', 'CanUseVol': '100'},
{'Code': '002558.SZ', 'Cbj': '13.392', 'TotalVol': '100', 'CanUseVol': '100'},
{'Code': '002624.SZ', 'Cbj': '7.405', 'TotalVol': '10000', 'CanUseVol':
'10000'},
{'Code': '159850.SZ', 'Cbj': '0.609', 'TotalVol': '100000', 'CanUseVol':
'100000'},
{'Code': '160416.SZ', 'Cbj': '1.678', 'TotalVol': '100000', 'CanUseVol':
'100000'},
{'Code': '300459.SZ', 'Cbj': '3.351', 'TotalVol': '10000', 'CanUseVol':
'10000'},
{'Code': '603444.SH', 'Cbj': '163.975', 'TotalVol': '100', 'CanUseVol': '100'},
{'Code': '688318.SH', 'Cbj': '117.425', 'TotalVol': '4000', 'CanUseVol':
'4000'}]
```

← 查询账户委托信息query_stock_orders

交易执行函数order_stock →

交易执行函数

对指定品种执行买卖下单操作

```py
def order_stock(cls,
                account_id:int = -1,
                stock_code:str = '',
                order_type:int = 0,
                order_volume:int = 0,
                price_type:int = 0,
                price:float = 0.0,
                notify:int = 0):
```

输入参数

参数	是否必选	参数类型	参数说明
account_id	Y	int	资金账号句柄
stock_code	Y	str	证券代码
order_type	Y	int	委托类型
order_volume	Y	int	委托数量
price_type	Y	int	报价类型
price	Y	float	委托价格
notify	Y	int	是否手动确认

- order_type可选类型： STOCK_BUY买(0)，STOCK_SELL卖(1)，CREDIT_BUY担保品买入(0)，CREDIT_SELL担保品卖出(1) ，CREDIT_FIN_BUY融资买入(69)，CREDIT_SLO_SELL融券卖出(70)，CREDIT_COV_BUY(71)，CREDIT_STK_REPAY(76)，ETF相关，FUNTURE相关，OPTION相关
- price_type可选类型：PRICE_MY自填价格(0)，PRICE_SJ市价(1)，PRICE_ZTJ涨停价(2)，PRICE_DTJ跌停价(3)
- 如果price_type为市价类型，具体是哪种市价，请在客户端->系统设置->参数中 进行设置
-
- 注：对于实盘交易账户是提示下单让用户确认。
- 注：交易需满足交易所规则。对于沪深交易所可转债品种，不支持市价交易方式。
-
- 实盘交易账户的自动下单请联系你的开户券商，开通并使用对应的支持TQ的版本。

返回数据

数据	默认返回	数据类型	数据说明
Value	Y	int	成功标志，0失败，1待用户确认，2成功
Wtbh	Y	str	委托编号，只有成功时才会返回
Msg	Y	str	返回提示信息

接口使用

```py
from tqcenter import tq
from tqcenter import tqconst
tq.initialize(__file__)

myAccount = tq.stock_account(account="1190008847", account_type="STOCK")
print(myAccount)
order_res = tq.order_stock(account_id=myAccount,
                           stock_code="688318.SH",
                           order_type=tqconst.STOCK_BUY,
                           order_volume=200,
                           price_type=tqconst.PRICE_MY,
                           price=160.0)
print(order_res)
```

数据样本

```
{'ErrorId': '0', 'Msg': '已发送信号至客户端，待用户确认！', 'Value': 1}
```

← 查询账户持仓信息query_stock_positions

撤单cancel_order_stock →

撤单

根据委托编号撤单

```py
    def cancel_order_stock(account_id:int = -1,
                           stock_code:str = '',
                           order_id:str = ''):
```

输入参数

参数	是否必选	参数类型	参数说明
account_id	Y	int	资金账号句柄
stock_code	Y	str	证券代码
order_id	Y	int	委托编号

返回数据

数据	默认返回	数据类型	数据说明
Value	Y	int	成功标志，0失败，1成功
Msg	Y	str	返回提示信息

- 撤单成功后Status状态会变成WTSTATUS_NULL无效单(0)、WTSTATUS_BCBC部分成交部分撤单(4)或WTSTATUS_ALLCD全部撤单(5)

接口使用

```py
from tqcenter import tq
from tqcenter import tqconst
tq.initialize(__file__)

myAccount = tq.stock_account(account="1190008847", account_type="STOCK")
print(myAccount)

stock_orders = tq.query_stock_orders(account_id=myAccount, stock_code="")
print(stock_orders)

cancel_res = tq.cancel_order_stock(account_id=myAccount,
                           stock_code=stock_orders[0]['Code'],
                           order_id=stock_orders[0]['Wtbh'])
```

```
print(cancel_res)

```

数据样本

```
{'Value': 1, 'ErrorId': '0', 'Msg': '提交撤单成功! '}
```

← 交易执行函数order_stock

HTTP方式调用 →

HTTP方式调用

除了tqcenter的调用方式，也可以直接通过以下方式调用。
POST http://127.0.0.1:17709/
使用时需要先开启支持TQ的通达信客户端。

method为TdxQuant中的函数方法，params为函数参数。

例如：

```
{
    "id":1,
    "method": "get_market_data",
    "params": {
        "stock_list":["688318.SH"],
        "count":5,
        "dividend_type":"none",
        "period":"1d"
    }
}
```

返回：

```
{
    "id": 1,
    "result": {
        "ErrorId": "0",
        "KlinePaged": true,
        "KlineTotal": {
            "688318.SH": 5
        },
        "Value": {
            "688318.SH": {
                "Amount": [
                    "61995.09",
                    "62455.38",
                    "59548.38",
                    "58073.54",
                    "38680.98"
                ],
                "Close": [
                    "122.45",
                    "120.32",
                    "117.84",
                    "112.00",
```

```
                    "82.70"
                ],
                "Date": [
                    "20260526",
                    "20260527",
                    "20260528",
                    "20260529",
                    "20260605"
                ],
                "ErrorId": "0",
                "ForwardFactor": [
                    "0.711862",
                    "0.711862",
                    "0.711862",
                    "0.711862",
                    "1.000000"
                ],
                "High": [
                    "124.58",
                    "125.61",
                    "120.32",
                    "120.20",
                    "85.24"
                ],
                "Low": [
                    "117.70",
                    "120.02",
                    "113.88",
                    "111.25",
                    "81.22"
                ],
                "Open": [
                    "119.71",
                    "121.28",
                    "120.30",
                    "118.19",
                    "84.00"
                ],
                "Time": [
                    "0",
                    "0",
                    "0",
                    "0",
                    "0"
                ],
                "VolInStock": [
                    "0",
                    "0",
                    "0",
                    "0",
                    "0"
                ],
                "Volume": [
                    "5086297.00",
                    "5081741.00",
                    "5104389.00",
```

```
                    "5017581.00",
                    "4627644.00"
                ]
            }
        },
        "has_more": false,
        "next_stock_page_index": 1,
        "stock_page_count": 1,
        "stock_page_index": 0,
        "stock_page_size": 100,
        "stock_total": 1
    }
}
```


← 撤单cancel_order_stock

AI平台调用 →

提供2个Skills包供各智能体平台安装

通达信TQ-Python
https://www.tdx.com.cn/products/autoup/skills/tdx-quant.zip

通达信TQ-Local
https://www.tdx.com.cn/products/autoup/skills/tdx-tq-local.zip

将以上zip解压后，在WorkBuddy,CodeX或TdxClaw等AI平台中安装。
使用时需要先开启支持TQ的通达信客户端。

通达信TQ-Python介绍

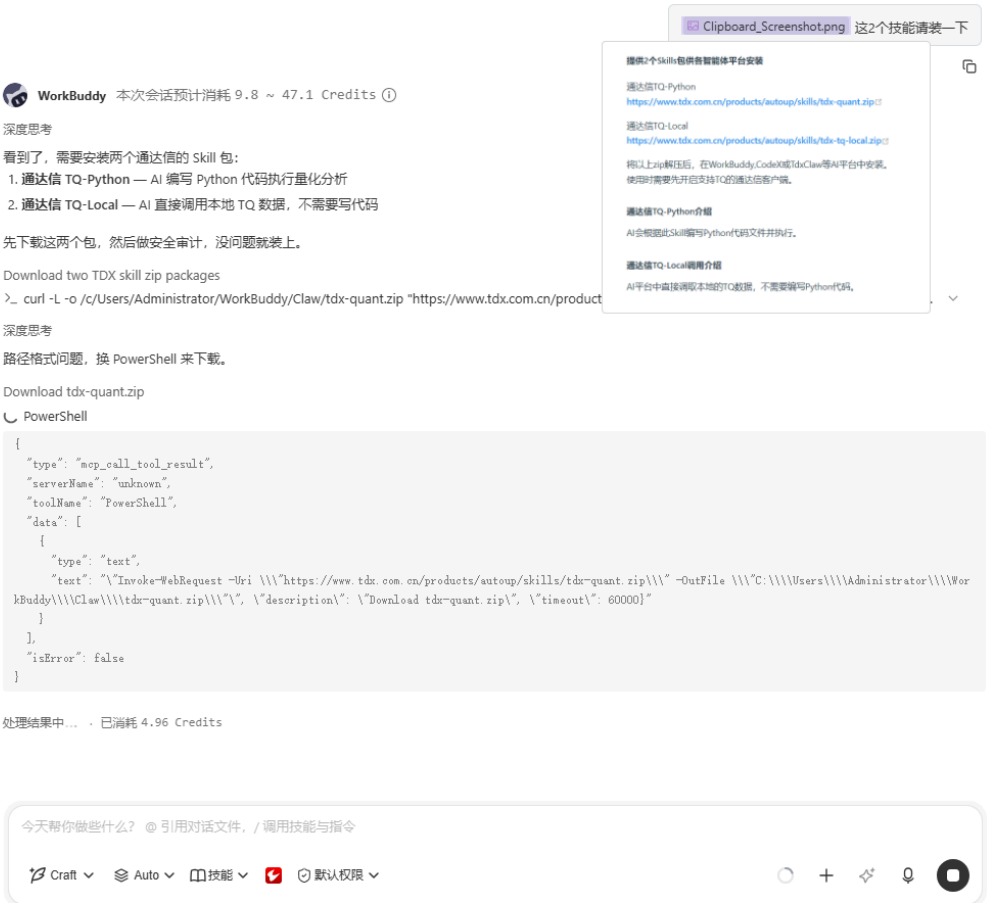
AI会根据此Skill编写Python代码文件并执行。

通达信TQ-Local调用介绍

AI平台中直接调取本地的TQ数据，不需要编写Python代码。

直接告诉AI安装这2个技能

例如：



使用通达信TQ-Local技能

例如：

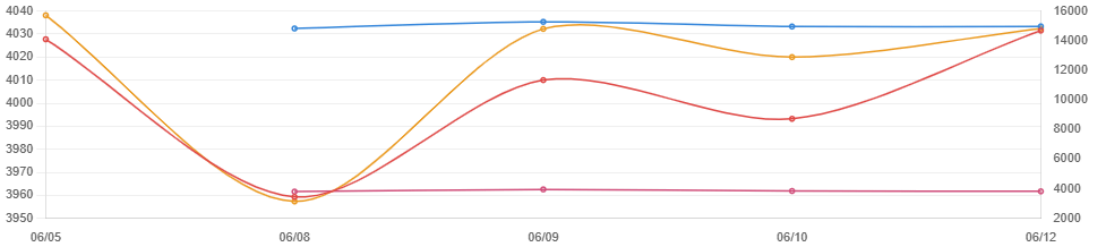
通达信TQ-Local 查查最近大盘的情况

WorkBuddy

工具调用 12 · 过程消息 6

A股大盘行情数据，含四大指数最新行情与近5日走势

上证指数	深证成指	创业板指	科创50
4,031.51	14,963.41	3,830.35	1,663.22
+44.50 (+1.12%)	+111.43 (+0.75%)	+19.10 (+0.50%)	+0.78 (+0.05%)
涨 1,700 / 跌 622	涨 2,067 / 跌 801	涨 930 / 跌 447	涨 367 / 跌 236
成交 1,537亿	成交 1,678亿	成交 811亿	成交 176亿
年初至今 +1.58%	年初至今 +10.64%	年初至今 +19.58%	年初至今 +23.73%



两市总成交	上涨家数	下跌家数	A股总市值
3.22万亿	3,767	1,423	~114万亿

深度思考

以上是最近一个交易日（6月12日周五）的大盘全貌。几个要点：

- 四大指数全线收红，**上证**以 +1.12% 领涨，成交明显放量
- 上涨家数（3767）远超下跌（1423），市场整体偏暖
- 年初至今分化明显：**科创50** +23.73% 和 **创业板** +19.58% 遥遥领先，上证仅 +1.58%
- 两市合计成交约 3.22万亿

通达信TQ-Local 查查我的帐户的持仓和资金情况

WorkBuddy

工具调用 4 · 过程消息 4

查询完成，以下是您的账户情况：

资金概况

项目	金额（人民币）
总资产	**¥1,091,314.05**
总市值	¥748,730.20
余额	¥342,584.05
可用余额	¥342,584.05

持仓明细

证券代码	持仓数量	可用数量	成本价
512170.SH	100	100	¥0.322
513120.SH	1,400	1,400	¥1.365
123247.SZ	1,600	1,600	¥147.486
159105.SZ	1,400	1,400	¥0.862
300199.SZ	3,600	3,600	¥25.108
300476.SZ	600	600	¥188.764
300779.SZ	1,000	1,000	¥214.644

目前持仓共 **7 只**证券，总市值约 **¥74.87 万**，现金余额约 **¥34.26 万**。如需查询具体股票的实时盈亏或名称，告诉我即可。

顺便将这几只ETF都加仓一倍

WorkBuddy

› 工具调用 5 · 过程消息 6

已发送三只ETF的加仓委托，详情如下：

证券代码	买入数量	委托价格	状态
512170.SH	100 股	¥0.33	已发送，待确认
513120.SH	1,400 股	¥1.12	已发送，待确认
159105.SZ	1,400 股	¥0.83	已发送，待确认

⚠ **请注意：**

1. **请在通达信客户端确认委托** — 订单已发送至客户端，需要您在弹出的确认窗口中点击"确认"才能正式提交。
2. **当前为非交易时间**（周六晚间），即使确认后订单也不会立即成交，通常会作为**预埋单**或留待**下周一开盘**时进入交易系统。
3. 委托价格基于周五收盘价并留有小幅上浮，实际成交价格以市场撮合为准。

如需查询委托状态或撤单，随时告诉我。

← HTTP方式调用

常量枚举 →

市场类型

名称	类型	数值	说明
.SZ	int	0	深圳交易所
.SH	int	1	上海交易所
.BJ	int	2	北京交易所
.NQ	int	44	新三板
.SHO	int	8	上海个股期权
.SZO	int	9	深圳个股期权
.HK	int	31	香港交易所
.US	int	74	美国股票
.CSI	int	62	中证指数
.CNI	int	102	国证指数
.HG	int	38	国内宏观指标
.CFF	int	47	中金期货
.CZC	int	28	郑州期货
.DCE	int	29	大连期货
.SHF	int	30	上海期货
.GFE	int	66	广州期货
.INE	int	30	上海能源
.HI	int	27	港股指数
.OF	int	33	开放式基金净值
.CFFO	int	7	中金所期权
.CZCO	int	4	郑州期货期权
.DCEO	int	5	大连期货期权
.SHFO	int	6	上海期货期权

名称	类型	数值	说明
.GFEO	int	67	广州期货期权
.QHZ	int	42	期货类指数

dividend_type复权类型

名称	类型	数值	说明
type	str	none	不复权
type	str	front	前复权
type	str	back	后复权

period周期入参类型

名称	类型	数值	说明
period	str	1m	1分钟
period	str	5m	5分钟
period	str	15m	15分钟
period	str	30m	30分钟
period	str	1h	60分钟（1小时）
period	str	1d	1天
period	str	1w	1周
period	str	1mon	1月
period	str	1q	1季
period	str	1y	1年
period	str	tick	分笔

order_type类型

名称	类型	数值	说明
STOCK_BUY	int	0	买
STOCK_SELL	int	1	卖
CREDIT_BUY	int	0	担保品买入

名称	类型	数值	说明
CREDIT_SELL	int	1	担保品卖出
CREDIT_FIN_BUY	int	69	融资买入
CREDIT_SLO_SELL	int	70	融券卖出
CREDIT_COV_BUY	int	71	买券还券
CREDIT_STK_REPAY	int	76	卖券还款
ETF_PURCHASE	int	45	基金申购
ETF_REDEMPTION	int	46	基金赎回
FUTURE_OPEN_LONG	int	101	期货开多
FUTURE_OPEN_SHORT	int	102	期货开空
FUTURE_CLOSE_LONG	int	103	期货平多
FUTURE_CLOSE_SHORT	int	104	期货平空
OPTION_OPEN_LONG	int	201	期权开多
OPTION_OPEN_SHORT	int	202	期权开空
OPTION_CLOSE_LONG	int	203	期权平多
OPTION_CLOSE_SHORT	int	204	期权平空

price_type类型

名称	类型	数值	说明
PRICE_MY	int	0	自填价
PRICE_SJ	int	1	市价
PRICE_ZTJ	int	2	涨停价/笼子上限
PRICE_DTJ	int	3	跌停价/笼子下限

Status类型

名称	类型	数值	说明
WTSTATUS_NULL	int	0	无效单
WTSTATUS_NOCJ	int	1	未成交
WTSTATUS_PARTCJ	int	2	部分成交

名称	类型	数值	说明
WTSTATUS_ALLCJ	int	3	全部成交
WTSTATUS_BCBC	int	4	部分成交部分撤单
WTSTATUS_ALLCD	int	5	全部撤单

← AI平台调用

回测及模拟交易 →

什么是量化交易

量化交易是指利用计算机科技并采用一定的数学模型去实现投资理念、实现投资策略的过程。简单的说，量化交易主要是做这样的事：

一个简单的投资想法 => 可执行的交易策略 => 可执行的代码程序 => 检验交易策略效果 => 实盘交易验证改进

Step 1：从一个简单的投资想法开始

投资想法即我们认为可能会盈利的投资方法、理念，比如熊市时期银行股是潜力股、复制基金经理的增强指数、金叉买入死叉卖出等等。这些想法通常以网络、书本和讲座等为载体，来源于投顾、同行以及自己的经验总结等等。

以一个简单的投资想法为例：

```py
如果遇到股价金叉，则买入
如果遇到股价死叉，则卖出
```

Step 2：完善这个想法，形成明确的可执行的交易策略

简单的投资想法通常比较模糊，我们需要将其细化成明确的可执行的策略，目的是为了能得到确定的信号进行交易操作。

一个可执行的交易策略至少需要明确以下几点:

1. Security：确定投资品种或范围
2. Condition：确定触发买/卖的具体条件
3. Quantity：确定买卖的数量/金额等

明确的可执行的交易策略的判断基准：根据交易策略的描述，不同的人在相同情形下，都能做出相同的交易操作。

上述关于金叉死叉的投资想法，显然它是不够明确的（可度量/可计算）。所以我们进一步细化：

```py
监测沪深300指数的所有成分股的收盘价
如果收盘价上穿收盘价的5日简单移动平均，则用全部可用资金买入该股票
如果收盘价的5日简单移动平均上穿收盘价，则卖出该股票所有持仓
```

现在，我们基本已经把之前的想法细化成了明确的可执行的交易策略。当然，可能还有些地方不够明确或者参数需要改动，这些可以随时想到随时修正，不必一次做到完美。

现在，我们想知道这样操作究竟会不会赚钱?

Step 3：编写一段代码，把交易策略转成可执行的代码程序

为了验证这个策略是否赚钱，我们需要把明确后的交易策略通过编程转成程序，计算机能根据历史数据/实时数据执行该策略产生模拟交易，或者根据实时数据执行该策略产生实盘交易。

把上述策略翻译成计算机可以识别的代码语言，即类似这样的代码：

```py

import pandas as pd
import vectorbt as vbt
from tqcenter import tq

tq.initialize(__file__)

# 解决 pandas future warning
pd.set_option('future.no_silent_downcasting', True)

# ======================== 核心配置（用户可直接修改这里）========================
target_start = '20240930'  # 【目标回测开始时间】(真正想回测的起始日)
target_end = '20250930'    # 【目标回测结束时间】
stock_code_list = ['688318.SH']     # 股票代码
window = 5          # MA指标周期（如MA5、MA10、MA20，改这里自动适配历史数据）
#
================================================================================

start_time = (pd.to_datetime(target_start) - pd.Timedelta(days=window +
10)).strftime('%Y%m%d')

# 1.获取价格数据
df_real = tq.get_market_data(
    field_list=['Close', 'Open'],
    stock_list=stock_code_list,
    start_time=start_time,
    end_time=target_end,
    dividend_type='front',
    period='1d',
    fill_data=True
)
close_df = tq.price_df(df_real, 'Close', column_names=stock_code_list)
open_df = tq.price_df(df_real, 'Open', column_names=stock_code_list)


# 2.买卖信号计算与生成
ma5_dynamic = vbt.MA.run(close_df, window=window).ma
ma5_dynamic.columns = close_df.columns

entries_raw = close_df.vbt.crossed_above(ma5_dynamic)
exits_raw = close_df.vbt.crossed_below(ma5_dynamic)

# 信号移位+1
entries_df = entries_raw.shift(1).fillna(False).astype(bool)
exits_df = exits_raw.shift(1).fillna(False).astype(bool)
```

```
46
47
48  # 3. 执行回测
49  portfolio = vbt.Portfolio.from_signals(
50      close=close_df,              # 净值计算用未复权收盘价
51      entries=entries_df,              # 延迟后的买入信号
52      exits=exits_df,                  # 延迟后的卖出信号
53      price=open_df,                 # 含滑点的成交价格
54      init_cash=100000,             #  初始资金10万元
55      fees=0.0003,                   # 手续费0.03%（双边）
56      freq='D',                      # 日线频率
57      size_granularity=100           # A股最小交易单位100股
58  )
59
60
61  # 4. 输出回测结果
62  print(f"\n======投资组合回测表现=====")
63  print(portfolio.stats())
64  print(f"\n======投资组合回测记录======")
    print(portfolio.trades.records_readable)
```

这样一来，刚才细化好的策略转成了代码，计算机就能理解并执行了。

Step 4：回测或者模拟交易，检验策略效果

基本的检验策略方法有回测和模拟交易两种方法。**核心区别是：回测是用历史数据模拟执行策略，模拟交易是用未来的实际数据模拟执行策略。。**

回测是让计算机能根据一段时间区间内的历史的数据来模拟执行该策略，根据结果评价并改进策略。如果结果不好，则需要分析原因并改进。如果结果不错，则可以考虑用模拟交易进一步验证。

模拟交易是让计算机能根据未来的实际数据模拟执行该策略一段时间区间，根据结果评价并改进策略。如果策略在回测与模拟交易的表现都非常好，我们可以考虑进行完全真金白银的实盘交易。

回测举例说明：

1. 策略环境：设定初始虚拟资产100万元；选择一段历史时间区间：20100101到20200101；把该时间区间的各种数据如收盘股价行情等发给计算机。
2. 策略执行：计算机利用这些数据模仿历史真实的市场，执行我们编写的策略程序。
3. 策略评估：计算机会出具一份报告，根据这个报告我们知道，在20100101期初的100万元，按照我们的策略交易到期末20200101，会怎样？一般包括盈亏情况，下单情况，持仓变化，以及一些统计指标等，根据此评估交易策略的好坏。

模拟交易举例说明：

1. 策略环境：设定初始的虚拟资产比如100万元，选择开始执行模拟交易的时间点，比如下周一。那么从下周一开始，股市开始交易，真实的行情数据就会实时地发送到计算机。
2. 策略执行：计算机利用真实的数据模仿真实的市场，执行你的策略代码输出买卖队列，模拟系统会记录每一笔买卖记录。

3. 策略评估：我们可以得到一份实时更新的策略评估报告，这报告类似于回测得到的报告，不同的是会根据实际行情变化更新；同样我们能据此评估交易策略的好坏。

Step 5：实盘执行交易策略，并持续优化改进策略

实盘交易就是让计算机能根据实际行情，用真实资金账号来自动下单交易。**注意，这时不再是用虚拟资产进行模拟交易，实盘交易账户上的盈亏都是真金白银。**
实盘交易一般也会给出一份类似模拟交易的投资分析报告，通过实时观察策略的实盘表现、根据投资理念的变化、市场状况的变化及时修正、改善和优化策略，使之保持持续盈利能力。

← 常量枚举

执行选股入板块 →

场景化示例

执行选股策略并加入客户端自定义板块

第一步：执行选股策略

```py
import pandas as pd
import numpy as np
from datetime import datetime
from tqcenter import tq

# 初始化tq
tq.initialize(__file__)

# 1. 基础配置（可修改项）
batch_codes = tq.get_stock_list_in_sector('通达信88')    # 目标板块
start_time = "20251025"                                  # 数据起始日期
target_end = datetime.now().strftime("%Y%m%d")           # 数据结束日期（当前日期）
N = 3                                                    # 目标连续上涨天数
block_code = 'LZXG'                                      # 自定义板块简称（必选）
block_name = '连涨选股'                                   # 自定义板块名称（必选）

# 2. 获取并整理收盘价数据
df_real = tq.get_market_data(
    field_list=['Close'],
    stock_list=batch_codes,
    start_time=start_time,
    end_time=target_end,
    dividend_type='front',  # 前复权
    period='1d',            # 日线
    fill_data=True          # 填充缺失数据
)
# 转换为「日期×股票代码」的收盘价宽表
close_df = tq.price_df(df_real, 'Close', column_names=batch_codes)

# 3. 标记每日是否上涨（核心判断逻辑）
is_up = close_df > close_df.shift(1)  # True=当日上涨，False=当日非上涨

# 4. 核心：计算连续上涨天数
# 步骤1：上涨日标记为1，非上涨日标记为NaN
up_mask = np.where(is_up, 1, np.nan)
up_mask_df = pd.DataFrame(up_mask, index=close_df.index,
columns=close_df.columns)

# 步骤2：前向填充 → 连续上涨阶段的非上涨日（NaN）会被1填充
filled_df = up_mask_df.ffill()

# 步骤3：累计非NaN值的数量（初步计数）
consec_up_days = filled_df.notna().cumsum()

```

```python
# 步骤4：非上涨日重置计数（关键步骤，实现“连续”效果）
reset_counts = consec_up_days.where(~is_up).ffill().fillna(0)
consec_up_days = (consec_up_days - reset_counts).astype(int)

# 5. 筛选符合条件的股票（连续上涨≥N天）
latest_date = consec_up_days.index[-1]  # 最新交易日
latest_consec_up = consec_up_days.loc[latest_date]  # 每只股票最新的连续上涨天数
target_stocks = latest_consec_up[latest_consec_up >=
N].sort_values(ascending=False)
target_stocks_list = target_stocks.index.tolist()  # 提取符合条件的股票代码列表

# 6. 先创建自定义板块，再执行添加/清空操作
print(f"\n=== 筛选结果（连续上涨≥{N}天）===")
# 第一步：创建自定义板块
try:
    tq.create_sector(block_code=block_code, block_name=block_name)
    print(f"✅ 已成功创建自定义板块「{block_name}（{block_code}）」")
except Exception as e:
    # 板块已存在时可能报错，此处捕获异常不中断流程
    print(f"ℹ️ 自定义板块创建提示：{e}（若提示已存在，可忽略此信息）")

# 第二步：处理板块成份股（添加/清空）
if not target_stocks.empty:
    # 打印筛选结果
    print(f"符合条件的股票共 {len(target_stocks)} 只：")
    for stock_code, days in target_stocks.items():
        print(f"{stock_code}：连续上涨 {days} 天")

    # 发送至自定义板块
    try:
        tq.send_user_block(block_code=block_code, stocks=target_stocks_list)
        print(f"\n✅ 已成功将股票添加至自定义板块「{block_name}（{block_code}）」")
    except Exception as e:
        print(f"\n❌ 添加自定义板块失败：{e}")

    # 发送提示消息至TQ策略管理器
    msg = f"MSG,筛选结果：{start_time}至{target_end}，连续上涨≥{N}天的股票共
{len(target_stocks)}只，已添加至「{block_name}（{block_code}）」"
    try:
        tq.send_message(msg)
        print("✅ 提示消息发送成功")
    except Exception as e:
        print(f"❌ 消息发送失败：{e}")
else:
    # 无符合条件股票时清空板块
    print("暂无符合条件的股票")
    try:
        tq.send_user_block(block_code=block_code, stocks=[])
        print(f"✅ 已清空自定义板块「{block_name}（{block_code}）」")
    except Exception as e:
        print(f"❌ 清空自定义板块失败：{e}")

    # 发送空结果提示
    msg = f"MSG,筛选结果：{start_time}至{target_end}，连续上涨≥{N}天的股票共0只，已
清空「{block_name}（{block_code}）」"
    try:
```

```
            tq.send_message(msg)
        except Exception as e:
            print(f"❌ 消息发送失败：{e}")
```

第二步：客户端查看执行效果

← 回测及模拟交易　　订阅行情涨幅突破实时预警 →

订阅行情涨幅突破实时预计

第一步：设置预警条件，并发送预警结果到客户端

```py
#订阅板块成分股行情，涨幅突破实时预警，首次突破后取消该证券行情订阅监控
import json
import time
import signal
import sys
from datetime import datetime, timedelta
from collections import defaultdict
from tqcenter import tq

# ==================== 全局配置 ====================
# 板块配置：支持多个板块/自定义板块
SECTOR_NAMES = ['通达信88']  # 可扩展为其他板块名称或代码
PRICE_RISE_THRESHOLD = 5.0  # 涨幅阈值>5%
ANTI_SHAKE_SECONDS = 10      # 防抖间隔
BATCH_SUBSCRIBE_SIZE = 50    # 分批订阅大小（避免单次订阅过多）

# 全局变量
SUBSCRIBE_CODES = []         # 动态获取的监控股票列表
last_warn_time = defaultdict(int)
EXIT_FLAG = False
TRIGGERED_STOCKS = set()     # 记录已首次触发预警的股票（避免重复监控/推送）

# ==================== 信号处理函数 ====================
def signal_handler(signum, frame):
    """处理Ctrl+C（SIGINT）信号"""
    global EXIT_FLAG
    print(f"\n\n[{datetime.now().strftime('%H:%M:%S')}] 接收到退出信号（Ctrl+C），
开始清理资源...")
    EXIT_FLAG = True
    # 强制取消订阅+关闭TDX
    try:
        unsubscribe_stocks()
    except Exception as e:
        print(f"取消订阅失败：{e}")

    print("资源清理完成，程序退出！")
    sys.exit(0)

# ==================== 工具函数（新增） ====================
def get_valid_stock_codes(sector_names):
    """
    从指定板块获取有效股票代码列表
    :param sector_names: 板块名称列表
    :return: 去重后的有效股票代码列表
```

```
    """
    valid_codes = set()  # 用集合去重
    for sector in sector_names:
        try:
            # 获取板块股票列表（TDX初始化后调用）
            sector_codes = tq.get_stock_list_in_sector(sector)
            if not sector_codes:
                print(f"[{datetime.now().strftime('%H:%M:%S')}] 警告：板块
{sector}未获取到股票列表")
                continue

            # 过滤无效代码（空值、格式错误）
            for code in sector_codes:
                if code and isinstance(code, str) and (code.endswith('.SH') or
code.endswith('.SZ')):
                    valid_codes.add(code)
                else:
                    print(f"[{datetime.now().strftime('%H:%M:%S')}] 过滤无效代码：
{code}")

        except Exception as e:
            print(f"[{datetime.now().strftime('%H:%M:%S')}] 获取板块{sector}股票列
表失败：{e}")
            import traceback
            traceback.print_exc()

    # 转为列表并排序
    valid_codes_list = sorted(list(valid_codes))
    print(f"[{datetime.now().strftime('%H:%M:%S')}] 从板块{sector_names}获取到有效
股票{len(valid_codes_list)}只：{valid_codes_list[:10]}...")  # 只打印前10个
    return valid_codes_list

def batch_subscribe(stocks, batch_size):
    """
    分批订阅股票（避免单次订阅过多）
    :param stocks: 股票列表
    :param batch_size: 每批订阅数量
    :return: 整体订阅结果（True/False）
    """
    total_success = True
    for i in range(0, len(stocks), batch_size):
        batch = stocks[i:i+batch_size]
        try:
            print(f"\n[{datetime.now().strftime('%H:%M:%S')}] 订阅第
{i//batch_size + 1}批股票（{len(batch)}只）：{batch[:5]}...")
            sub_res = tq.subscribe_hq(stock_list=batch,
callback=price_rise_callback)
            if not sub_res:
                print(f"[{datetime.now().strftime('%H:%M:%S')}] 第{i//batch_size
+ 1}批订阅失败：{sub_res}")
                total_success = False
            else:
                print(f"[{datetime.now().strftime('%H:%M:%S')}] 第{i//batch_size
+ 1}批订阅成功：{sub_res}")
        except Exception as e:
            print(f"[{datetime.now().strftime('%H:%M:%S')}] 第{i//batch_size + 1}
```

```
批订阅异常: {e}")
            total_success = False
    return total_success

def unsubscribe_single_stock(stock_code):
    """
    取消单只股票的订阅（首次触发后不再监控）
    :param stock_code: 股票代码
    :return: 取消结果（True/False）
    """
    try:
        un_sub_res = tq.unsubscribe_hq(stock_list=[stock_code])
        if un_sub_res:
            # 从全局订阅列表中移除
            if stock_code in SUBSCRIBE_CODES:
                SUBSCRIBE_CODES.remove(stock_code)
            return True
        return False
    except Exception as e:
        print(f"[{datetime.now().strftime('%H:%M:%S')}] 取消{stock_code}订阅失败:
{e}")
        return False

# ==================== 核心回调函数 ====================
def price_rise_callback(data_str):
    try:
        code_json = json.loads(data_str)
        code = code_json.get('Code')

        # 前置过滤：无效数据/非监控股票/已触发过的股票（直接返回，不输出日志）
        if (code_json.get('ErrorId') != "0" or not code) or \
           (code not in SUBSCRIBE_CODES) or \
           (code in TRIGGERED_STOCKS):
            return

        # 获取最新行情数据
        report_ptr = tq.get_full_tick(code)

        latest_price = 0.0
        pre_close = 0.0

        if report_ptr:
            latest_price = round(float(report_ptr['Now']), 2)
            pre_close = round(float(report_ptr['LastClose']), 2)

            if pre_close <= 0 and latest_price > 0:
                pre_close = latest_price - 0.01

        # 过滤最新价/昨收价无效的情况
        if latest_price <= 0 or pre_close <= 0:
            return

        # 计算涨幅
        rise_rate = round(((latest_price - pre_close) / pre_close) * 100, 2) if
pre_close > 0 else 0.0

```

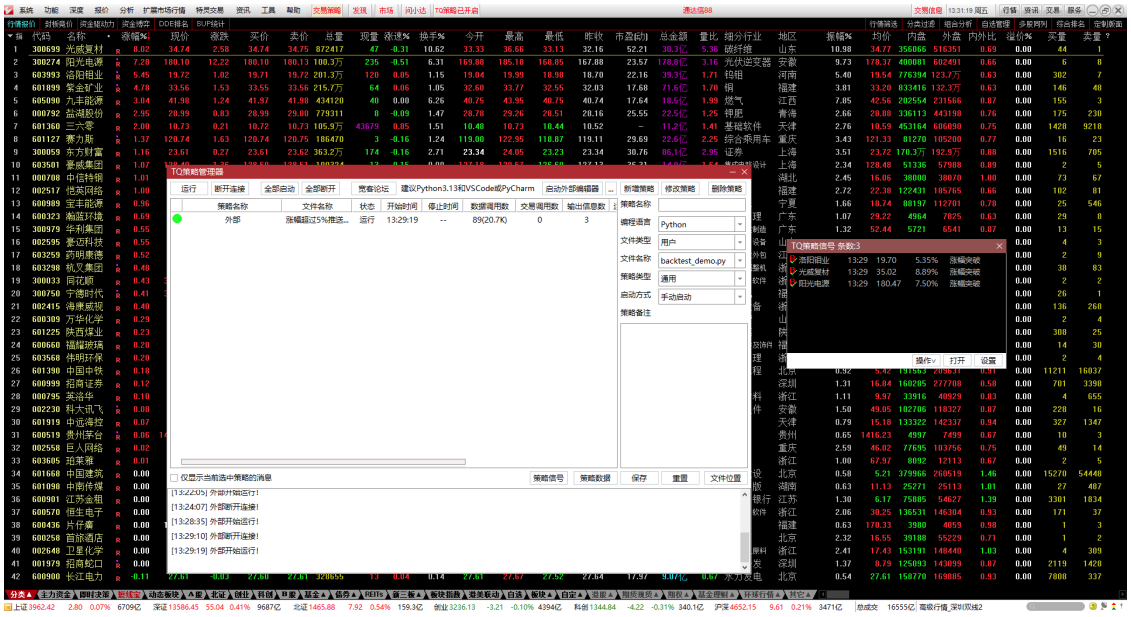
```
            # 仅处理满足涨幅阈值+防抖的情况
            if rise_rate > PRICE_RISE_THRESHOLD:
                current_time = int(time.time())
                if current_time - last_warn_time[code] < ANTI_SHAKE_SECONDS:
                    return

                # 标记为已触发，后续不再处理
                TRIGGERED_STOCKS.add(code)
                last_warn_time[code] = current_time

                # 取消该股票的订阅（不再监控）
                unsubscribe_single_stock(code)

                # 发送预警
                warn_time = datetime.now().strftime("%Y%m%d%H%M%S")
                reason = (
                    f"涨幅突破"
                )

                try:
                    # 成交量用实际值，无则填0
                    volume = report_ptr.get('Volume', '0') if report_ptr else '0'
                    warn_res = tq.send_warn(
                        stock_list=[code],
                        time_list=[warn_time],
                        price_list=[str(latest_price)],
                        close_list=[str(pre_close)],
                        volum_list=[volume],
                        bs_flag_list=['0'],
                        warn_type_list=['3'],
                        reason_list=[reason],
                        count=1
                    )
                    print(f"[{datetime.now().strftime('%H:%M:%S')}] {reason}")
                    print(f"[{datetime.now().strftime('%H:%M:%S')}] 预警发送结果:
{warn_res}")
                    print(f"[{datetime.now().strftime('%H:%M:%S')}] 已取消{code}订
阅，后续不再监控")
                except Exception as e:
                    print(f"\n[{datetime.now().strftime('%H:%M:%S')}] {code} 发送预警
失败: {e}")

        except Exception as e:
            print(f"\n[{datetime.now().strftime('%H:%M:%S')}] 回调函数执行异常: {e}")
            import traceback
            traceback.print_exc()

        return None

    # ==================== 订阅/取消订阅函数====================
    def subscribe_stocks():
        """订阅股票（分批订阅+容错）"""
        if not SUBSCRIBE_CODES:
            print(f"\n[{datetime.now().strftime('%H:%M:%S')}] 无有效股票可订阅，跳过订
阅流程")
            return False
```

```python

    print(f"\n开始批量订阅股票（总计{len(SUBSCRIBE_CODES)}只）")
    sub_result = batch_subscribe(SUBSCRIBE_CODES, BATCH_SUBSCRIBE_SIZE)
    print(f"批量订阅最终结果：{'成功' if sub_result else '部分/全部失败'}")
    return sub_result

def unsubscribe_stocks():
    """取消订阅（分批取消）"""
    if not SUBSCRIBE_CODES:
        print(f"\n[{datetime.now().strftime('%H:%M:%S')}] 无已订阅股票，跳过取消订阅流程")
        return False

    print(f"\n开始批量取消订阅股票（总计{len(SUBSCRIBE_CODES)}只）")
    total_success = True
    for i in range(0, len(SUBSCRIBE_CODES), BATCH_SUBSCRIBE_SIZE):
        batch = SUBSCRIBE_CODES[i:i+BATCH_SUBSCRIBE_SIZE]
        try:
            print(f"取消第{i//BATCH_SUBSCRIBE_SIZE + 1}批订阅：{batch[:5]}...")
            un_sub_ptr = tq.unsubscribe_hq(stock_list=batch)
            if not un_sub_ptr:
                print(f"第{i//BATCH_SUBSCRIBE_SIZE + 1}批取消失败：{un_sub_ptr}")
                total_success = False
        except Exception as e:
            print(f"第{i//BATCH_SUBSCRIBE_SIZE + 1}批取消异常：{e}")
            total_success = False
    print(f"批量取消订阅最终结果：{'成功' if total_success else '部分/全部失败'}")
    return total_success

# ==================== 主程序 ====================
if __name__ == "__main__":
    # 注册SIGINT信号处理（优先于默认的KeyboardInterrupt）
    signal.signal(signal.SIGINT, signal_handler)

    # 1. 初始化TDX（仅执行1次，无重试）
    try:
        tq.initialize(__file__)
        print(f"程序启动时间：{datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
        print(f"TDX初始化成功")
    except Exception as e:
        print(f"TDX初始化失败：{e}")
        exit(1)

    # 2. 获取板块股票列表
    SUBSCRIBE_CODES = get_valid_stock_codes(SECTOR_NAMES)
    if not SUBSCRIBE_CODES:
        print("未获取到任何有效股票代码，程序退出")
        exit(1)

    # 3. 订阅股票
    subscribe_stocks()

    # 4. 运行提示
    print(f"\n=== 涨幅监控启动 ===")
    print(f"监控板块：{SECTOR_NAMES}")
    print(f"监控股票总数：{len(SUBSCRIBE_CODES)}")
```

```
print(f"涨幅阈值: >{PRICE_RISE_THRESHOLD}%")
print(f"防抖间隔: {ANTI_SHAKE_SECONDS}秒")
print(f"分批订阅大小: {BATCH_SUBSCRIBE_SIZE}只/批")
print("按 Ctrl+C 退出程序...\n")

# 5. 通过全局标记控制退出
try:
    while not EXIT_FLAG:
        time.sleep(0.1)
except Exception as e:
    print(f"主循环异常: {e}")
    # 兜底清理
    unsubscribe_stocks()
```

第二步：打开通达信金融终端查看运行结果

通达信金融终端



← 执行选股入板块　　计算调仓信号并快速买卖 →

计算调仓信号并快速买卖

第一步：计算信号并发送预警，以

```py
from datetime import datetime, timedelta
from tqcenter import tq as tdxdata
import vectorbt as vbt
import pandas as pd

# 初始化
tdxdata.initialize(__file__)
run_time = datetime.now()
run_time_str = run_time.strftime("%Y-%m-%d %H:%M:%S")
# 预警时间戳（格式：YYYYMMDDHHMMSS）
warn_time = run_time.strftime("%Y%m%d%H%M%S")

# ====================== 1. 配置参数 ======================
N = 5  # 均线周期
batch_codes = tdxdata.get_stock_list_in_sector('通达信88')
end_date = run_time.strftime("%Y%m%d")
start_date = (run_time - timedelta(days=2 * N + 20)).strftime("%Y%m%d")

# ====================== 2. 获取并处理数据 ======================
# 获取日线Close数据（保留完整索引用于日期筛选）
df_real = tdxdata.get_market_data(
    field_list=['Close'],
    stock_list=batch_codes,
    start_time=start_date,
    end_time=end_date,
    dividend_type='front',
    period='1d',
    fill_data=True
)
close_df = tdxdata.price_df(df_real, 'Close', column_names=batch_codes)

# 计算均线+生成信号
ma = vbt.MA.run(close_df, window=N).ma
ma.columns = close_df.columns
entries = close_df.vbt.crossed_above(ma)  # 上穿（买入）
exits = close_df.vbt.crossed_below(ma)    # 下穿（卖出）
latest_date = close_df.index[-1]  # 今日日期（DataFrame最后一行）
# 获取上一个工作日日期
prev_date = close_df.index[-2] if len(close_df.index) >= 2 else latest_date

# ====================== 3. 筛选最新买卖信号 ======================
buy_signals = {}
sell_signals = {}

```

```python
# 遍历股票筛选信号
for code in batch_codes:
    # 确保股票有足够的交易数据
    if code not in close_df.columns:
        continue

    # 今日收盘价
    today_close = close_df.loc[latest_date, code]
    # 上一个工作日收盘价
    prev_close = close_df.loc[prev_date, code] if len(close_df.index) >= 2 else
today_close

    # 买入信号：最新日期Close上穿均线
    if entries.loc[latest_date, code]:
        buy_signals[code] = {
            'today_close': round(today_close, 2),    # 今日close
            'prev_close': round(prev_close, 2),      # 上一个工作日close
            'ma_price': round(ma.loc[latest_date, code], 2)
        }
    # 卖出信号：最新日期Close下穿均线
    if exits.loc[latest_date, code]:
        sell_signals[code] = {
            'today_close': round(today_close, 2),    # 今日close
            'prev_close': round(prev_close, 2),      # 上一个工作日close
            'ma_price': round(ma.loc[latest_date, code], 2)
        }

# ==================== 4. 生成并发送MSG ====================
def send_msg(content):
    msg = f"MSG,{content}"
    print(msg)
    try:
        tdxdata.send_message(msg)
    except Exception as e:
        print(f"发送失败：{e}")

# 统计行
stat_line = (
    f"运行时间：{run_time_str}，均线周期：{N}天，"
    f"买入信号数：{len(buy_signals)} 只，卖出信号数：{len(sell_signals)} 只"
)

print("\n=== MSG格式（TQ策略管理器显示区域）===")
send_msg(stat_line)

# 处理买入信号
if buy_signals:
    send_msg(f"=== 买入信号（Close上穿{N}日均线）===")
    for idx, (code, info) in enumerate(buy_signals.items(), 1):
        line = f"{idx}. {code}：买入信号，今日Close:{info['today_close']}，昨日
Close:{info['prev_close']}"
        send_msg(line)

# 处理卖出信号
if sell_signals:
    send_msg(f"=== 卖出信号（Close下穿{N}日均线）===")
```

```
    for idx, (code, info) in enumerate(sell_signals.items(), 1):
        line = f"{idx}. {code}: 卖出信号，今日Close:{info['today_close']}，昨日
Close:{info['prev_close']}"
        send_msg(line)

# 无信号的情况
if not buy_signals and not sell_signals:
    send_msg(f"运行时间: {run_time_str}，均线周期: {N}天，无买入或卖出信号")

# ==================== 5. 调用send_warn接口发送预警 ====================
def send_trade_warn():
    """发送买卖信号对应的预警（精简版，仅保留核心逻辑）"""
    # 合并所有信号用于发送预警
    all_signals = []
    if buy_signals:
        all_signals.extend([(code, info, '买入') for code, info in
buy_signals.items()])
    if sell_signals:
        all_signals.extend([(code, info, '卖出') for code, info in
sell_signals.items()])

    if not all_signals:
        print("\n无预警信息需要发送")
        return

    # 构造预警参数列表
    codes = []
    time_list = []
    price_list = []       # 今日close
    close_list = []       # 上一个工作日close
    volum_list = []
    bs_flag_list = []
    warn_type_list = []
    reason_list = []

    for code, info, trade_type in all_signals:
        codes.append(code)
        time_list.append(warn_time)
        price_list.append(str(info['today_close']))    # 替换为今日close
        close_list.append(str(info['prev_close']))     # 替换为上一个工作日close
        volum_list.append('0')
        bs_flag_list.append('0' if trade_type == '买入' else '1')
        warn_type_list.append('1')
        reason_list.append(f"{trade_type}信号")

    # 调用预警接口
    try:
        warn_res = tdxdata.send_warn(
            stock_list=codes,
            time_list=time_list,
            price_list=price_list,
            close_list=close_list,
            volum_list=volum_list,
            bs_flag_list=bs_flag_list,
            warn_type_list=warn_type_list,
            reason_list=reason_list,
```

```
            count=len(codes)
        )
        print(f"\n预警发送完成，共发送 {len(codes)} 条预警，接口返回：{warn_res}")
    except Exception as e:
        print(f"\n预警发送失败：{e}")

# 执行预警发送
send_trade_warn()

print("\n所有消息发送完成！")
tdxdata.close()
```

第二步:双击TQ策略信号，快速打开闪电买卖，根据输出的买/卖信号打开买/卖界面

注意：须保证交易账号已登录。

← 订阅行情涨幅突破实时预警　　结合VBT回测示例 →

VBT简单回测并输出图形

```py
# 注意：
# 1/目前调用的vectorbt三方库函数vbt.Portfolio.from_signals不支持分红送股等权益变动，该demo仅做示例。

import pandas as pd
import vectorbt as vbt    #VSCODE-终端安装1. pip install vectorbt -i https://pypi.tuna.tsinghua.edu.cn/simple    安装2.pip install plotly 用于打印html交互式图形
from tqcenter import tq
from datetime import datetime


tq.initialize(__file__)

# 解决 pandas future warning
pd.set_option('future.no_silent_downcasting', True)
pd.set_option('display.float_format', lambda x: f"{x:.10f}".rstrip('0').rstrip('.') if '.' in f"{x:.10f}" else f"{x}")

# ======================== 核心配置（用户可直接修改这里）========================
target_start = '20250701'  # 【目标回测开始时间】(真正想回测的起始日)
target_end = '20251231'    # 【目标回测结束时间】
stock_code_list = ['688318.SH']     # 股票代码
window = 5          # MA指标周期（如MA5、MA10、MA20，改这里自动适配历史数据）
# ============================================================
start_time = (pd.to_datetime(target_start) - pd.Timedelta(days=window + 10)).strftime('%Y%m%d')

# 1.获取价格数据
df_real = tq.get_market_data(
    field_list=['Close', 'Open'],
    stock_list=stock_code_list,
    start_time=start_time,
    end_time=target_end,
    dividend_type='front',
    period='1d',
    fill_data=True
)
close_df = tq.price_df(df_real, 'Close', column_names=stock_code_list)
open_df = tq.price_df(df_real, 'Open', column_names=stock_code_list)

# 2.买卖信号计算与生成
ma5_dynamic = vbt.MA.run(close_df, window=window).ma.ffill()
ma5_dynamic.columns = close_df.columns

entries_df =
close_df.vbt.crossed_above(ma5_dynamic).shift(1).fillna(False).astype(bool)
```

```
exits_df =
close_df.vbt.crossed_below(ma5_dynamic).shift(1).fillna(False).astype(bool)

print(f"\n信号生成统计:")
print(f"买入信号总数: {entries_df.sum().sum()}")
print(f"卖出信号总数: {exits_df.sum().sum()}")

# 3. 执行回测
portfolio = vbt.Portfolio.from_signals(
    close=close_df,             # 净值计算用未复权收盘价
    entries=entries_df,         # 延迟后的买入信号
    exits=exits_df,             # 延迟后的卖出信号
    price=open_df,              # 含滑点的成交价格
    init_cash=100000,           # 初始资金10万元
    fees=0.0003,                # 手续费0.03%（双边）
    freq='D',                   # 日线频率
    size_granularity=100        # A股最小交易单位100股
)

# ========== vbt绘图 ==========
portfolio[stock_code_list[0]].plot().show()

# 4. 输出回测结果
print(f"\n" + "="*60)
print(f"投资组合回测表现")
print("="*60)
stats_df = portfolio.stats()
print(stats_df)

print(f"\n" + "="*60)
print(f"投资组合回测记录")
print("="*60)
trades_df_original = portfolio.trades.records_readable.copy()
print(trades_df_original.to_string())
```

← 计算调仓信号并快速买卖

20260122公众号文章 →

通达信TQ策略介绍和应用示例

以下是20250122通达信趋势财经公众号发布文章 涉及的策略py文件；
发送序列数据的TQMA510技术指标在stratexldata.py 下面的注释中。

fiststrategy.py

```py
import numpy as np
import pandas as pd
from tqcenter import tq
import time
import json
import os
# 将工作目录切换到当前脚本文件所在的目录
os.chdir(os.path.dirname(os.path.abspath(__file__)))
# 之后，相对路径就会基于脚本所在目录进行解析


"""
    这里是tq的简单使用示例
    使用时请确保已经启动通达信客户端并登录
    取消对应注释即可运行对应功能
"""

"""
    参数设置
"""
codes = ["688318.SH"] #传入的股票代码格式必须是标准格式：6位数+市场后缀（.SH/.SZ/.JJ等）
startime = "20250620" #传入的时间格式必须是：YYYYMMDD 或 YYYYMMDDHHMMSS
endtime = "20250801"
period = '1d' #K线周期：1d/1w/1m/5m/15m/30m/60m等
dividend_type='none' #复权类型：none-不复权，front-前复权，back-后复权

#初始化
tq.initialize(__file__) #所有策略连接通达信客户端都必须调用此函数进行初始化

'''
    刷新行情缓存 刷新后5分钟内取最新report和k线数据不会触发刷新
'''
# refresh_cache = tq.refresh_cache()
# print(refresh_cache)

'''
    缓存历史K线 目前仅支持1m 5m 1d三种类型数据 不建议一次更新太多，会堵塞策略和客户端
'''
# refresh_kline = tq.refresh_kline(stock_list=['688318.SH'],period='1d')
# print(refresh_kline)
```

```
'''
    获取K线数据  获取K线数据需要先在客户端中下载对应盘后数据，调用会触发客户端刷新数据，耗时过长请耐心等待
    field_list可以筛选返回字段，默认返回全部字段 比如 field_list=['Open','Close'] 就是只取开盘价和收盘价
    count可以设置每只股票取的数据量
    暂时不支持获取分笔数据
    开高低收单位为元，成交量单位为手，成交额单位为万元
'''
df = tq.get_market_data(
        field_list=[],
        stock_list=['600519.SH'],
        start_time='20251208',
        end_time='20251210',
        count=-1,
        dividend_type='none',
        period='1d',
        fill_data=False
    )
print(df)
```

strategywarn.py

```py
import numpy as np
import pandas as pd
from tqcenter import tq
import time
import json

"""
    这里是tq的简单使用示例
    使用时请确保已经启动通达信客户端并登录
    取消对应注释即可运行对应功能
"""

"""
    参数设置
"""
codes = ["688318.SH"] #传入的股票代码格式必须是标准格式：6位数+市场后缀（.SH/.SZ/.JJ等）
startime = "20250620" #传入的时间格式必须是：YYYYMMDD 或 YYYYMMDDHHMMSS
endtime = "20250801"
period = '1d' #K线周期：1d/1w/1m/5m/15m/30m/60m等
dividend_type='none' #复权类型：none-不复权，front-前复权，back-后复权

#初始化
tq.initialize(__file__) #所有策略连接通达信客户端都必须调用此函数进行初始化


'''
    发送预警信号给通达信客户端的TQ策略界面
    price_list close_list volum_list bs_flag_list warn_type_list 均要求为纯数字字
```

```
符串List
    bs_flag_list 0买1卖2未知
    reason_list每个元素有效长度为25个汉字（50个英文）
'''
warn_res = tq.send_warn(stock_list =
['688318.SH','688318.SH','600519.SH','600519.SH'],
            time_list =
['20251215141115','20251215142100','20251215143101','20251215145001'],
            price_list= ['123.45','133.45','1823.45','1853.45'],
            close_list= ['122.50','132.50','1822.50','1822.50'],
            volum_list= ['1000','2000','15000','15000'],
            bs_flag_list= ['0','','2','1'],
            warn_type_list= ['0','','2','1'],
            reason_list= ['价格突破预警线','收盘价突破预警线','成交量突破预警线','价
格下破预警线'],
            count=4)
print(warn_res)
```

TQHuiCe.py

```py
import numpy as np
import pandas as pd
from tqcenter import tq
import time
import json
"""
    这里是tq的简单使用示例
    使用时请确保已经启动通达信客户端并登录
    取消对应注释即可运行对应功能
"""
#初始化
tq.initialize(__file__) #所有策略连接通达信客户端都必须调用此函数进行初始化
bt_data = tq.send_bt_data(stock_code = '688318.SH',
                          time_list = ['20260120141100','20260120141400'],
                          data_list = [['1','143.41','200','0','0','0'],
['0','0','0','1','143.48','200']],
                          count = 2)
print(bt_data)
```

stratexldata.py

```py
#展示每日持仓量能看数据 验证开仓和手续费的    #用悬挂目录的时候  读不出来
from tqcenter import tq
import pandas as pd
import numpy as np
import math
from datetime import datetime
import sys


# ==================== 技术指标计算函数 ====================
def calculate_ema(series, window):
    """计算指数移动平均"""
```

```python
    return series.ewm(span=window, adjust=False).mean()

def calculate_sma(series, window):
    """计算简单移动平均"""
    return series.rolling(window=window).mean()

def calculate_llv(series, window):
    """计算周期内最低值"""
    return series.rolling(window=window).min()

def calculate_hhv(series, window):
    """计算周期内最高值"""
    return series.rolling(window=window).max()

def calculate_ma(series, window):
    """计算简单移动平均 (别名，与calculate_sma功能相同)"""
    return calculate_sma(series, window)

def ref(series, periods):
    """引用若干周期前的数据"""
    return series.shift(periods)

def calculate_cross_signal(fast_series, slow_series):
    """
    计算金叉信号序列。
    规则：当快线从下方上穿慢线时，标记为1（金叉），否则为0。
    注意：这是一个简单的信号，未考虑信号持续期。
    """
    # 判断条件：今日快线 > 慢线，且昨日快线 <= 慢线
    cross_up = (fast_series > slow_series) & (ref(fast_series, 1) <=
ref(slow_series, 1))
    return cross_up.astype(int)

def get_benchmark_data(benchmark_code='000300.SH', count=60):
    """
    获取基准品种数据
    """
    try:
        # 获取基准市场数据
        benchmark_data = tq.get_market_data(
            field_list=['Open', 'High', 'Low', 'Close'],
            stock_list=[benchmark_code],
            period='1d',
            count=count,
            dividend_type='front'  # 前复权数据
        )

        # 提取收盘价序列
        benchmark_close = benchmark_data['Close'][benchmark_code]

        # 计算基准收益率序列
        benchmark_returns = benchmark_close.pct_change().fillna(0)

        # 计算基准净值（从1开始）
        benchmark_net_value = (1 + benchmark_returns).cumprod()

```

```
        # 计算基准年化收益率
        trading_days = len(benchmark_net_value)
        if trading_days > 0:
            benchmark_total_return = benchmark_net_value.iloc[-1] - 1
            benchmark_annual_return = (1 + benchmark_total_return) ** (252 /
trading_days) - 1
        else:
            benchmark_total_return = 0
            benchmark_annual_return = 0

        return {
            'close': benchmark_close,
            'returns': benchmark_returns,
            'net_value': benchmark_net_value,
            'total_return': benchmark_total_return,
            'annual_return': benchmark_annual_return
        }

    except Exception as e:
        print(f"获取基准数据时发生错误: {e}")
        # 返回空数据
        return {
            'close': pd.Series(),
            'returns': pd.Series(),
            'net_value': pd.Series(),
            'total_return': 0,
            'annual_return': 0
        }

def calculate_daily_statistics(df, benchmark_data, initial_capital=100000,
fee_rate=0.0003):
    """
    计算每日的回测统计指标序列
    输入:
        df: 包含价格数据、买卖信号的DataFrame
        benchmark_data: 基准数据字典
        initial_capital: 初始资金
        fee_rate: 手续费率（双边）
    输出: 包含每日统计指标的DataFrame
    """
    # 初始化变量
    capital = initial_capital
    position = 0  # 持仓数量
    rest_cash = capital  # 剩余现金
    hold = False  # 是否持仓

    # 创建结果列表
    daily_stats = []
    open_count = 0  # 累计开仓次数
    close_count = 0  # 累计平仓次数
    win_count = 0  # 累计盈利交易次数
    trade_records = []  # 交易记录

    # 获取基准数据
    benchmark_close = benchmark_data['close']
    benchmark_net_value = benchmark_data['net_value']
```

```python

    # 确保基准数据与策略数据时间对齐
    if len(benchmark_close) != len(df):
        print(f"警告：基准数据长度({len(benchmark_close)})与策略数据长度({len(df)})
不一致")
        # 这里简单处理，取相同长度的数据
        min_len = min(len(benchmark_close), len(df))
        benchmark_close = benchmark_close.iloc[:min_len]
        benchmark_net_value = benchmark_net_value.iloc[:min_len]
        df = df.iloc[:min_len]

    # 遍历数据执行交易
    for i in range(len(df)):
        current_price = df['close'].iloc[i]
        buy_signal = df['buyxh'].iloc[i] if i < len(df) else 0
        sell_signal = df['sellxh'].iloc[i] if i < len(df) else 0

        # 记录交易前的状态
        pre_open_count = open_count
        pre_close_count = close_count
        pre_win_count = win_count

        # 金叉买入信号
        if buy_signal == 1 and not hold:
            # 计算可买整百股数量
            max_shares = int(rest_cash / current_price / 100) * 100
            if max_shares > 0:
                # 计算手续费
                trade_amount = max_shares * current_price
                fee = trade_amount * fee_rate

                # 执行买入
                position = max_shares
                rest_cash = rest_cash - trade_amount - fee
                hold = True
                open_count += 1

                # 记录交易
                trade_records.append({
                    'date': df.index[i],
                    'type': 'buy',
                    'price': current_price,
                    'shares': position,
                    'fee': fee
                })

        # 死叉卖出信号
        elif sell_signal == 1 and hold:
            # 计算卖出金额
            trade_amount = position * current_price
            fee = trade_amount * fee_rate

            # 执行卖出
            rest_cash = rest_cash + trade_amount - fee
            position = 0
            hold = False
```

```
            close_count += 1

            # 检查交易是否盈利
            if len(trade_records) > 0 and trade_records[-1]['type'] == 'buy':
                buy_price = trade_records[-1]['price']
                if current_price > buy_price:
                    win_count += 1

            # 记录交易
            trade_records.append({
                'date': df.index[i],
                'type': 'sell',
                'price': current_price,
                'shares': position,
                'fee': fee
            })

        # 计算当日市值和净值
        if hold:
            daily_value = rest_cash + position * current_price
        else:
            daily_value = rest_cash

        daily_net_value = daily_value / initial_capital

        # 计算胜率（到当前日期为止）
        total_trades_to_date = open_count + close_count
        win_rate_to_date = (win_count / total_trades_to_date * 100) if
total_trades_to_date > 0 else 0

        # 计算年化收益率（到当前日期为止）
        if i > 0:
            total_return_to_date = daily_net_value - 1
            trading_days_to_date = i + 1
            annual_return_to_date = (1 + total_return_to_date) ** (252 /
trading_days_to_date) - 1 if trading_days_to_date > 0 else 0
        else:
            annual_return_to_date = 0

        # 获取基准净值（使用真实的基准数据）
        if i < len(benchmark_net_value):
            current_benchmark_net_value = benchmark_net_value.iloc[i]
        else:
            current_benchmark_net_value = 1.0

        # 计算基准年化收益率（到当前日期为止）
        if i > 0 and i < len(benchmark_net_value):
            benchmark_total_return_to_date = current_benchmark_net_value - 1
            benchmark_annual_return_to_date = (1 +
benchmark_total_return_to_date) ** (252 / (i + 1)) - 1 if i > 0 else 0
        else:
            benchmark_annual_return_to_date = 0

        # 计算贝塔值（到当前日期为止）
        if i > 1:
            # 计算策略收益率序列
```

```
            strategy_returns = []
            for j in range(i + 1):
                if j == 0:
                    strategy_returns.append(0)
                else:
                    prev_capital = daily_stats[j-1]['capital'] if j > 0 else
initial_capital
                    curr_capital = daily_value if j == i else daily_stats[j]
['capital']
                    strategy_return = (curr_capital / prev_capital) - 1
                    strategy_returns.append(strategy_return)

            # 计算基准收益率序列
            benchmark_returns_to_date =
benchmark_close.iloc[:i+1].pct_change().fillna(0).values

            # 计算协方差和方差
            if len(strategy_returns) > 1 and len(benchmark_returns_to_date) > 1:
                cov_matrix = np.cov(strategy_returns, benchmark_returns_to_date)
                beta = cov_matrix[0, 1] / cov_matrix[1, 1] if cov_matrix[1, 1]
!= 0 else 1.0
            else:
                beta = 1.0
        else:
            beta = 1.0

        # 收集每日统计数据
        daily_stats.append({
            'date': df.index[i],
            'capital': daily_value,
            'net_value': daily_net_value,
            'open_count': open_count,
            'close_count': close_count,
            'win_rate': win_rate_to_date,
            'annual_return': annual_return_to_date * 100,
            'benchmark_net_value': current_benchmark_net_value,
            'benchmark_annual_return': benchmark_annual_return_to_date * 100,
            'beta': beta,
            'position': position,  # 持仓量
            'hold': hold
        })

    # 转换为DataFrame
    stats_df = pd.DataFrame(daily_stats)
    stats_df.set_index('date', inplace=True)

    # 计算最大回撤序列
    if len(stats_df) > 0:
        stats_df['rolling_max'] = stats_df['net_value'].cummax()
        stats_df['drawdown'] = (stats_df['net_value'] - stats_df['rolling_max'])
/ stats_df['rolling_max']
        stats_df['max_drawdown'] = stats_df['drawdown'].cummin()

    # 计算夏普比率序列
    if len(stats_df) > 1:
        # 计算策略收益率
```

```
        returns_list = []
        for i in range(len(stats_df)):
            if i == 0:
                returns_list.append(0)
            else:
                prev_capital = stats_df['capital'].iloc[i-1]
                curr_capital = stats_df['capital'].iloc[i]
                returns_list.append((curr_capital / prev_capital) - 1)

        stats_df['returns'] = returns_list

        # 计算滚动夏普比率
        sharpe_list = []
        for i in range(len(stats_df)):
            if i == 0:
                sharpe_list.append(0)
            else:
                returns_to_date = stats_df['returns'].iloc[:i+1]
                # 使用2%作为无风险利率
                risk_free_rate = 0.02
                excess_returns = returns_to_date - risk_free_rate/252
                sharpe = excess_returns.mean() * math.sqrt(252) /
returns_to_date.std() if returns_to_date.std() != 0 else 0
                sharpe_list.append(sharpe)
        stats_df['sharpe_ratio'] = sharpe_list

    return stats_df, trade_records

# ==================== 主程序 ====================
def main():
    # 初始化TQ
    tq.initialize(__file__)

    # 股票列表（示例）
    stocks = ['688800.SH', '688318.SH', '688981.SH']
    # 基准品种代码
    benchmark_code = '000300.SH'

    # 使用for循环遍历股票列表
    for stock_code in stocks:
        print(f"处理股票: {stock_code}")
        print("-" * 50)

        try:
            # 获取股票市场数据
            market_data = tq.get_market_data(
                field_list=['Open', 'High', 'Low', 'Close'],
                stock_list=[stock_code],
                period='1d',
                count=60,
                dividend_type='front'  # 前复权数据
            )

            # 构建DataFrame
            df = pd.DataFrame({
                'open': market_data['Open'][stock_code],
```

```
                'high': market_data['High'][stock_code],
                'low': market_data['Low'][stock_code],
                'close': market_data['Close'][stock_code]
            })

            print("原始K线数据前5行:")
            print(df.head())
            print("-" * 50)

            # 计算技术指标
            df['ma5'] = calculate_ma(df['close'], 5)
            df['ma10'] = calculate_ma(df['close'], 10)

            # 计算金叉信号
            df['buyxh'] = calculate_cross_signal(df['ma5'], df['ma10'])
            df['sellxh'] = calculate_cross_signal(df['ma10'], df['ma5'])

            print("添加技术指标与信号后的数据前15行:")
            print(df[['close', 'ma5', 'ma10', 'buyxh', 'sellxh']].head(15))
            print("-" * 50)

            # 获取基准数据
            print(f"获取基准品种 {benchmark_code} 数据...")
            benchmark_data = get_benchmark_data(benchmark_code, count=len(df))

            if len(benchmark_data['close']) == 0:
                print("警告：未能获取基准数据，使用简化计算")
                # 使用简化基准计算
                benchmark_close = df['close']
                benchmark_returns = benchmark_close.pct_change().fillna(0)
                benchmark_net_value = (1 + benchmark_returns).cumprod()
                benchmark_data = {
                    'close': benchmark_close,
                    'returns': benchmark_returns,
                    'net_value': benchmark_net_value,
                    'total_return': benchmark_net_value.iloc[-1] - 1 if
len(benchmark_net_value) > 0 else 0,
                    'annual_return': 0
                }

            print(f"基准数据获取成功，共{len(benchmark_data['close'])}个交易日")
            print(f"基准总收益率: {benchmark_data['total_return']*100:.2f}%")
            print(f"基准年化收益率: {benchmark_data['annual_return']*100:.2f}%")
            print("-" * 50)

            # 计算每日回测统计指标序列
            stats_df, trade_records = calculate_daily_statistics(
                df,
                benchmark_data,
                initial_capital=100000,
                fee_rate=0.0003  # 0.03%手续费
            )

            # 输出最后一天的统计结果
            if len(stats_df) > 0:
                last_stats = stats_df.iloc[-1]
```

```
                print("最终回测统计指标:")
                print(f"开仓次数: {last_stats['open_count']}")
                print(f"平仓次数: {last_stats['close_count']}")
                print(f"单位净值: {last_stats['net_value']:.4f}")
                print(f"基准净值: {last_stats['benchmark_net_value']:.4f}")
                print(f"胜率: {last_stats['win_rate']:.2f}%")
                print(f"年化收益率: {last_stats['annual_return']:.2f}%")
                print(f"基准年化收益率:
{last_stats['benchmark_annual_return']:.2f}%")
                print(f"贝塔值: {last_stats['beta']:.4f}")
                print(f"最大回撤: {last_stats['max_drawdown']*100:.2f}%")
                print(f"夏普比率: {last_stats['sharpe_ratio']:.4f}")
                print(f"持仓量: {last_stats['position']}股")
                print("-" * 50)

            # 准备发送给TQ的数据
            time_list = df.index.strftime('%Y%m%d').tolist()
            # print(time_list)
            # 扩展data_list，包含所有需要的指标（每日序列）
            data_list = []
            for i, (_, row) in enumerate(df.iterrows()):
                # 基础技术指标
                ma5_value = row['ma5'] if not pd.isna(row['ma5']) else 0.0
                ma10_value = row['ma10'] if not pd.isna(row['ma10']) else 0.0
                buyxh_value = row['buyxh'] if not pd.isna(row['buyxh']) else 0
                sellxh_value = row['sellxh'] if not pd.isna(row['sellxh']) else
0

                # 获取该日期的回测指标
                if i < len(stats_df):
                    daily_stats = stats_df.iloc[i]
                    open_count_val = daily_stats['open_count']
                    close_count_val = daily_stats['close_count']
                    net_value_val = daily_stats['net_value']
                    benchmark_net_val = daily_stats['benchmark_net_value']
                    win_rate_val = daily_stats['win_rate']
                    annual_return_val = daily_stats['annual_return']
                    benchmark_annual_val =
daily_stats['benchmark_annual_return']
                    beta_val = daily_stats['beta']
                    max_drawdown_val = daily_stats['max_drawdown'] * 100  # 转换
为百分比
                    sharpe_val = daily_stats['sharpe_ratio'] if 'sharpe_ratio'
in daily_stats else 0
                    capital_val = daily_stats['capital']
                    position_val = daily_stats['position']  # 持仓量
                else:
                    # 默认值
                    open_count_val = 0
                    close_count_val = 0
                    net_value_val = 1.0
                    benchmark_net_val = 1.0
                    win_rate_val = 0
                    annual_return_val = 0
                    benchmark_annual_val = 0
                    beta_val = 1.0
```

```
                    max_drawdown_val = 0
                    sharpe_val = 0
                    capital_val = 100000
                    position_val = 0

                # 构建数据条目
                formatted_entry = [
                    f"{ma5_value:.2f}",                    # ID 1: MA5
                    f"{ma10_value:.2f}",                   # ID 2: MA10
                    str(int(buyxh_value)),                 # ID 3: 买入信号
                    str(int(sellxh_value)),                # ID 4: 卖出信号
                    f"{open_count_val}",                   # ID 5: 开仓次数（累计
到当前日期）
                    f"{close_count_val}",                  # ID 6: 平仓次数（累计
到当前日期）
                    f"{net_value_val:.4f}",                # ID 7: 单位净值（当前
日期）
                    f"{benchmark_net_val:.4f}",            # ID 8: 基准净值（当前
日期）
                    f"{win_rate_val:.2f}",                 # ID 9: 胜率（累计到当
前日期）
                    f"{annual_return_val:.2f}",            # ID 10: 年化收益率（累
计到当前日期）
                    f"{benchmark_annual_val:.2f}",         # ID 11: 基准年化收益率
（累计到当前日期）
                    f"{beta_val:.4f}",                     # ID 12: 贝塔值
                    f"{max_drawdown_val:.2f}",             # ID 13: 最大回撤（累计
到当前日期）
                    f"{sharpe_val:.4f}",                   # ID 14: 夏普比率（累计
到当前日期）
                    f"{capital_val:.2f}",                  # ID 15: 每日资金
                    f"{position_val}"                      # ID 16: 持仓量（每日持
仓数量）
                ]
                data_list.append(formatted_entry)

            print(f"准备发送的data_list (前5个周期，共{len(data_list)}个周期):")
            for i in range(min(5, len(data_list))):
                print(f"日期 {time_list[i]}: {data_list[i]}")
            print("-" * 50)

            # 发送回测数据到TQ
            bt_data = tq.send_bt_data(
                stock_code,
                time_list=time_list,
                data_list=data_list,
                count=60
            )
            print("发送回测数据结果:")
            print(bt_data)
            print("-" * 50)

            # 输出交易记录
            if trade_records:
                print("交易记录:")
                for record in trade_records:
```

```
                    print(f"{record['date']}: {record['type']}
{record['shares']}股 @ {record['price']:.2f}, 手续费: {record['fee']:.2f}")

            print(f"股票 {stock_code} 处理完成! ")
            print("=" * 60)

        except Exception as e:
            print(f"处理股票 {stock_code} 时发生错误: {e}")
            print("跳过该股票，继续处理下一个...")
            print("-" * 50)
            continue

    # 关闭TQ连接
    tq.close()
    print("所有股票处理完毕! ")
    print("程序执行完毕。")

    # ==================== 通达信公式使用提示 ====================
    print("\n" + "="*60)
    print("通达信公式管理器中使用提示:")
    print("="*60)
    print("""
将数据发送到TQ策略界面后，您可以在通达信公式管理器中创建技术指标公式，
使用 SIGNALS_TQ(ID, TYPE) 函数来引用这些序列数据并在K线上展示。

例如，创建一个名为"TQMA510"的公式，代码可以如下:

MA5:SIGNALS_TQ(1,0);        {引用ID=1的数据(MA5)}
MA10:SIGNALS_TQ(2,0);       {引用ID=2的数据(MA10)}

{交易信号}
BUY_SIGNAL:=SIGNALS_TQ(3,0); {买入信号}
SELL_SIGNAL:=SIGNALS_TQ(4,0);{卖出信号}

{回测指标展示 - 这些指标会随着时间轴移动而动态变化}
开仓次数:SIGNALS_TQ(5,0),COLORRED;
平仓次数:SIGNALS_TQ(6,0),COLORGREEN;
单位净值:SIGNALS_TQ(7,0),COLORWHITE;
基准净值:SIGNALS_TQ(8,0),COLORYELLOW;
胜率:SIGNALS_TQ(9,0),COLORMAGENTA;
年化收益率:SIGNALS_TQ(10,0),COLORCYAN;
基准年化收益率:SIGNALS_TQ(11,0),COLORLIBLUE;
贝塔值:SIGNALS_TQ(12,0),COLORBROWN;
最大回撤:SIGNALS_TQ(13,0),COLORGRAY;
夏普比率:SIGNALS_TQ(14,0),COLORLIMAGENTA;
每日资金:SIGNALS_TQ(15,0),COLORLIGRAY;
持仓量:SIGNALS_TQ(16,0),COLORLIRED;  {显示每日持仓量}

{绘制交易信号图标}
DRAWICON(BUY_SIGNAL, LOW, 1);
DRAWICON(SELL_SIGNAL, HIGH, 2);


函数说明:
SIGNALS_TQ(ID, TYPE)
    ID: TQ数据中的序号 (1-16)，对应data_list子列表中的位置。
```

```
    TYPE: 处理方式。
        1 - 平滑处理，没有自定义数据的周期返回上一周期的值。
        0 - 不做平滑处理。
        2 - 没有数据则为0。
    """)
    print("="*60)

if __name__ == "__main__":
    main()
```

sendfile.py

```py

import numpy as np
import pandas as pd
from tqcenter import tq
import time
import json
"""
    这里是tq的简单使用示例
    使用时请确保已经启动通达信客户端并登录
    取消对应注释即可运行对应功能
"""
#初始化
tq.initialize(__file__) #所有策略连接通达信客户端都必须调用此函数进行初始化
file = "513100.txt"
tq.send_file(file)
```

sendfilepdf.py

```py
import numpy as np
import pandas as pd
from tqcenter import tq
import time
import json
"""
    这里是tq的简单使用示例
    使用时请确保已经启动通达信客户端并登录
    取消对应注释即可运行对应功能
"""
#初始化
tq.initialize(__file__) #所有策略连接通达信客户端都必须调用此函数进行初始化
file = "min.pdf"
tq.send_file(file)
```

← 结合VBT回测示例

20260302公众号文章 →

打通通达信量化任督二脉：公式与Python双向数据互通闭环

以下是[20250122通达信趋势财经公众号发布文章]涉及的策略例子的完整代码。

> 利用MACD公式筛选金叉信号.py
> 用途：批量处理版本，一次计算全市场MACD指标，筛选金叉股票

```py
from tqcenter import tq

'''
    利用此示例需要先在客户端下载全A股盘后数据，不然结果不准确
    通过MACD指标公式选出最新交易日金叉的股票
'''

tq.initialize(__file__)

#先获取A股全部股票
all_stocks = tq.get_stock_list(market='5')

print("正在处理，请等待...")
import time

# 开始计时
start_time = time.time()

macd_stocks = []
mul_zb_result = tq.formula_process_mul_zb(
    formula_name='MACD',
    formula_arg='12,26,9',
    xsflag=6,
    return_count=2,
    return_date=False,
    stock_list=all_stocks,
    # stock_list=['600722.SH'],
    stock_period='1d',
    count=100,
    dividend_type=1)
# print(mul_zb_result)

if mul_zb_result:
    for key in mul_zb_result:
        if key != "ErrorId":
            if len(mul_zb_result[key]['DIF']) >= 2 and len(mul_zb_result[key]['DEA']) >= 2:
                if float(mul_zb_result[key]['DIF'][-2]) < float(mul_zb_result[key]['DEA'][-2]) and float(mul_zb_result[key]['DIF'][-1]) >= float(mul_zb_result[key]['DEA'][-1]):
                    macd_stocks.append(key)
```

```
42
43
44  print("今日MACD金叉股票列表: ")
45  print(macd_stocks)
46  print("符合MACD金叉条件的股票数量: ", len(macd_stocks))
47  # 结束计时
48  end_time = time.time()
49
50  # 计算时间差
51  execution_time = end_time - start_time
52  print(f"执行时间: {execution_time:.6f} 秒")  # 保留6位小数
    print(f"执行时间: {execution_time * 1000:.2f} 毫秒")  # 转换为毫秒

    zxg_result = tq.send_user_block(block_code='', stocks=macd_stocks)
```

利用MACD公式筛选金叉信号for单.py

用途：for循环版本，逐只股票计算MACD指标，适合小数量测试对比

```py
1   from tqcenter import tq
2
3   '''
4
5       利用此示例需要先在客户端下载全A股盘后数据，不然结果不准确
6
7       通过MACD指标公式选出最新交易日金叉的股票
8
9   '''
10  tq.initialize(__file__)
11
12  #先获取A股全部股票
13
14  all_stocks = tq.get_stock_list(market='5')
15
16  print("正在处理，请等待...")
17
18  import time
19
20  # 开始计时
21
22  start_time = time.time()
23
24  macd_stocks = []
25
26  for stock in all_stocks:
27
28      try:
29
30          # 1. 设置股票数据
31
32          tq.formula_set_data_info(
33              stock_code=stock,
34              stock_period='1d',
35              count=100,  # 需要足够的数据计算MACD
36              dividend_type=1  # 前复权
```

```
        )

        # 2. 获取MACD指标

        macd_result = tq.formula_zb(
            formula_name='MACD',
            formula_arg='12,26,9',
            xsflag=6
        )

        # 3. 获取DIF和DEA值，判断金叉

        if macd_result and 'Data' in macd_result:
            dif_values = macd_result['Data']['DIF']
            dea_values = macd_result['Data']['DEA']

            if len(dif_values) >= 2 and len(dea_values) >= 2:
                dif_prev = float(dif_values[-2])  # 前一天的DIF
                dif_now = float(dif_values[-1])   # 今天的DIF
                dea_prev = float(dea_values[-2])  # 前一天的DEA
                dea_now = float(dea_values[-1])   # 今天的DEA

                # MACD金叉信号：昨天DIF<DEA，今天DIF>=DEA
                if dif_prev < dea_prev and dif_now >= dea_now:
                    macd_stocks.append(stock)
                    print(f"MACD金叉信号: {stock}, DIF: {dif_prev:.4f}
→{dif_now:.4f}, DEA: {dea_prev:.4f}→{dea_now:.4f}")

    except Exception as e:
        print(f"处理股票 {stock} 时出错: {e}")
        continue

print("今日MACD金叉股票列表: ")
print(macd_stocks)
print("符合MACD金叉条件的股票数量: ", len(macd_stocks))

# 结束计时

end_time = time.time()

# 计算时间差

execution_time = end_time - start_time

print(f"执行时间: {execution_time:.6f} 秒")  # 保留6位小数
print(f"执行时间: {execution_time * 1000:.2f} 毫秒")  # 转换为毫秒

zxg_result = tq.send_user_block(block_code='', stocks=macd_stocks)
```

订阅Handlebar.py

用途：订阅方式实时预警，使用subscribe_hq，每6秒运行一次回调（适合100只以内股票）

```py
import datetime

```

```python
import json

import sys

sys.path.append('C:/new_tdx_test2025/PYPlugins/user')

from tqcenter import tq

tq.initialize('订阅Handlebar.py')

"""

这里是外部运行的初始化模式把tqcenter目录加上，再import它，    initialize参数****.py标准的py文件名就行

"""

# 获取A股全部股票（测试时限制数量）# 订阅不能超过100只

all_stocks = tq.get_stock_list(market='5')[:50]

def get_real_time_data(stock_code):

    """

    获取股票的实时行情数据

    根据通达信TQ接口文档，这里需要调用相应的数据获取函数

    """

    try:

        # 获取最近两天的数据，用于获取前一日收盘价

        market_data = tq.get_market_data(

            field_list=['Close', 'Volume'],

            stock_list=[stock_code],

            count=2,  # 获取2天数据，用于获取前一日收盘价

            period='1d',

            dividend_type='none',

            fill_data=True

        )

        if market_data and 'Close' in market_data:

            close_df = market_data['Close']

            if not close_df.empty:
```

```
                # 获取最新收盘价
                last_price = close_df.iloc[-1][stock_code]
                # 获取前一日收盘价
                if len(close_df) >= 2:
                    prev_close = close_df.iloc[-2][stock_code]
                else:
                    prev_close = '0.00'
                # 获取成交量
                if 'Volume' in market_data:
                    volume_df = market_data['Volume']
                    volume = volume_df.iloc[-1][stock_code] if not
volume_df.empty else '0'
                else:
                    volume = '0'
                return str(last_price), str(prev_close), str(volume)
    except Exception as e:
        print(f"获取{stock_code}实时数据失败: {e}")
    return '0.00', '0.00', '0'
def my_callback_func(data_str):
    print("Callback received data:", data_str)
    code_json = json.loads(data_str)
    print(f"codes = {code_json.get('Code')}")
    upn_stocks = []  # 用于存放符合UPN公式选股条件的股票列表
    for stock in all_stocks:
        formula_set_res = tq.formula_set_data_info(
            stock_code=stock,
            stock_period='1d',
            count=20,
```

```
            dividend_type=1

        )

        if formula_set_res:

            # 使用UPN公式选股，参数'3'表示3日上涨

            formula_xg = tq.formula_xg(formula_name='UPN', formula_arg='3')

            print(f"formula_xg = {formula_xg}")

            if formula_xg and 'Data' in formula_xg and 'UP3' in
formula_xg['Data']:

                up3_data = formula_xg['Data']['UP3']

                if up3_data and len(up3_data) > 0 and up3_data[-1] is not None:

                    if up3_data[-1] != '0':  #之前版本是0.00也行 这里要改一下

                        upn_stocks.append(stock)

    print("符合UPN公式选股条件的股票列表: ")

    print(upn_stocks)

    print("符合UPN公式选股条件的股票数量: ", len(upn_stocks))

    # 为选出的股票发送预警

    if upn_stocks:

        send_warnings_for_stocks(upn_stocks)

    return None

def send_warnings_for_stocks(stock_list):

    """为股票列表发送预警信息"""

    if not stock_list:

        return

    # 获取当前时间，格式化为YYYYMMDDHHMMSS

    current_time = datetime.datetime.now().strftime("%Y%m%d%H%M%S")

    # 准备预警参数列表

    stock_count = len(stock_list)

    # 初始化列表

    time_list = []
```

```
171
172         price_list = []
173
174         close_list = []
175
176         volum_list = []
177
178         # 为每只股票获取实时数据
179
180         for stock in stock_list:
181
182             last_price, prev_close, volume = get_real_time_data(stock)
183
184             time_list.append(current_time)
185
186             price_list.append(last_price)
187
188             close_list.append(prev_close)
189
190             volum_list.append(volume)
191
192         # 其他固定参数
193
194         reason_list = ["3天连涨"] * stock_count  # 根据实际选股条件修改
195
196         bs_flag_list = ['0'] * stock_count  # 买卖标志：0
197
198         warn_type_list = ['0'] * stock_count  # 预警类型：0
199
200         # 调用send_warn函数发送预警
201
202         try:
203
204             warn_res = tq.send_warn(
205
206                 stock_list=stock_list,
207
208                 time_list=time_list,
209
210                 price_list=price_list,
211
212                 close_list=close_list,
213
214                 volum_list=volum_list,
215
216                 bs_flag_list=bs_flag_list,
217
218                 warn_type_list=warn_type_list,
219
220                 reason_list=reason_list,
221
222                 count=stock_count
223
224             )
225
226             print("预警发送结果：", warn_res)
```

```
        # 根据搜索结果[6](@ref)，预警发送成功后会在TQ策略信号窗口展示

        # 预警图标对应bs_flag_list的每个元素的整数值，0买为红色B，1卖为绿色S，2未知为黄色双叠三角形

        # 双击买卖预警信号记录可以直接打开闪电下单进行买卖操作

    except Exception as e:

        print(f"发送预警失败: {e}")

# 订阅行情

sub_hq = tq.subscribe_hq(stock_list=all_stocks, callback=my_callback_func)

print("订阅结果: ", sub_hq)

# 可选：设置定时执行或保持运行

import time

try:

    print("程序运行中，按Ctrl+C停止...")

    while True:

        time.sleep(60)  # 每分钟检查一次

except KeyboardInterrupt:

    print("程序终止")

    # 取消订阅

    if sub_hq:

        tq.unsubscribe_hq(stock_list=all_stocks)

tq.close()
```

> 📄 **定时器实时预警Handlebar效果.py**
> 用途：定时器方式实时预警，每分钟运行一次（可覆盖全市场几千只股票）

```py
#定时器实时预警

定时器实时预警Handlebar效果.py
```python
#定时器实时预警

import datetime

import json
```
```

```python

import sys

import time

sys.path.append('C:/new_tdx_test2025/PYPlugins/user')

from tqcenter import tq

tq.initialize('定时器实时预警Handlebar效果.py')

# 获取A股全部股票（测试时限制数量）

all_stocks = tq.get_stock_list(market='5')[:150]

def get_real_time_data(stock_code):

    """

    获取股票的实时行情数据

    根据通达信TQ接口文档，这里需要调用相应的数据获取函数

    """

    try:

        # 获取最近两天的数据，用于获取前一日收盘价

        market_data = tq.get_market_data(

            field_list=['Close', 'Volume'],

            stock_list=[stock_code],

            count=2,  # 获取2天数据，用于获取前一日收盘价

            period='1d',

            dividend_type='none',

            fill_data=True

        )

        if market_data and 'Close' in market_data:

            close_df = market_data['Close']

            if not close_df.empty:

                # 获取最新收盘价

                last_price = close_df.iloc[-1][stock_code]

                # 获取前一日收盘价
```

```
                if len(close_df) >= 2:

                    prev_close = close_df.iloc[-2][stock_code]

                else:

                    prev_close = '0.00'

                # 获取成交量

                if 'Volume' in market_data:

                    volume_df = market_data['Volume']

                    volume = volume_df.iloc[-1][stock_code] if not
volume_df.empty else '0'

                else:

                    volume = '0'

                return str(last_price), str(prev_close), str(volume)

    except Exception as e:

        print(f"获取{stock_code}实时数据失败: {e}")

    return '0.00', '0.00', '0'

def run_upn_selection():

    """

    执行UPN公式选股并发送预警

    这个函数将每分钟执行一次

    """

    print(f"\n{'='*50}")

    print(f"执行时间: {datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")

    print(f"{'='*50}")

    upn_stocks = []  # 用于存放符合UPN公式选股条件的股票列表

    for stock in all_stocks:

        formula_set_res = tq.formula_set_data_info(

            stock_code=stock,

            stock_period='1d',

```

```
            count=5,

            dividend_type=1

        )

        if formula_set_res:

            # 使用UPN公式选股，参数'3'表示3日上涨

            formula_xg = tq.formula_xg(formula_name='UPN', formula_arg='3')

            if formula_xg and 'Data' in formula_xg and 'UP3' in
formula_xg['Data']:

                up3_data = formula_xg['Data']['UP3']

                if up3_data and len(up3_data) > 0 and up3_data[-1] is not None:

                    if up3_data[-1] != '0':

                        upn_stocks.append(stock)

    print("符合UPN公式选股条件的股票列表: ")

    print(upn_stocks)

    print("符合UPN公式选股条件的股票数量: ", len(upn_stocks))

    # 为选出的股票发送预警

    if upn_stocks:

        send_warnings_for_stocks(upn_stocks)

    else:

        print("本次选股未发现符合条件的股票")

def send_warnings_for_stocks(stock_list):

    """为股票列表发送预警信息"""

    if not stock_list:

        return

    # 获取当前时间，格式化为YYYYMMDDHHMMSS

    current_time = datetime.datetime.now().strftime("%Y%m%d%H%M%S")

    # 准备预警参数列表

    stock_count = len(stock_list)

    # 初始化列表
```

```
178
179     time_list = []
180
181     price_list = []
182
183     close_list = []
184
185     volum_list = []
186
187     # 为每只股票获取实时数据
188
189     for stock in stock_list:
190
191         last_price, prev_close, volume = get_real_time_data(stock)
192
193         time_list.append(current_time)
194
195         price_list.append(last_price)
196
197         close_list.append(prev_close)
198
199         volum_list.append(volume)
200
201     # 其他固定参数
202
203     reason_list = ["3天连涨"] * stock_count  # 根据实际选股条件修改
204
205     bs_flag_list = ['0'] * stock_count  # 买卖标志：0
206
207     warn_type_list = ['0'] * stock_count  # 预警类型：0
208
209     # 调用send_warn函数发送预警
210
211     try:
212
213         warn_res = tq.send_warn(
214
215             stock_list=stock_list,
216
217             time_list=time_list,
218
219             price_list=price_list,
220
221             close_list=close_list,
222
223             volum_list=volum_list,
224
225             bs_flag_list=bs_flag_list,
226
227             warn_type_list=warn_type_list,
228
229             reason_list=reason_list,
230
231             count=stock_count
232
233         )
```

```
234
235         print("预警发送结果: ", warn_res)
236
237         # 根据搜索结果[5](@ref)，预警发送成功后会在TQ策略信号窗口展示
238
239         # 预警图标对应bs_flag_list的每个元素的整数值，0买为红色B，1卖为绿色S，2未知为
240 黄色双叠三角形
241
242         # 双击买卖预警信号记录可以直接打开闪电下单进行买卖操作
243
244     except Exception as e:
245
246         print(f"发送预警失败: {e}")
247
248 # 主循环: 每分钟执行一次选股
249
250 def main_loop():
251
252     """
253
254     主循环函数，每分钟执行一次选股
255
256     使用time.sleep()实现定时执行[6](@ref)
257
258     """
259
260     print("UPN选股预警系统启动")
261
262     print(f"开始时间: {datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
263
264     print(f"监控股票数量: {len(all_stocks)}")
265
266     print("="*50)
267
268     execution_count = 0
269
270     try:
271
272         while True:
273
274             execution_count += 1
275
276             print(f"\n第{execution_count}次执行选股...")
277
278             # 执行选股逻辑
279
280             run_upn_selection()
281
282             # 计算下次执行时间
283
284             next_time = datetime.datetime.now() + datetime.timedelta(minutes=1)
285
286             print(f"下次执行时间: {next_time.strftime('%Y-%m-%d %H:%M:%S')}")
287
288             # 等待60秒[6](@ref)
289
```

```
            time.sleep(60)

    except KeyboardInterrupt:

        print("\n程序被用户中断")

    except Exception as e:

        print(f"程序运行出错: {e}")

    finally:

        print("UPN选股预警系统已停止")
# 启动主循环

if __name__ == "__main__":

main_loop()
```

利用UPN公式筛选出近期连涨股.py

用途：批量处理版本，使用UPN公式筛选3日连涨股票

```py
from tqcenter import tq

import time

'''

    利用此示例需要先在客户端下载全A股盘后数据，不然结果不准确

'''

tq.initialize(__file__)

#先获取A股全部股票

all_stocks = tq.get_stock_list(market='5')

print("正在处理，请等待...")

upn_stocks = []

mul_xg_result = tq.formula_process_mul_xg(

    formula_name='UPN',

    formula_arg='3',

    return_count=1,

    return_date=False,

    stock_list=all_stocks,
```

```
    stock_period='1d',

    count=5,

    dividend_type=1)

# print(mul_xg_result)

if mul_xg_result:

    for key in mul_xg_result:

        if key != "ErrorId":

            if mul_xg_result[key]['UP3'] and mul_xg_result[key]['UP3'][-1] ==
'1':

                upn_stocks.append(key)

print("符合UPN公式选股条件的股票列表：")

print(upn_stocks)

print("符合UPN公式选股条件的股票数量：", len(upn_stocks))

zxg_result = tq.send_user_block(block_code='', stocks=upn_stocks)
```

← 20260122公众号文章

常见问题 →

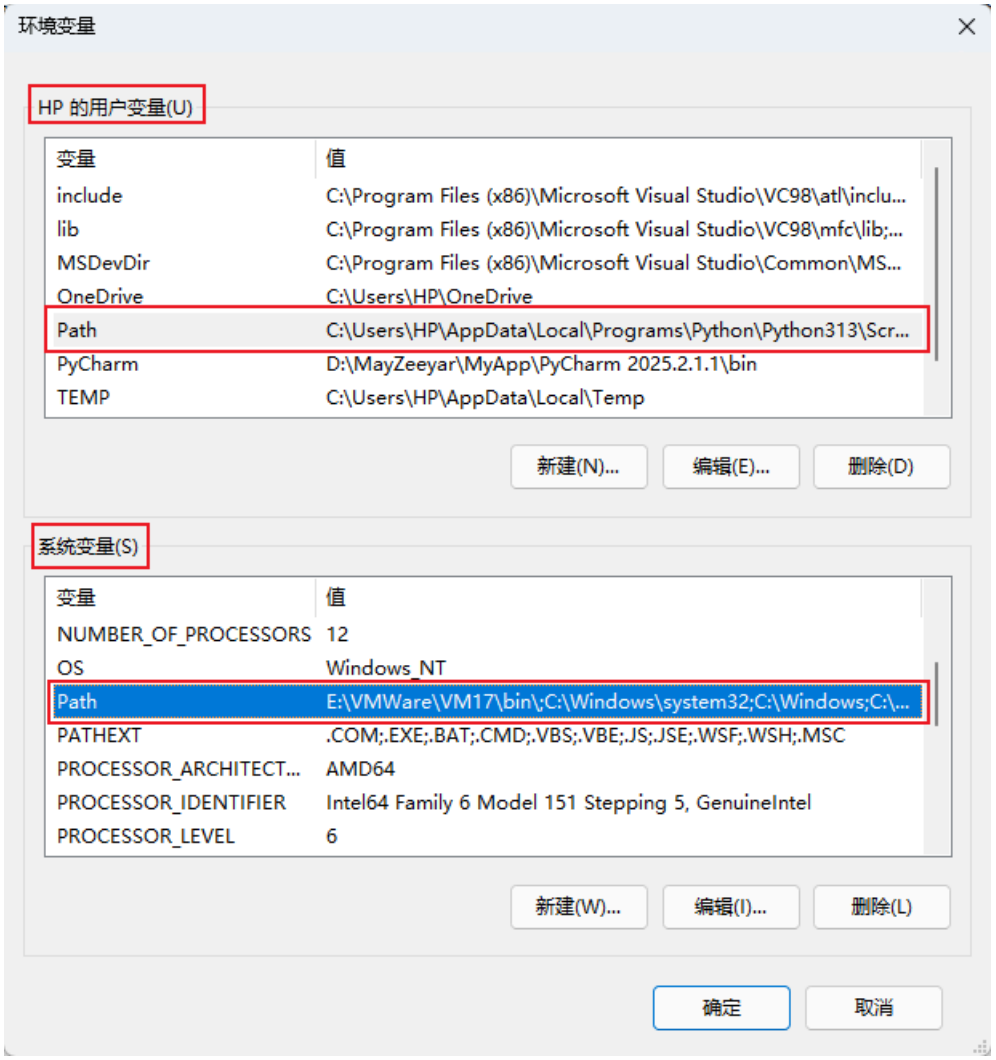
Q：运行的python文件可不可以随便放，不一定在PYPlugins\user目录下？

A： 可以。在import tqcenter前添加通达信安装目录\PYPlugins\user这个绝对路径。

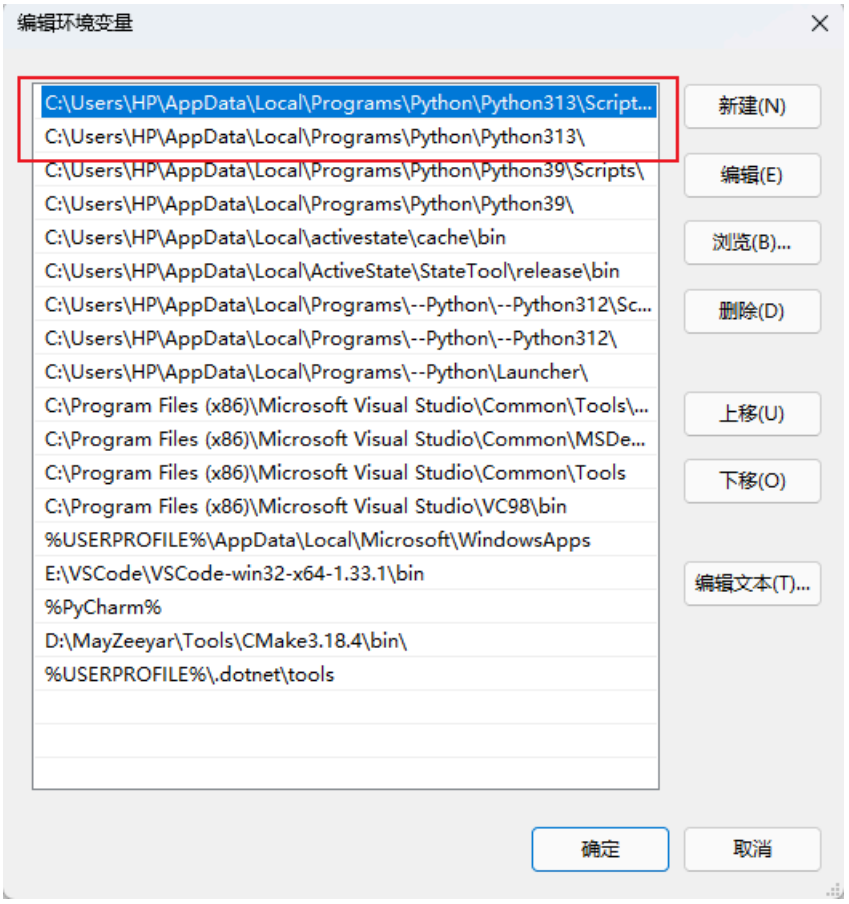
```py
import sys
sys.path.append('C:/new_tdx64/PYPlugins/user')
from tqcenter import tq
tq.initialize(__file__)
```

Q：无法内部执行策略之如何把python路径添加到PATH中

A： 内部执行python策略时，会寻找用户设定的默认python解释器执行python策略，所以必须在操作系统<高级系统设置>--->环境变量设置里，配置python路径。



如图所示，环境变量中分为用户变量和系统变量，都有PATH，在这两个中添加python路径都可生效，但是用户变量的优先级高于系统变量，所以图中仅在用户变量中的PATH中添加python路径。



图中可见，PATH中可以配置多个版本的python，但是最后生效为最上面的，每个版本的python需要配置两个路径。

Q：出现类似以下的报错怎么办？

```py
FileNotFoundError: Could not find module 'F:\tdx\new_tdx_600\PYPlugins\TPythClient.dll' (or one of its dependencies). Try using the full path with constructor syntax.
```

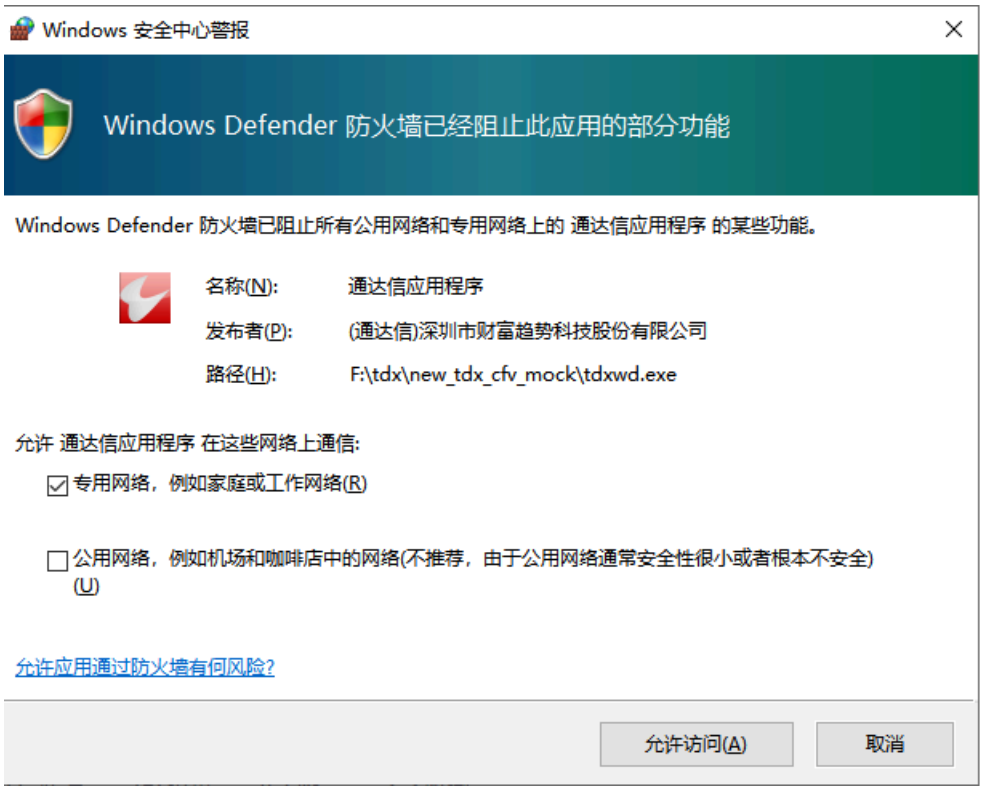
A： 这通常是TPythClient.dll缺少依赖库导致的，请检查TPythClient.dll同目录下（../PYPlugins/）是否有tdxrpcx64.dll，通常是杀毒软件误杀此dll导致，需要重装或给予白名单确保tdxrpcx64.dll不会被杀毒软件误杀。

Q：外部运行的py文件报已经存在运行的，怎么处理？

A： 请在TQ策略管理器找到这个正在运行的已经运行出错的OutSide策略，点删除策略删除它。

Q：菜单一直显示“正在开启TQ策略..”

A： 是否有以下这个提示？如果有，请允许访问。



Q：获取的数据count=5，返回的指标值怎么前面的是none?

```
'ZF': [None, '-0.18', '1.20', '0.64'],
```

A： formula_set_res = tq.formula_set_data_info(stock_code=stock,stock_period='1d', count=4,dividend_type=1)这里的count=4 是获取最近4根k线的数据用于计算指标，所以最近4根k的数据

ZF:(C-REF(C,1))/REF(C,1)*100;这个式子的只能计算出 最后4根k的涨幅值。

所以在获取指标值时注意获取k线数目要覆盖到最大参数值，否则计算结果会为空。

Q：为什么同一个选股公式，用formula_process_mul_xg选股的结果比客户端条件选股中得到的结果少?

A： 请确认formula_process_mul_xg中的count参数是否合理？数据个数要满足公式计算中的数据要求。客户端的条件选股中使用了所有的本地数据。

Q：如何选出分钟内主力净额排名靠前的股票?

A： 可以用一定时间间隔获取主力净额输出值，然后用这次值减上次值的差额排序筛选全市场找出来。

{ZLJE 自定义指标}

超B:=L2_AMO(0,0)/10000.0;

大B:=L2_AMO(1,0)/10000.0;

中B:=L2_AMO(2,0)/10000.0;

小B:=L2_AMO(3,0)/10000.0;

超S:=L2_AMO(0,1)/10000.0;

大S:=L2_AMO(1,1)/10000.0;

中S:=L2_AMO(2,1)/10000.0;
小S:=L2_AMO(3,1)/10000.0;
主力净额:(超B+大B)-(超S+大S),NODRAW;

> 实现示例完整代码

```py
import sys
import time

sys.path.append('C:/new_tdx_test2025/PYPlugins/user')
from tqcenter import tq

tq.initialize('0303zlje.py')

# 先获取A股全部股票
all_stocks = tq.get_stock_list(market='5')[:100]
# all_stocks=['300911.SZ', '600635.SH', '000890.SZ', '603155.SH', '301448.SZ',
'600010.SH', '600011.SH', '600012.SH', '600013.SH', '600014.SH']
print("正在处理，请等待...")
start_date = '20240601'
end_date = '20240630'

# 开始计时
start_time = time.time()

macd_stocks = []
pre_mul_zb_result = {}
mul_zb_result = {}
curr_val = 0
countjs = 1
pre_val=0
ce_val=0
# 添加最大循环次数限制，防止无限循环
max_iterations = 10  # 设置最大迭代次数

while countjs <= max_iterations:
    # 保存之前的值
    pre_mul_zb_result = mul_zb_result.copy()  # 使用copy()避免引用问题

    # 获取新的值
    mul_zb_result = tq.formula_process_mul_zb(
        formula_name='ZLJE',
        formula_arg='',
        xsflag=6,
        return_count=2,
        return_date=True,
        stock_list=all_stocks,
        stock_period='1d',
        count=-1,
        start_time=start_date,
        end_time=end_date,
        dividend_type=1
    )

```

```
    print("当前结果:", mul_zb_result)
    print("前一结果:", pre_mul_zb_result)

    countjs += 1

    # 检查是否有有效的数据
    if mul_zb_result and countjs >= 2:  # 至少需要两次才能比较
        diff_list = []
        for key in mul_zb_result:
            if key != "ErrorId":
                # 安全检查
                if (key in mul_zb_result and
                    '主力净额' in mul_zb_result[key] and
                    len(mul_zb_result[key]['主力净额']) >= 1 and
                    key in pre_mul_zb_result and
                    '主力净额' in pre_mul_zb_result[key] and
                    len(pre_mul_zb_result[key]['主力净额']) >= 1):

                    curr_val = mul_zb_result[key]['主力净额'][-1]['Value']
                    pre_val = pre_mul_zb_result[key]['主力净额'][-1]['Value']
                    ce_val = float(curr_val) - float(pre_val)
                    diff_list.append((key, ce_val))

                    print(f"股票 {key}: 当前值={curr_val}, 前值={pre_val}, 差值=
{ce_val}")
        # 按差值从大到小排序，输出前5名
        if diff_list:
            diff_list.sort(key=lambda x: x[1], reverse=True)
            print("主力净额变化前5名:")
            for i, (code, diff) in enumerate(diff_list[:5], 1):
                print(f"{i}. {code}: {diff:.2f}")
        else:
            print("无有效差值数据")


    # 等待一段时间再下一次循环
    time.sleep(180)

print("处理完成")
```

Q：tq策略弹框，这个双击之后默认弹出下单窗口，能不能设置为不弹出窗口，双击只是显示该票的K线或者分时！？

A： 可以

send_warn入参bs_flag_list N List[str] 买卖标志：0买1卖2未知 其中2就是普通预警，双击打开个股界

面

Q：TDXQUANT个别日期的OHLC和通达信金融终端64位客户端数据不一致?

```py
#tdxmytest.py
from tqcenter import tq
tq.initialize(__file__)
df = tq.get_market_data(
field_list=[],
stock_list=['000656.SZ'],
start_time='20020402',
end_time='20020402',
count=1,
dividend_type='front',
period='1d',
fill_data=True
)
print(df)
```

运行输出

PS D:\projects\PlatformIOProjects\mutouren123> &
C:/Users/PC/AppData/Local/Python/pythoncore-3.14-64/python.exe
d:/stock/new_tdx64/PYPlugins/user/tdxmytest.py
TQ数据接口初始化成功，使用路径: d:\stock\new_tdx64\PYPlugins\user\tdxmytest.py
{'High': 000656.SZ
2002-04-02 9.5, 'Amount': 000656.SZ
2002-04-02 5608.31, 'Close': 000656.SZ
2002-04-02 9.21, 'ForwardFactor': 000656.SZ
2002-04-02 0.0, 'Open': 000656.SZ
2002-04-02 9.45, 'Low': 000656.SZ
2002-04-02 8.91, 'Volume': 000656.SZ
2002-04-02 6110600.0}
TQ数据连接已关闭

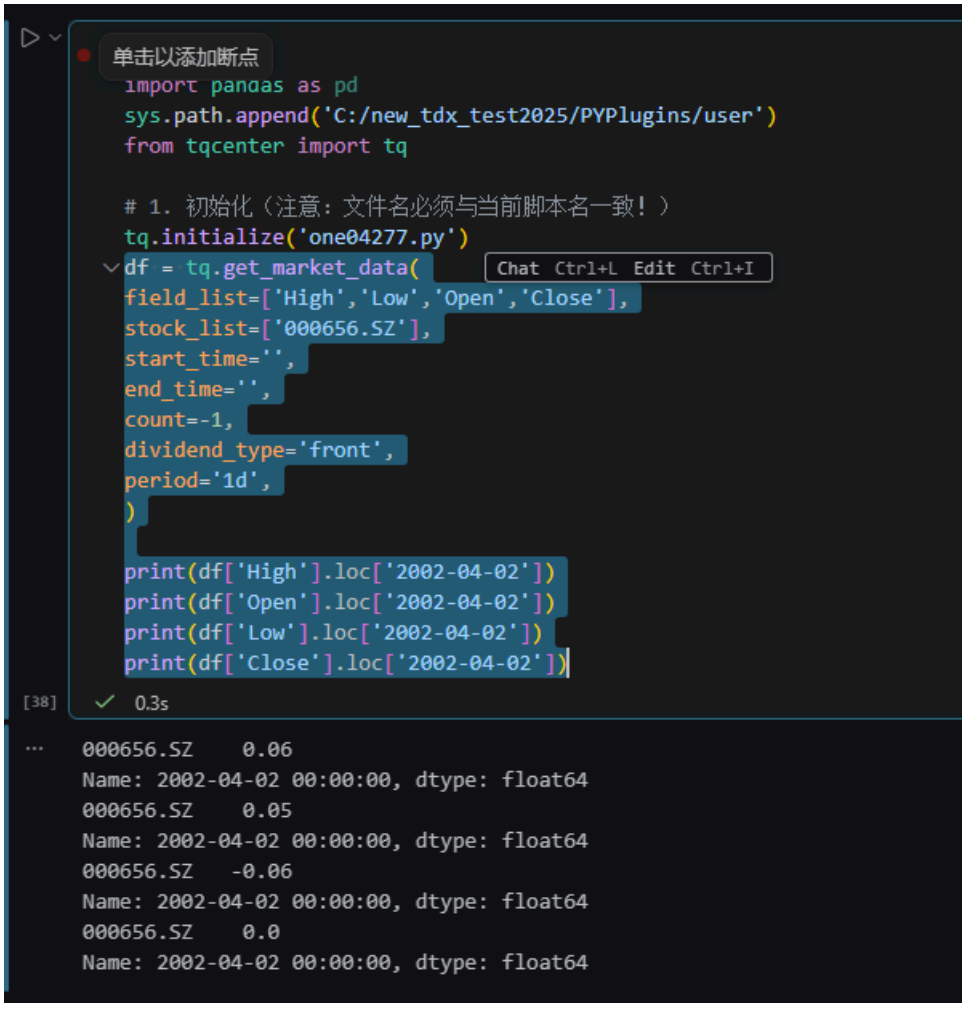
客户端日线前复权显示

日期(OHLC)
20020402(0.05,0.06,-0.06,0.00)

A：你要获取到全部k线信息下的 某日前复权数据 必须取全部数据

```py
df = tq.get_market_data(
field_list=['High','Low','Open','Close'],
stock_list=['000656.SZ'],
start_time='',
end_time='',
count=-1,
dividend_type='front',
period='1d',
)

print(df['High'].loc['2002-04-02'])
print(df['Open'].loc['2002-04-02'])
print(df['Low'].loc['2002-04-02'])
print(df['Close'].loc['2002-04-02'])
```

取全数据信息后(count=-1)才能和你客户端全部k线加载下的前复权数据对比



Q：TQ打开个股详情页的功能调用怎么写？

A：下面是两种进入个股打开个股界面的方式

```py
from tqcenter import tq
tq.initialize(__file__)
```

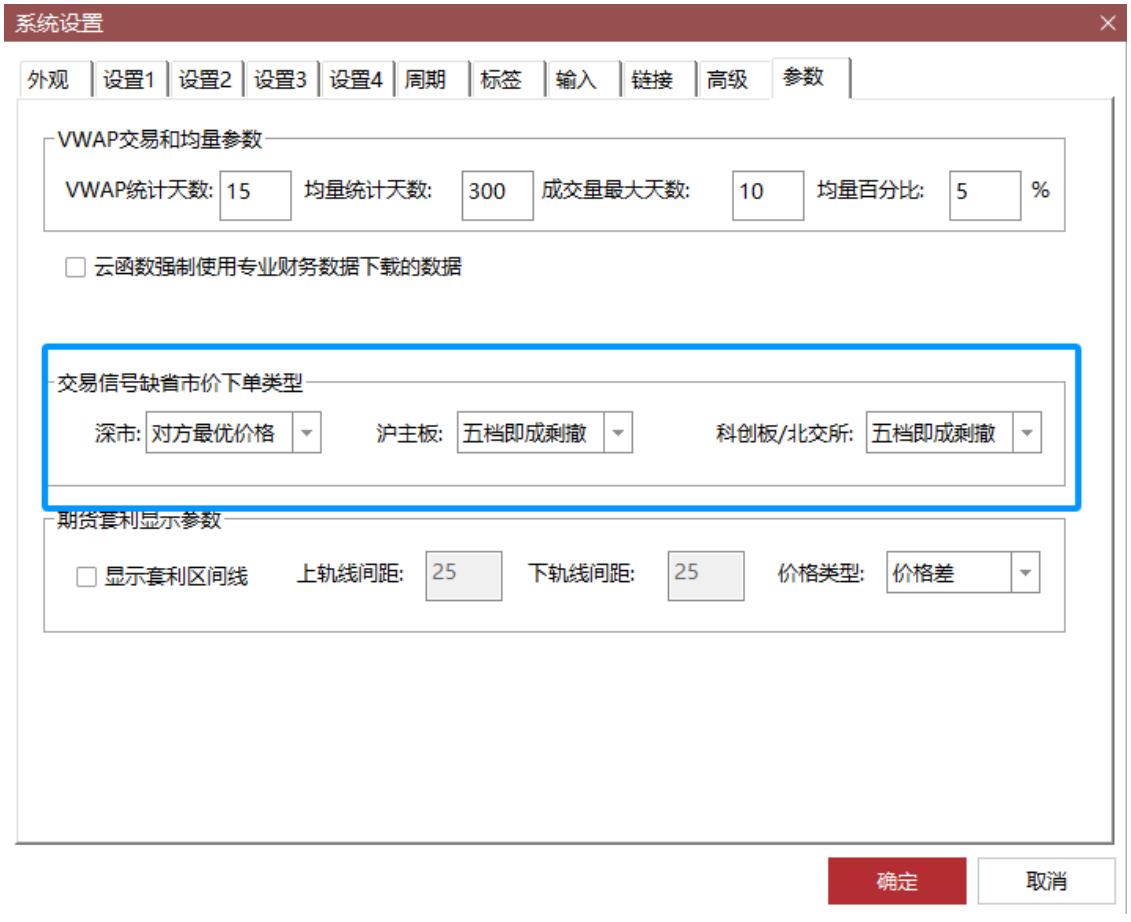
```
# exec_res1 = tq.exec_to_tdx(url='http://www.treeid/breed_1#688318')

# print(exec_res1)

exec_res2 =tq.exec_to_tdx(url='http://www.treeid/code_688318')

print(exec_res2)
```

Q：为什么市价下单和涨停下单，交易信号弹窗里面的委托看着像反了？

A：

市价下单，需要手动确认为了怎么都能成功直接取了涨停价为弹窗委托价，实际真正下单选择了市价委托方式。采取的是图5市价相关的设置。

涨停价下单，大概率为了买到股票，但是限价单本身是受笼子价格限制的品种，你用涨停价委托就废单，所以委托价采取了图4地方涨跌停价下单相关设置。



← 20260302公众号文章